

Lucas Palmiro de Freitas

Projeto CoCAD
Sistema Generativo Multimodal de CAD
Paramétrico

São Paulo, SP

2025

Lucas Palmiro de Freitas

Projeto CoCAD
Sistema Generativo Multimodal de CAD Paramétrico

Trabalho de conclusão de curso apresentado
ao Departamento de Engenharia de Computa-
ção e Sistemas Digitais da Escola Politécnica
da Universidade de São Paulo para obtenção
do Título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Orientador: Prof. Dr. Victor Takashi Hayashi

São Paulo, SP

2025

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio convencional ou eletrônico, para fins de estudo e pesquisa, desde que citada a fonte.

Catálogo-na-publicação

de Freitas, Lucas

Projeto CoCAD - Sistema Generativo Multimodal de CAD Paramétrico /
L. de Freitas -- São Paulo, 2025.

102 p.

Trabalho de Formatura - Escola Politécnica da Universidade de São
Paulo. Departamento de Engenharia de Computação e Sistemas Digitais.

1.INTELIGÊNCIA ARTIFICIAL 2.REDES NEURAIS 3.APRENDIZAGEM
PROFUNDA 4.RECONHECIMENTO DE IMAGEM 5.RECONHECIMENTO DE
TEXTO I.Universidade de São Paulo. Escola Politécnica. Departamento de
Engenharia de Computação e Sistemas Digitais II.t.

“What I cannot create, I do not understand.”

Richard Feynman

Agradecimentos

Agradeço primeiramente a Deus, pela vida, pela força nos momentos de incerteza e pela serenidade necessária para enfrentar cada etapa desta jornada. Sem Sua graça e orientação, este trabalho não teria sido possível.

Este trabalho representa o resultado de uma trajetória construída com o apoio, a orientação e o carinho de muitas pessoas que fizeram parte da minha formação pessoal e acadêmica.

Agradeço aos meus pais, Carlos e Luciana, pelo amor incondicional, pela força nos momentos difíceis e por sempre acreditarem no meu potencial. Aos meus avós, que me ensinaram, cada um à sua maneira, o valor do trabalho, da honestidade e da educação como caminho de transformação.

Agradeço profundamente ao meu orientador, Prof. Victor, pela orientação atenta, pelas discussões técnicas, pela paciência e pela confiança depositada ao longo de todo o desenvolvimento deste projeto. Sua experiência e generosidade foram fundamentais para que este trabalho alcançasse maturidade.

Estendo os agradecimentos aos professores e colegas da Escola Politécnica da USP, da Télécom Paris e do Institut Polytechnique de Paris, cujos conhecimentos, desafios e interações moldaram minha formação e ampliaram minha visão acadêmica e profissional.

Agradeço também às equipes e colegas da Dassault Systèmes e do BTG Pactual, instituições onde tive a oportunidade de aprender, crescer e aplicar parte do que desenvolvo aqui. Em especial, agradeço às equipes que acompanharam meu estágio, pelos diálogos, incentivos e trocas que contribuíram diretamente para este projeto.

Aos amigos que estiveram presentes durante toda essa jornada — seja oferecendo ajuda técnica, conselhos, revisões, companhia ou simplesmente apoio emocional — meu sincero agradecimento. Cada um contribuiu de forma única para que esse trabalho fosse possível.

Por fim, sou grato a todos que, direta ou indiretamente, fizeram parte desse caminho e ajudaram a transformar este projeto em realidade.

Resumo

Este trabalho apresenta o desenvolvimento do CoCAD, um sistema multimodal para geração de modelos CAD paramétricos a partir de entradas textuais e visuais, fundamentado em técnicas de inteligência artificial generativa. O objetivo principal foi investigar a viabilidade de integrar representações sequenciais de CAD, descrições em linguagem natural e informações visuais em um pipeline unificado capaz de produzir modelos geométrica e estruturalmente consistentes. A metodologia foi organizada em três módulos centrais: um autoencoder treinado para aprender um espaço latente robusto de sequências CAD; um módulo de condicionamento multimodal que extrai e combina representações de imagem e texto; e um gerador latente condicionado baseado em *flow matching*. Também foi desenvolvida uma interface interativa que integra todo o pipeline, permitindo gerar, visualizar e exportar sólidos CAD. Os resultados obtidos indicam que o autoencoder alcança elevado desempenho de reconstrução simbólica e geométrica, enquanto o gerador latente aproxima-se parcialmente do espaço latente real, apresentando melhor desempenho no modo multimodal. As sequências CAD geradas mostraram-se plausíveis e coerentes, embora ainda inferiores à fidelidade obtida pelo autoencoder, evidenciando limitações relacionadas à modelagem do espaço latente e à qualidade das descrições textuais disponíveis. Conclui-se que o CoCAD demonstra a viabilidade da geração multimodal de CAD paramétrico e estabelece fundamentos promissores para avanços futuros em modelos generativos estruturados, condicionamento texto-visual e integração com fluxos de engenharia digital.

Palavras-chave: IA generativa, CAD, multimodalidade, aprendizado profundo, flow matching, autoencoder.

Abstract

This work presents the development of CoCAD, a multimodal system for generating parametric CAD models from textual and visual inputs, grounded in contemporary generative artificial intelligence techniques. The main objective was to investigate the feasibility of integrating sequential CAD representations, natural language descriptions, and visual information into a unified pipeline capable of producing geometrically and structurally consistent models. The methodology was organized into three core modules: an autoencoder trained to learn a robust latent space of CAD sequences; a multimodal conditioning module that extracts and fuses image and text representations; and a conditional latent generator based on flow matching. An interactive interface was also developed to integrate the entire pipeline, enabling the generation, visualization, and export of CAD solids. The results indicate that the autoencoder achieves strong symbolic and geometric reconstruction performance, while the latent generator partially approximates the real latent space, showing its best results in the multimodal setting. The generated CAD sequences were plausible and coherent, although still below the fidelity reached by the autoencoder, revealing limitations related to latent space modeling and to the quality of the textual descriptions in the dataset. It is concluded that CoCAD demonstrates the feasibility of multimodal parametric CAD generation and establishes a solid foundation for future advances in structured generative models, text-visual conditioning, and integration with digital engineering workflows.

Keywords: generative AI, CAD, multimodality, deep learning, flow matching, autoencoder.

Lista de ilustrações

Figura 1 – Exemplo de modelo CAD paramétrico feito na ferramenta SolidWorks (Dassault Systèmes, 2025b).	13
Figura 2 – Representações 3D: (a) <i>mesh</i> , (b) <i>voxel</i> , (c) <i>point cloud</i> , (d) <i>B-Rep</i> e (e) sequência de comandos CAD paramétricos.	18
Figura 3 – Ilustração de um processo de construção CAD seguindo o paradigma <i>sketch-extrude</i>	20
Figura 4 – Representação de um modelo CAD como sequência de comandos <i>sketch-extrude</i> no DeepCAD.	23
Figura 5 – Arquitetura DeepCAD: <i>Transformer Autoencoder</i> com <i>latent GAN</i> para geração CAD.	23
Figura 6 – Representação hierárquica do SkexGen com múltiplas sequências. . . .	24
Figura 7 – Arquitetura do SkexGen: VQ-VAE com múltiplos <i>codebooks</i> e geração autoregressiva.	24
Figura 8 – Representação hierárquica do HNC-CAD com estrutura em árvore. . .	25
Figura 9 – Arquitetura do HNC-CAD: VQ-VAE hierárquico com múltiplos <i>codebooks</i> e geração autoregressiva.	25
Figura 10 – Arquitetura do GenCAD: pipeline de quatro etapas para geração de programas CAD condicionados por imagens.	26
Figura 11 – <i>Pipeline</i> CADCrafter: codificador de geometria, difusão prévia e decodificação otimizada por preferência permitem a geração de programas CAD a partir de imagens do mundo real.	27
Figura 12 – <i>Pipeline</i> de anotação automática do Text2CAD, utilizando modelos de linguagem e multimodais para gerar descrições textuais em múltiplos níveis de detalhe a partir de modelos CAD.	28
Figura 13 – Arquitetura do Text2CAD: Transformer autoregressivo mapeia descrições textuais em sequências CAD paramétricas.	29
Figura 14 – As cinco visões do RM-ODP original.	35
Figura 15 – Transformação das visões originais do RM-ODP na versão adaptada utilizada neste projeto.	37
Figura 16 – Relação entre as visões da versão adaptada do RM-ODP.	38
Figura 17 – Diagrama de casos de uso do sistema de geração de modelos CAD. . .	44
Figura 18 – Diagrama de Componentes do Sistema CoCAD.	51
Figura 19 – Diagrama de sequência para geração de modelo CAD a partir de imagem. .	55
Figura 20 – Diagrama de sequência para geração de modelo CAD a partir de texto. .	56
Figura 21 – Diagrama de sequência para geração multimodal de modelos CAD. . .	56

Figura 22 – Visão geral dos três módulos de aprendizado de máquina do CoCAD: (1) Autoencoder de Sequências CAD, (2) Extração e Fusão Multimodal, (3) Gerador Condicional de Latentes por Flow Matching.	62
Figura 23 – Fluxo completo de inferência do CoCAD, desde o processamento multi- modal até a geração final do sólido CAD.	76
Figura 24 – Interface interativa do CoCAD implementada em Gradio.	77
Figura 25 – Interface de geração CAD do CoCAD durante um teste funcional utili- zando uma amostra do <i>split</i> de teste.	87

Lista de tabelas

Tabela 1 – Comparação dos principais trabalhos relacionados	16
Tabela 2 – Requisitos Funcionais do sistema CoCAD	48
Tabela 3 – Requisitos Não Funcionais do sistema CoCAD	49
Tabela 4 – Tamanhos das divisões do conjunto Text2CAD antes e após o filtro . .	64
Tabela 5 – Resultados de reconstrução CAD do autoencoder do CoCAD.	81
Tabela 6 – Comparação da reconstrução CAD com métodos da literatura.	81
Tabela 7 – Similaridade entre latentes reais e previstos pelo flow matching.	82
Tabela 8 – Comparação da geração CAD com CoCAD, CADCrafter e Text2CAD.	84
Tabela 9 – Síntese de atendimento aos requisitos funcionais do CoCAD.	92
Tabela 10 – Síntese de atendimento aos requisitos não funcionais do CoCAD.	94

Sumário

1	INTRODUÇÃO	13
1.1	Motivação	13
1.2	Objetivo	14
1.3	Justificativa	15
1.4	Organização do Trabalho	17
2	ASPECTOS CONCEITUAIS	18
2.1	Representações 3D e escolha da representação CAD	18
2.1.1	Limitações de representações geométricas tradicionais	18
2.1.2	Sequências de comandos CAD (<i>sketch-extrude</i>)	19
2.1.3	O paradigma <i>sketch-extrude</i>	20
2.1.4	Representação adotada neste trabalho	21
2.2	Trabalhos relacionados em geração de CAD paramétrico	22
2.2.1	DeepCAD (2021)	22
2.2.2	SkexGen (2022)	23
2.2.3	HNC-CAD (2023)	25
2.2.4	GenCAD (2025)	26
2.2.5	CADCrafter (2025)	27
2.2.6	Text2CAD (2024)	28
2.2.7	Síntese e justificativa da escolha	29
2.3	Modelos fundamentais de visão, linguagem e geração	30
2.3.1	Arquiteturas Transformer e modelos GPT	30
2.3.2	Modelo de visão DINOv2	30
2.3.3	Modelo de linguagem BERT	31
2.3.4	Modelos generativos baseados em <i>flow matching</i>	31
2.3.5	Blocos residuais com condicionamento FiLM	33
2.4	Síntese	34
3	MÉTODO DO TRABALHO	35
3.1	O Modelo RM-ODP Original	35
3.2	Versão Adaptada do RM-ODP para o Projeto CoCAD	36
3.2.1	Relação entre as Visões na Versão Adaptada	38
3.3	Fases de Desenvolvimento do Projeto	38
4	ESPECIFICAÇÃO TÉCNICA	41
4.1	Visão Empresarial	41

4.2	Visão de Requisitos	42
4.2.1	Histórias de Usuário	42
4.2.1.1	História de Usuário 1 — Geração de modelo CAD a partir de imagem	42
4.2.1.2	História de Usuário 2 — Geração de modelo CAD a partir de texto	43
4.2.1.3	História de Usuário 3 — Geração de modelo CAD multimodal (imagem + texto)	43
4.2.2	Casos de Uso	44
4.2.2.1	Caso de Uso 1 – Geração de Modelo CAD a partir de Imagem	44
4.2.2.2	Caso de Uso 2 – Geração de Modelo CAD a partir de Texto	46
4.2.2.3	Caso de Uso 3 – Geração de Modelo CAD Multimodal (Imagem + Texto)	47
4.2.3	Requisitos Funcionais	48
4.2.4	Requisitos Não Funcionais	49
5	DESENVOLVIMENTO DO TRABALHO	50
5.1	Visão de Arquitetura	50
5.1.1	Modelagem Estática	51
5.1.2	Modelagem Dinâmica	54
5.2	Visão Tecnológica	57
5.2.1	PyTorch e PyTorch Lightning	57
5.2.2	Hydra para Gerenciamento de Configurações	58
5.2.3	Modelos Pré-treinados: DINOv2 e BERT	58
5.2.4	Ambiente de Treinamento: Google Colab	59
5.2.5	Hugging Face para Acesso a Dados e Modelos	59
5.2.6	Ferramentas de Visualização e Gerenciamento de Treinamento	59
5.2.7	OCP / CadQuery para Reconstrução e Exportação CAD	60
5.2.8	Gradio para Interface Interativa	60
5.2.9	Linguagem Python e Bibliotecas de Apoio	60
5.3	Projeto e Implementação	61
5.3.1	Visão Geral da Arquitetura de Aprendizado de Máquina	61
5.3.2	Conjunto de Dados Utilizado	63
5.3.2.1	Modelos CAD do DeepCAD	63
5.3.2.2	Imagens e Textos do Text2CAD	63
5.3.3	Modelos e Arquiteturas dos Módulos de Aprendizado de Máquina	64
5.3.3.1	Módulo 1: Autoencoder de Sequências CAD	64
5.3.3.2	Módulo 2: Extração e Fusão Multimodal	66
5.3.3.3	Módulo 3: Gerador Condicional de Latentes por Flow Matching	68
5.3.4	Fluxo Completo de Treinamento e Teste	70
5.3.5	Configurações e Estratégias de Treinamento dos Modelos	72
5.3.5.1	Treinamento do Autoencoder de Sequências CAD	72
5.3.5.2	Treinamento do Modelo de <i>Latent Flow Matching</i> com Condicionamento Multimodal	73
5.3.5.3	Learning Rate Schedulers	74

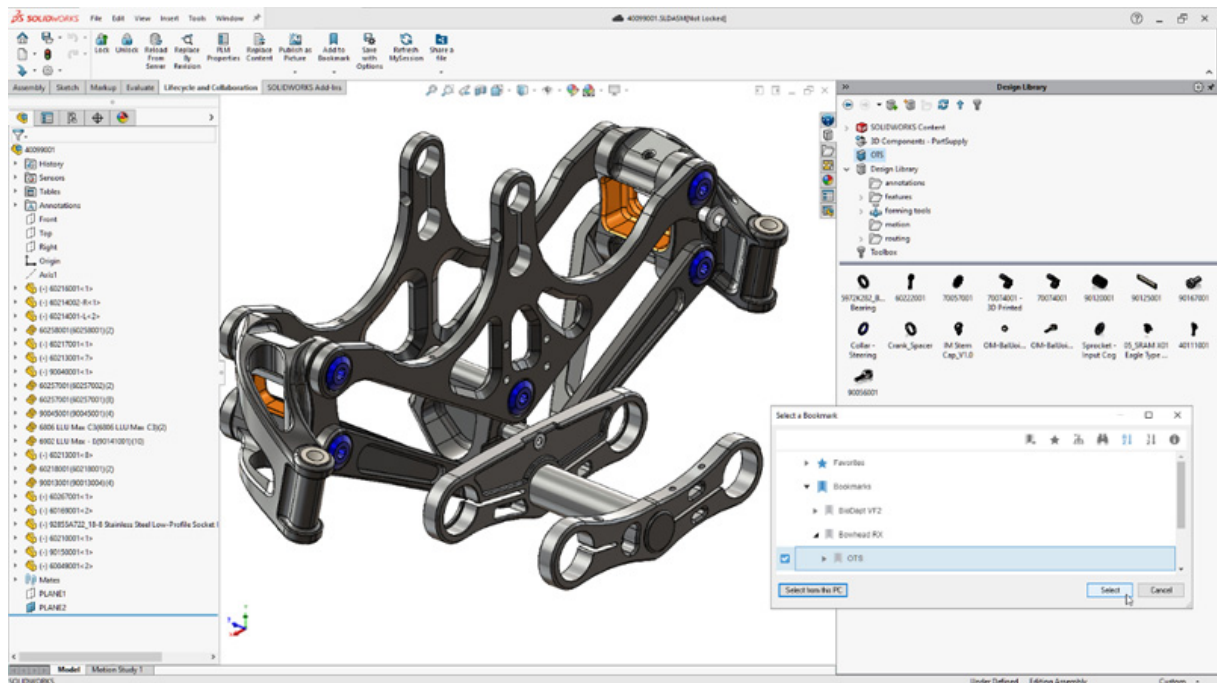
5.3.6	Pipeline de Inferência e Interface Interativa	75
5.4	Testes e Avaliação	77
5.4.1	Métricas Utilizadas	78
5.4.1.1	Command Accuracy	78
5.4.1.2	Parameter Accuracy	79
5.4.1.3	Chamfer Distance (Mediana)	79
5.4.1.4	Invalid Ratio	79
5.4.1.5	MSE e Cosine Similarity entre Latentes	80
5.4.2	Avaliação 1: Reconstrução CAD pelo Autoencoder	80
5.4.3	Avaliação 2: Similaridade entre Latentes Gerados e Latentes Reais	82
5.4.4	Avaliação 3: Geração de Sequências CAD a partir de Latentes Previstos	83
5.4.5	Teste Funcional da Interface de Geração CAD	86
6	CONSIDERAÇÕES FINAIS	89
6.1	Conclusões do Projeto de Formatura	89
6.2	Síntese de Atendimento aos Requisitos	91
6.3	Contribuições	96
6.4	Perspectivas de Continuidade	96
	REFERÊNCIAS	98

1 Introdução

1.1 Motivação

O *Computer-Aided Design* (CAD) desempenha um papel central no ciclo de desenvolvimento de produtos modernos em setores como aeroespacial, automotiva, arquitetura e manufatura avançada. Modelos CAD paramétricos constituem a base para operações posteriores de simulação, prototipagem, análise e fabricação, desempenhando papel essencial na precisão geométrica e na rastreabilidade do processo de engenharia, como ilustrado na Figura 1. Apesar da evolução das ferramentas de CAD ao longo das últimas décadas, a construção de modelos permanece uma atividade majoritariamente manual, exigindo conhecimento especializado e envolvendo operações repetitivas, tais como criação de esboços, extrusões, restrições geométricas e etapas sucessivas de refinamento.

Figura 1 – Exemplo de modelo CAD paramétrico feito na ferramenta SolidWorks ([Dassault Systèmes, 2025b](#)).



Fonte: ([VISIATIV, 2024](#))

Esse caráter intensivo e manual do processo de modelagem abre espaço para abordagens baseadas em inteligência artificial (IA) capazes de otimizar e acelerar o fluxo de trabalho. Os recentes avanços em modelos generativos baseados em aprendizado profundo — especialmente arquiteturas baseadas em *Transformer*, como os modelos GPT (*Generative Pre-trained Transformer*) ([RADFORD et al., 2018](#); [BROWN et al., 2020](#)),

capazes de gerar texto de forma coerente e contextual, e modelos de difusão empregados para geração de imagens, como DALL · E 2 (RAMESH et al., 2022) e Stable Diffusion (ROMBACH et al., 2022) — vêm demonstrando capacidade crescente de produzir conteúdos estruturados e semanticamente consistentes a partir de instruções textuais ou visuais. Esses desenvolvimentos impulsionaram o surgimento de trabalhos que aproximam IA do domínio do CAD paramétrico, buscando automatizar partes do processo de modelagem e ampliar a acessibilidade dessas ferramentas.

Representações CAD como sequências de operações paramétricas, inauguradas com DeepCAD (WU; XIAO; ZHENG, 2021), abriram caminho para modelos capazes de aprender não apenas a geometria final, mas também o processo de criação. Trabalhos posteriores, como GenCAD (ALAM; AHMED, 2025), CADCrafter (CHEN et al., 2025) e Text2CAD (KHAN et al., 2024b), expandiram essa linha, explorando condicionamentos visuais ou textuais. Ainda assim, cada um deles apresenta limitações importantes, especialmente no que diz respeito à ausência de fusão multimodal e à dificuldade de generalizar para cenários mais complexos e próximos da prática industrial.

Nesse contexto, surge a questão que orienta este trabalho:

Como integrar métodos avançados de visão computacional e modelagem de linguagem para gerar modelos CAD paramétricos a partir de entradas intuitivas (imagem e texto), preservando a estrutura editável e o controle geométrico exigidos por aplicações industriais?

1.2 Objetivo

O objetivo geral deste trabalho é desenvolver um **sistema generativo multimodal** capaz de produzir modelos CAD paramétricos a partir de entradas visuais e textuais, de forma integrada, consistente e compatível com fluxos de engenharia. De maneira mais específica, o sistema proposto busca:

- Permitir entradas em diferentes modalidades: imagem, texto ou ambas simultaneamente.
- Gerar sequências de comandos CAD válidas, incluindo entidades geométricas de esboço e operações volumétricas.
- Fornecer uma interface interativa que permita visualizar e exportar o modelo CAD gerado.
- Integrar modelos de visão computacional e modelos de linguagem a um módulo generativo baseado em *flow matching*, utilizado para prever representações latentes CAD consistentes.

Assim, o trabalho compreende tanto uma investigação metodológica quanto a implementação de um sistema funcional denominado *CoCAD*.

1.3 Justificativa

Os avanços recentes em modelos generativos aplicados ao domínio CAD evidenciam uma área em consolidação, mas ainda marcada por lacunas significativas. O trabalho seminal **DeepCAD** (WU; XIAO; ZHENG, 2021) introduziu a representação de modelos CAD como sequências paramétricas — abordagem que possibilitou treinar redes neurais sobre o processo construtivo, e não apenas sobre geometria final. Entretanto, trata-se de um modelo restrito à geração incondicional e limitado a operações *sketch-extrude*, oferecendo pouca flexibilidade quanto à integração com modalidades perceptuais.

Trabalhos subsequentes, como **SkexGen** (XU et al., 2022) e **Hierarchical Neural Coding** ou **HNC-CAD** (XU et al., 2023), investigaram representações hierárquicas e mecanismos de autocompleção, mas continuaram limitados à geração a partir de ruído ou à reconstrução, sem explorar condicionamentos visuais ou textuais.

O primeiro avanço significativo em geração condicionada ocorre com o **GenCAD** (ALAM; AHMED, 2025), que associa imagens a sequências CAD por meio de contrastive learning e modelos de difusão latente. No entanto, seu condicionamento é restrito a uma única vista renderizada da imagem, não explora fusão multimodal, e não incorpora informação textual complementar.

O **CADCrafter** (CHEN et al., 2025) expande essa ideia ao tratar imagens reais, utilizando *geometry encoders*, *multi-view to single-view distillation* e *Direct Preference Optimization* (DPO) para garantir maior validade geométrica. Apesar de robusto, permanece limitado exclusivamente à modalidade visual, sem suporte a descrições textuais. Além disso, a indisponibilidade pública do dataset de avaliação compromete a reprodutibilidade do trabalho.

Por outro lado, o **Text2CAD** (KHAN et al., 2024b) foca no condicionamento textual, oferecendo um extenso dataset com múltiplos níveis de detalhamento descritivo. Porém, sua abordagem não explora entradas visuais e não realiza fusão entre texto e imagem, o que reduz sua aplicabilidade em cenários em que ambas as modalidades são complementares.

O panorama atual é sintetizado na Tabela 1 (mais detalhes sobre cada um dos trabalhos serão apresentados no Capítulo 2).

Diante desse panorama, observa-se que:

- Entre os trabalhos consultados que adotam representações CAD baseadas em sequên-

Tabela 1 – Comparação dos principais trabalhos relacionados

Aspecto	DeepCAD (2021)	SkeXGen (2022)	HNC-CAD (2023)	GenCAD (2025)	CADCrafter (2025)	Text2CAD (2024)
Representação CAD	Seq. paramétrica (sketch-extrude)	Estrutura hierárquica de sketches	Hierarquia loop/profile/solid + seq. CAD	DeepCAD + imagens multiescala	DeepCAD + imagens multivista	Seq. inspirada no DeepCAD + textos
Arquitetura	Transformer AE + I-GAN	Transformer VQ-VAE (3 codebooks)	Hierarchical VQ-VAE + Transformer enc./dec.	Transformer AE + ResNet18 + latent diffusion	Geometry encoder + DINOv2 + Diffusion Transformer + DPO	Transformer auto-regressivo
Tipo de geração	Incondicional	Incondicional + controlável	Incondicional + autocompleção + edição estrutural	Condicionada em imagem	Condicionada em imagem (real e sintética)	Condicionada em texto
Principais Contribuições	- Representação seq. CAD - Grande dataset - Base da linha de pesquisa	- Forte <i>disentanglement</i> - Boa representação de sketches	- Alta expressividade - Suporte à edição hierárquica	- Primeiro modelo cond. por imagem - Contraste imagem-CAD - Difusão latente	- Pipeline robusto - Generaliza a imagens reais - DPO melhora validade	- Grande dataset textual - Múltiplos níveis de detalhe - Bom controle semântico
Limitações	Sem condicionamento	Sem condicionamento	- Sem condicionamento - Custo computacional elevado	- Uma única <i>view</i> - Sem multimodalidade	- Sem multimodalidade - Dataset e código não publicados	- Sem multimodalidade - Arquitetura simples

cias de comandos derivadas do *DeepCAD*, não foi identificada nenhuma abordagem que integre **texto e imagem simultaneamente** como forma de condicionamento para a geração de modelos CAD;

- As abordagens consultadas ainda não exploram técnicas modernas de *flow matching*, um método contínuo de modelagem generativa que aprende campos de velocidade para transformar distribuições de forma estável e eficiente em espaços latentes (LIU; GONG; LIU, 2022; LIPMAN et al., 2023);
- Há uma tendência crescente na indústria de manufatura e engenharia pela adoção de ferramentas baseadas em inteligência artificial generativa, que têm potencial para transformar etapas de concepção e design, automatizando tarefas repetitivas, acelerando ciclos de prototipagem, ampliando a produtividade de equipes técnicas, reduzindo custos e democratizando o acesso ao design paramétrico (GROUP, 2023; COMPANY, 2023; Siemens AG, 2015; AUTODESK, 2023).

Assim, o presente trabalho justifica-se por abordar lacunas evidentes no estado da arte, propondo um sistema generativo multimodal baseado em fusão de visão computacional e linguagem natural, associado a um decodificador paramétrico capaz de reconstruir modelos CAD editáveis. Essa abordagem tem relevância científica e aplicação prática, ampliando o acesso ao design assistido por computador, reduzindo barreiras técnicas e promovendo maior integração entre etapas de concepção e prototipagem.

1.4 Organização do Trabalho

Este trabalho está estruturado em seis capítulos, organizados de forma a conduzir o leitor desde o contexto motivador até os resultados obtidos e as perspectivas futuras do projeto.

- O **Capítulo 1** apresenta o contexto geral do estudo, incluindo a motivação, os objetivos, a justificativa e a estrutura do trabalho.
- O **Capítulo 2** discute os aspectos conceituais necessários para a compreensão do projeto, abordando representações 3D, fundamentos de CAD paramétrico, trabalhos relacionados e modelos fundamentais de visão, linguagem e geração. Esse capítulo estabelece a base teórica que orienta as decisões metodológicas adotadas.
- O **Capítulo 3** descreve o método de trabalho empregado, fundamentado na adaptação do modelo RM-ODP ao contexto do CoCAD. São apresentadas as visões do modelo, sua relação e as fases de desenvolvimento do projeto.
- O **Capítulo 4** detalha a especificação técnica do sistema, incluindo as visões empresarial e de requisitos, as histórias de usuário, os casos de uso, e a definição dos requisitos funcionais e não funcionais que guiam o desenvolvimento.
- O **Capítulo 5** apresenta o desenvolvimento da solução. São descritas a arquitetura implementada, as tecnologias utilizadas, o conjunto de dados, os modelos de aprendizado profundo adotados, as estratégias de treinamento, o pipeline de inferência e a interface interativa, bem como os experimentos conduzidos e os resultados obtidos.
- O **Capítulo 6** reúne as considerações finais, incluindo as conclusões alcançadas, a síntese do atendimento aos requisitos, as contribuições do trabalho e as perspectivas de continuidade para pesquisas e aprimoramentos futuros.

2 Aspectos Conceituais

Este capítulo apresenta os principais conceitos e trabalhos que fundamentam o projeto CoCAD, com ênfase na escolha da representação de modelos CAD como sequências de comandos paramétricos, nos métodos de geração de modelos CAD baseados nessa representação e nos modelos fundamentais de visão, linguagem e geração utilizados no desenvolvimento do sistema.

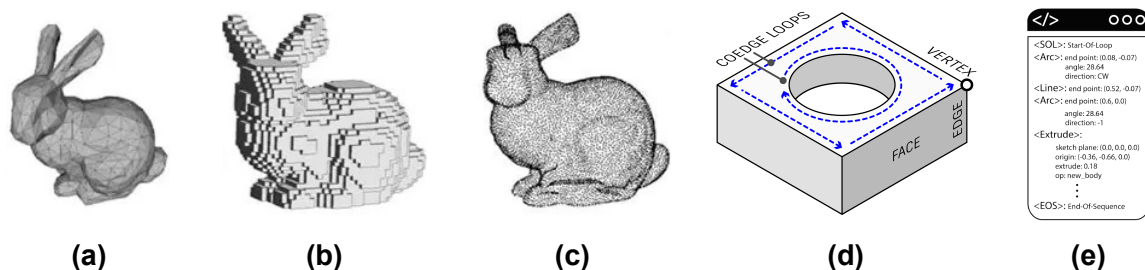
2.1 Representações 3D e escolha da representação CAD

2.1.1 Limitações de representações geométricas tradicionais

Os avanços recentes em modelagem e síntese de formas 3D — incluindo abordagens baseadas em *meshes* (GKIOXARI; MALIK; JOHNSON, 2020), *voxels* (SITZMANN et al., 2019) e *point clouds* (YANG et al., 2019) — impulsionaram significativamente o campo de visão computacional e computação gráfica. Paralelamente, representações paramétricas sólidas, como a *Boundary Representation* (B-Rep), também têm sido estudadas em modelos generativos modernos, como o *BrepGen* (XU et al., 2024). Entretanto, essas representações apresentam limitações importantes quando utilizadas em fluxos de trabalho industriais baseados em CAD paramétrico.

A Figura 2 ilustra essas diferentes formas de representação geométrica; suas principais limitações para uso em fluxos de trabalho industriais baseados em CAD são resumidas a seguir.

Figura 2 – Representações 3D: (a) *mesh*, (b) *voxel*, (c) *point cloud*, (d) *B-Rep* e (e) sequência de comandos CAD paramétricos.



Fonte: Adaptado de (DOHYEON, 2025) (a, b, c), (WILLIS, 2025) (d) e (ALAM; AHMED, 2025) (e).

- **Mesheres:** descrevem superfícies por vértices, arestas e faces. Embora adequadas para visualização, não capturam a lógica de construção paramétrica nem o histórico

do projeto. Pequenas falhas topológicas podem invalidar o modelo para análise ou manufatura.

- **Voxels:** representam a geometria por meio de uma grade regular. Apesar de simples e compatíveis com convoluções, sofrem com limitações severas de resolução, resultando em formas pouco precisas para peças mecânicas.
- **Point clouds:** armazenam apenas pontos amostrados da superfície, sem conectividade explícita ou estrutura paramétrica. A ausência de topologia dificulta a imposição de restrições geométricas e a utilização direta em fluxos CAD.
- **Boundary Representation (B-Rep):** padrão industrial que descreve sólidos por faces, arestas e vértices parametricamente. Apesar da precisão e compacidade, B-Reps são altamente relacionais e suscetíveis a inconsistências; além disso, registram apenas a geometria final, não o processo de criação.

Essas limitações motivam o uso de representações mais alinhadas ao fluxo de construção paramétrico presente em ferramentas CAD modernas, como sequências de comandos (*CAD programs*), que preservam explicitamente o histórico de operações.

2.1.2 Sequências de comandos CAD (*sketch-extrude*)

Para superar as limitações das representações puramente geométricas, trabalhos recentes passam a representar modelos CAD como *sequências de comandos*, também chamadas de programas CAD ou *Sequential Parametric CAD Scripts* (SPECS). Nessa abordagem, cada modelo é descrito como uma sequência ordenada de operações de edição (como esboçar um perfil e extrudi-lo), incluindo seus parâmetros.

Essa representação apresenta diversas vantagens:

- **Captura do processo de projeto:** em vez de descrever apenas a geometria resultante, a sequência codifica a história de construção da peça, alinhada ao fluxo de trabalho de engenharia em sistemas CAD paramétricos.
- **Interpretabilidade e editabilidade:** as sequências podem ser compiladas ou importadas em softwares CAD, permitindo que projetistas humanos inspecionem, modifiquem ou estendam o modelo gerado.
- **Geometria limpa e válida:** como as operações (*sketch*, *extrude*, operações booleanas etc.) são comandos nativos do kernel geométrico, os modelos resultantes tendem a ser sólidos válidos, com superfícies nítidas e adequados para manufatura.

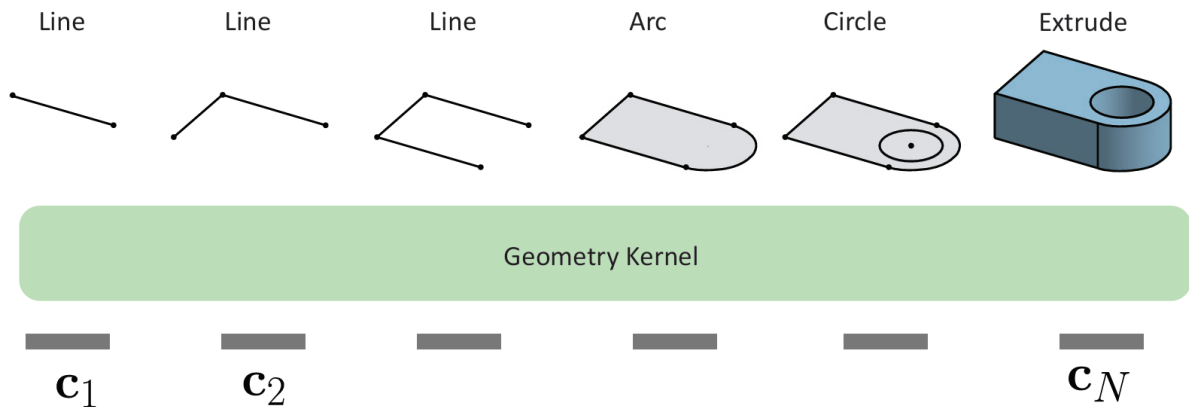
- **Ponte com B-Reps:** as sequências de comandos podem ser compiladas em modelos B-Rep por meio de *geometry kernels*, preservando compatibilidade com sistemas industriais e ferramentas de análise.

Além disso, a estrutura sequencial é análoga à da linguagem natural: sentenças são compostas por tokens ordenados, assim como um modelo CAD pode ser descrito por uma sequência de comandos e parâmetros. Essa analogia torna a representação particularmente adequada para modelos baseados em *Transformers* e outras arquiteturas de aprendizado de sequência.

2.1.3 O paradigma *sketch-extrude*

A maior parte dos datasets públicos e modelos recentes adota um subconjunto restrito, mas extremamente expressivo, do espaço de operações de CAD: esboçar perfis 2D (*sketching*) e extrudi-los para formar sólidos 3D (*extrusion*). Esse paradigma reflete o fluxo de trabalho de muitos projetistas, que constroem peças a partir de perfis em planos de referência. Um exemplo de processo de construção *sketch-extrude* é ilustrado na Figura 3.

Figura 3 – Ilustração de um processo de construção CAD seguindo o paradigma *sketch-extrude*.



Fonte: (WU; XIAO; ZHENG, 2021)

Esboço (*sketch*).

Um esboço define um perfil 2D fechado em um plano. Ele é composto por primitivas geométricas como:

- **Retas:** definidas por pontos inicial e final;
- **Arcos:** definidos por centro, raio e intervalo angular;
- **Círculos:** definidos por centro e raio.

Essas primitivas são combinadas em *loops* fechados, que podem ser extrudidos.

Extrusão (*extrude*).

Uma operação de extrusão toma um loop 2D e gera um sólido 3D. Entre seus parâmetros, destacam-se:

- **Direção:** unidirecional, simétrica ou até uma face existente;
- **Profundidade:** comprimento da extrusão;
- **Operação booleana:** união, subtração ou interseção com a geometria existente.

Apesar de sua aparente simplicidade, o paradigma *sketch-extrude* é capaz de gerar uma ampla gama de peças industriais. O trabalho DeepCAD (WU; XIAO; ZHENG, 2021) mostra que mais de 90% dos comandos utilizados por projetistas em cenários reais pertencem a essa classe, o que justifica sua adoção como base de representação neste projeto.

2.1.4 Representação adotada neste trabalho

O CoCAD adota a formulação de sequência de comandos inspirada no DeepCAD (WU; XIAO; ZHENG, 2021) e nos trabalhos subsequentes GenCAD (ALAM; AHMED, 2025), CADCrafter (CHEN et al., 2025) e Text2CAD (KHAN et al., 2024b). Cada modelo CAD é descrito como uma sequência ordenada de comandos paramétricos:

- Cada comando (por exemplo, *SKETCH*, *EXTRUDE*) é representado por um vetor de dimensão fixa que codifica:
 - o tipo de operação (esboço, extrusão, fim de sequência, etc.);
 - parâmetros geométricos quantizados (coordenadas, raios, ângulos);
 - parâmetros categóricos (direção de extrusão, tipo de operação booleana, etc.).
- A sequência é truncada ou preenchida (*padding*) até um comprimento máximo pré-definido, com uso de um token especial de fim de sequência (EOS) para marcar o término efetivo do programa CAD.

Essa escolha de representação é motivada por três fatores principais:

1. **Aderência à literatura recente:** trata-se da mesma família de representações empregadas por DeepCAD, GenCAD, CADCrafter e Text2CAD, o que facilita comparações e reuso de datasets e *pipelines*.

2. **Compatibilidade com o objetivo do projeto:** o foco do CoCAD é a geração condicionada de modelos paramétricos a partir de imagens e texto. Trabalhar diretamente em uma sequência CAD paramétrica evita a necessidade de reconstrução posterior a partir de *meshes* ou nuvens de pontos.
3. **Equilíbrio entre expressividade e simplicidade:** embora existam representações hierárquicas mais ricas (como SkexGen e HNC-CAD), a representação baseada em comandos *sketch-extrude* oferece uma linguagem suficientemente expressiva para peças industriais simples e intermediárias, mantendo a complexidade do modelo em um nível manejável.

2.2 Trabalhos relacionados em geração de CAD paramétrico

Nesta seção são apresentados os principais trabalhos que adaptam aprendizado profundo à geração de modelos CAD a partir de sequências de comandos, com ênfase naqueles que servem de base para o CoCAD.

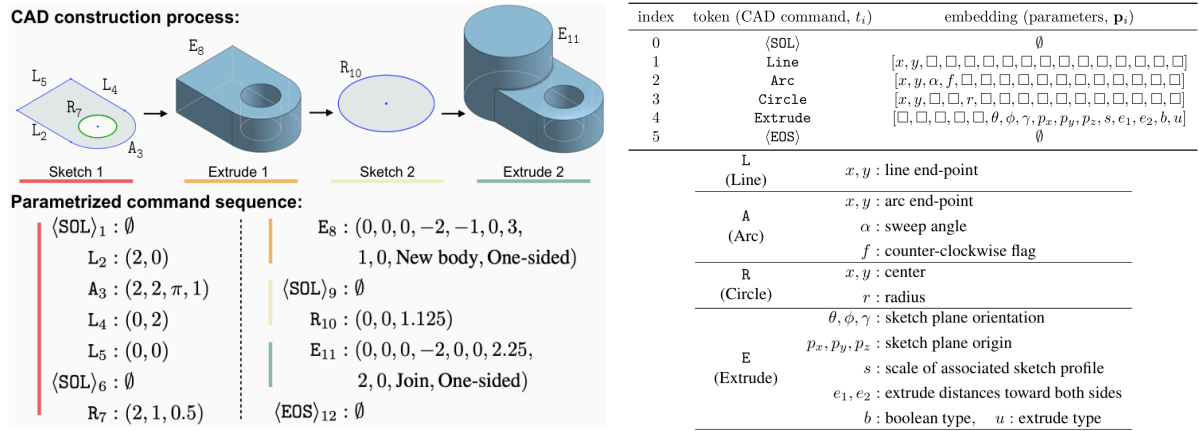
2.2.1 DeepCAD (2021)

O DeepCAD (WU; XIAO; ZHENG, 2021) representa um marco na área ao introduzir a primeira grande base de dados pública de modelos CAD paramétricos na forma de sequências de operações *sketch-extrude*. O dataset foi construído a partir do ABC Dataset, extraíndo programas CAD escritos em FeatureScript (Onshape) e filtrando apenas aqueles compostos por esboços e extrusões, resultando em 178.238 sequências de comandos CAD. A maioria das sequências contém menos de 40 comandos, o que reflete peças mecânicas de complexidade baixa a moderada. A representação de cada modelo como uma sequência de comandos é ilustrada na Figura 4.

Cada comando é codificado como um vetor de dimensão fixa que inclui o tipo de operação e até 16 parâmetros discretizados. Coordenadas e raios são quantizados em 256 níveis, enquanto parâmetros categóricos são representados como *one-hot*. A sequência é completada com o token EOS e *padding* até um comprimento máximo.

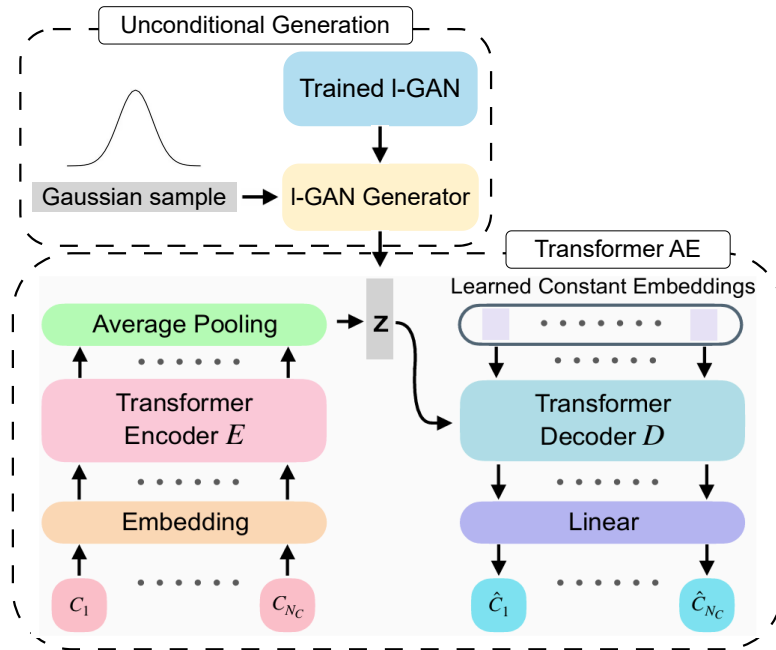
Arquiteturalmente, o DeepCAD propõe um autoencoder baseado em *Transformers*: o codificador comprime a sequência em um vetor latente e o decodificador reconstrói a sequência. Para geração, é introduzido um GAN no espaço latente: um gerador MLP mapeia ruído gaussiano para vetores latentes, que são decodificados em sequências CAD. O modelo realiza apenas **geração incondicional**, sem qualquer forma de condicionamento por imagem ou texto, mas estabelece a linha de pesquisa de modelagem sequencial de CAD que será estendida pelos trabalhos subsequentes. Essa arquitetura é ilustrada na Figura 5.

Figura 4 – Representação de um modelo CAD como sequência de comandos *sketch-extrude* no DeepCAD.



Fonte: Adaptado de (WU; XIAO; ZHENG, 2021)

Figura 5 – Arquitetura DeepCAD: *Transformer Autoencoder* com *latent GAN* para geração CAD.

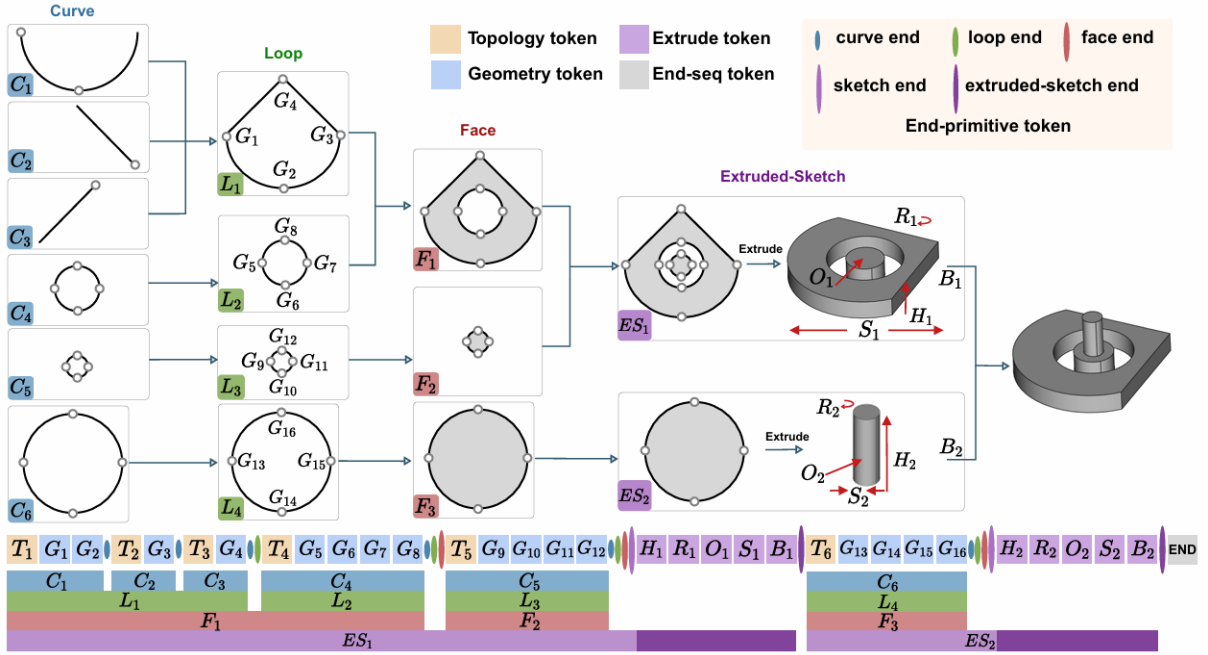


Fonte: Adaptado de (WU; XIAO; ZHENG, 2021)

2.2.2 SkexGen (2022)

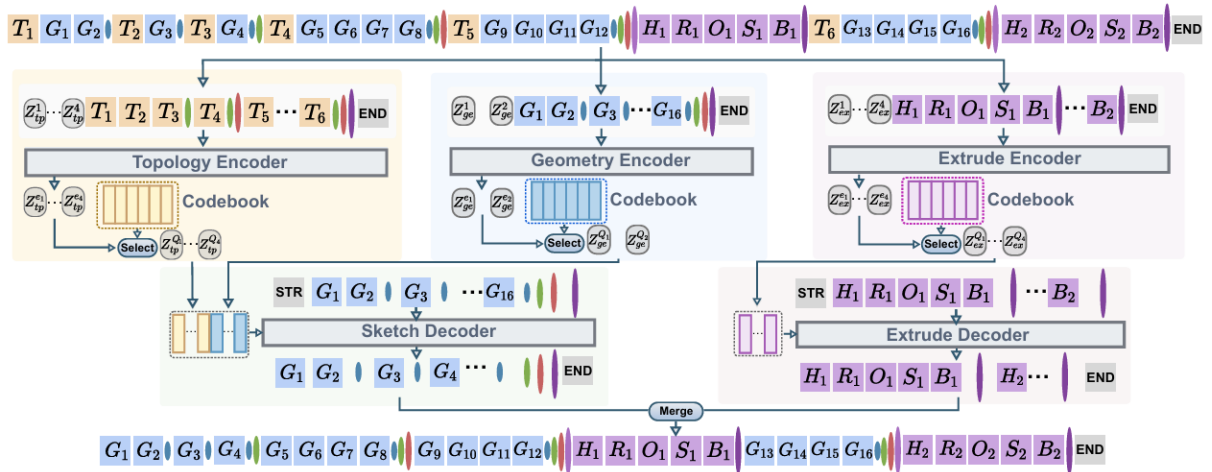
O SkexGen (XU et al., 2022) propõe uma representação hierárquica mais rica para sketches e extrusões, decompondo o modelo em múltiplos níveis: curvas, loops, esboços e extrusões. Além da sequência CAD completa, o trabalho introduz sequências específicas para topologia, geometria e extrusão, cada uma quantizada em dicionários separados. A Figura 6 ilustra essa representação hierárquica.

Figura 6 – Representação hierárquica do SkexGen com múltiplas sequências.



Fonte: (XU et al., 2022)

A arquitetura baseia-se em um VQ-VAE com três *codebooks* distintos e geração autoregressiva. Isso permite *disentanglement* entre variações de topologia, geometria 2D e parâmetros de extrusão, oferecendo maior controle sobre o tipo de variação aplicada ao modelo. Contudo, assim como o DeepCAD, o SkexGen permanece restrito à **geração incondicional** e à auto-compleção, sem incorporar condicionamento explícito por imagens ou textos. A arquitetura é ilustrada na Figura 7.

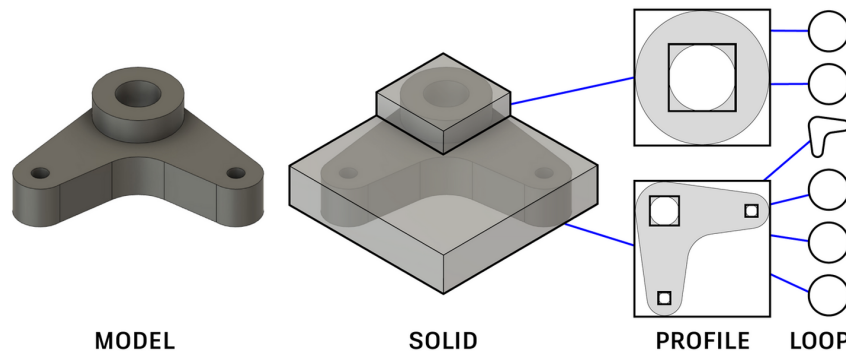
Figura 7 – Arquitetura do SkexGen: VQ-VAE com múltiplos *codebooks* e geração autoregressiva.

Fonte: (XU et al., 2022)

2.2.3 HNC-CAD (2023)

O Hierarchical Neural Coding for CAD (HNC-CAD) (XU et al., 2023) aprofunda a ideia de representações hierárquicas propondo uma estrutura explícita em árvore com três níveis: *Solid-Profile-Loop*. Essa representação dissocia o esqueleto 3D (caixas englobantes dos sólidos), perfis 2D e parâmetros de curvas, mantendo também a sequência CAD completa. A Figura 8 ilustra essa representação hierárquica.

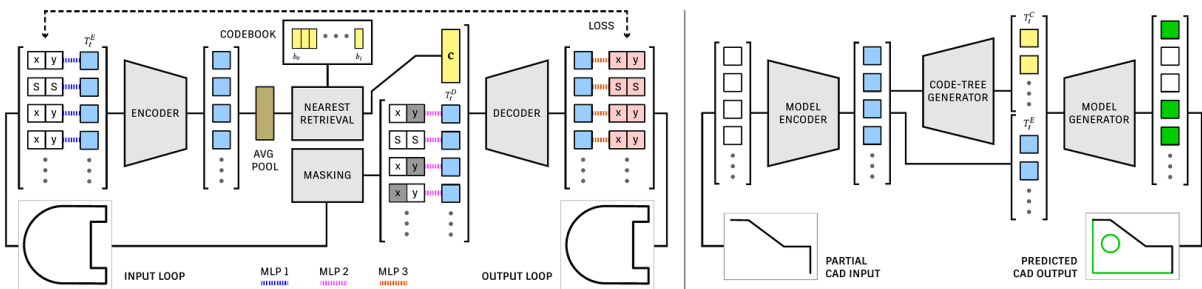
Figura 8 – Representação hierárquica do HNC-CAD com estrutura em árvore.



Fonte: (XU et al., 2023)

O modelo utiliza um VQ-VAE hierárquico, com *codebooks* distintos para cada nível da árvore, combinado a *Transformers* para geração autoregressiva. Essa arquitetura permite edições localizadas (por exemplo, alterar apenas um perfil ou um loop) sem afetar todo o modelo, introduzindo mecanismos de controle mais finos. Em contrapartida, o custo computacional é elevado e o foco permanece em **geração incondicional**, autocompleção e edições baseadas em códigos latentes, sem multimodalidade. A arquitetura é ilustrada na Figura 9.

Figura 9 – Arquitetura do HNC-CAD: VQ-VAE hierárquico com múltiplos *codebooks* e geração autoregressiva.

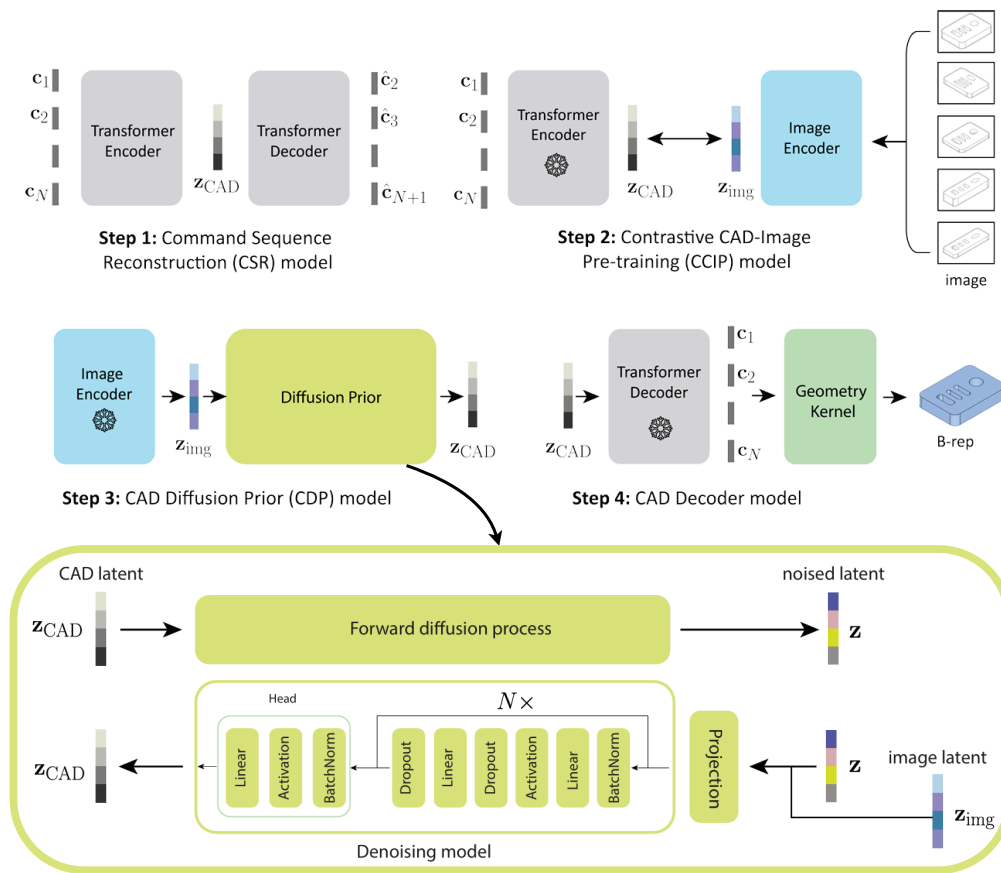


Fonte: (XU et al., 2023)

2.2.4 GenCAD (2025)

O GenCAD (ALAM; AHMED, 2025) é o primeiro trabalho dessa linha a abordar de forma sistemática a **geração condicionada em imagens**. No lado CAD, o modelo adota essencialmente a mesma representação sequencial *sketch-extrude* do DeepCAD. No lado visual, são geradas imagens sintéticas dos modelos em múltiplas escalas, que servem como entrada para um extrator de características convolucional (ResNet-18).

Figura 10 – Arquitetura do GenCAD: pipeline de quatro etapas para geração de programas CAD condicionados por imagens.



Fonte: (ALAM; AHMED, 2025)

A arquitetura, apresentada na Figura 10, é organizada em quatro módulos principais:

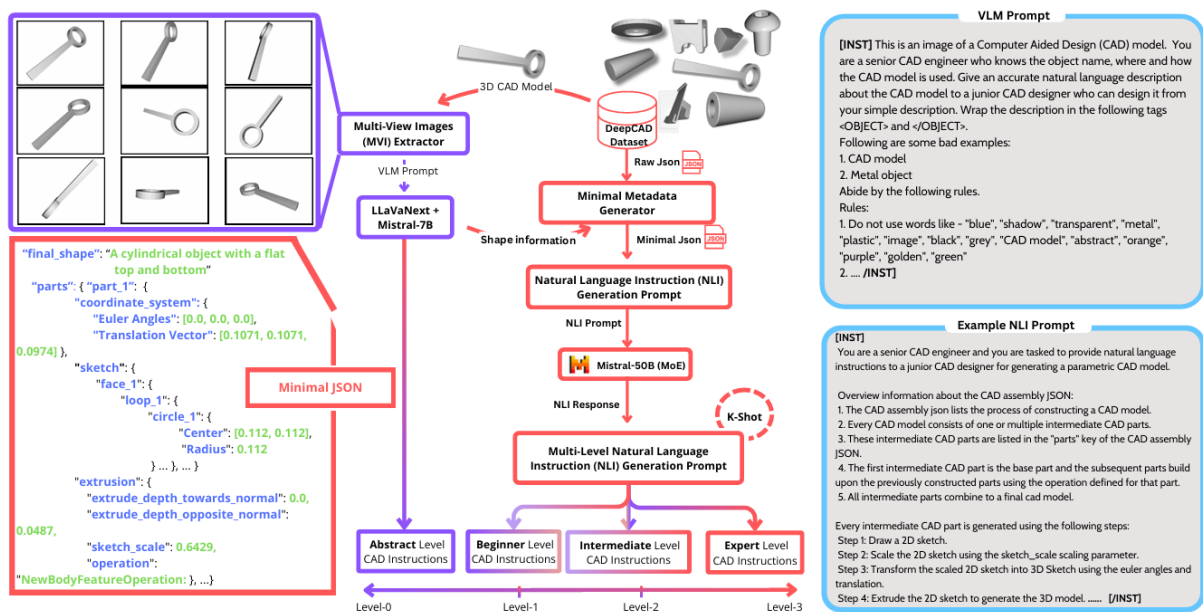
1. **CSR (Command Sequence Reconstruction):** um Transformer autoregressivo aprende a reconstruir sequências CAD, produzindo um código latente compacto para cada modelo.
2. **CCIP (Contrastive CAD-Image Pretraining):** as representações latentes de CAD são alinhadas a embeddings de imagem por meio de uma perda contrastiva, criando um espaço multimodal compartilhado.

Apesar de sua robustez, o CADCrafter limita-se à **modalidade imagem**, não explorando condicionamento textual, e, até o momento, não disponibiliza publicamente o dataset completo nem o código, o que dificulta a reprodutibilidade e o reuso direto.

2.2.6 Text2CAD (2024)

O Text2CAD (KHAN et al., 2024b) investiga o problema complementar: a geração de modelos CAD a partir de descrições textuais. O trabalho propõe um *pipeline* de anotação automática para o dataset DeepCAD, utilizando modelos de linguagem (como Mistral) e modelos multimodais (como LLaVA-NeXT) para gerar instruções em linguagem natural em múltiplos níveis de detalhe (de descrições abstratas a especificações quase paramétricas). A Figura 12 ilustra esse *pipeline* de anotação.

Figura 12 – *Pipeline* de anotação automática do Text2CAD, utilizando modelos de linguagem e multimodais para gerar descrições textuais em múltiplos níveis de detalhe a partir de modelos CAD.

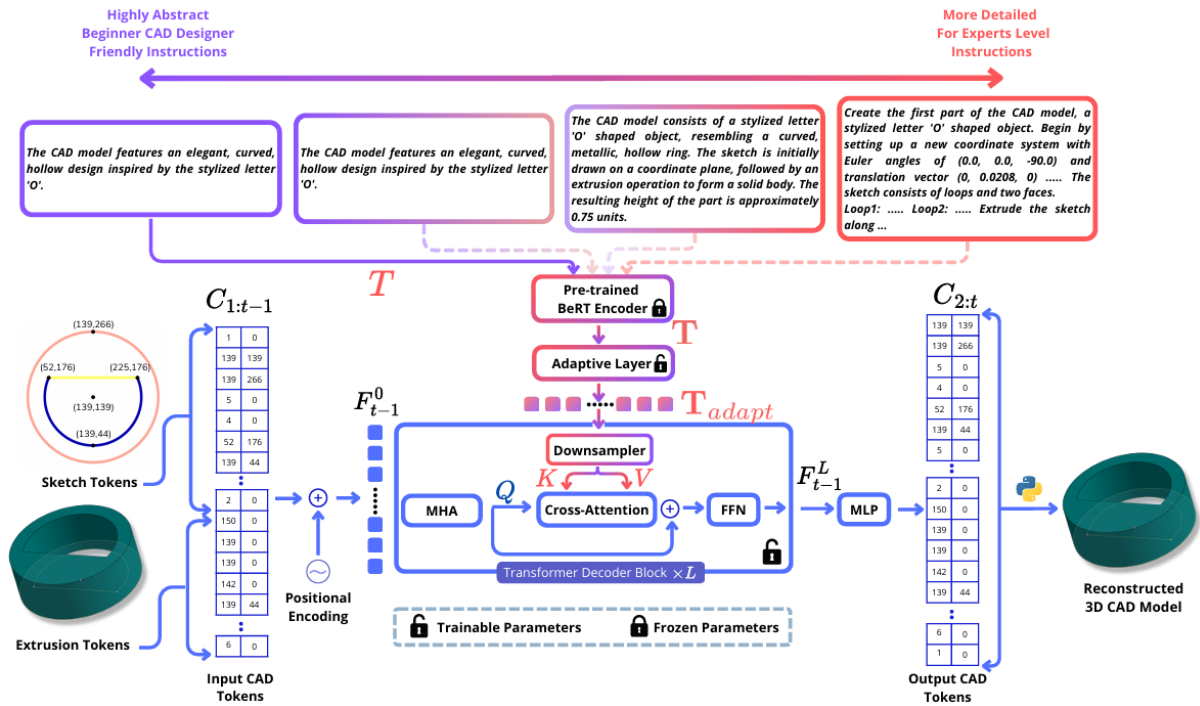


Fonte: (KHAN et al., 2024b)

A representação CAD é inspirada no DeepCAD, com adaptações para melhor alinhar comandos e descrições textuais. Um Transformer autoregressivo é treinado de forma end-to-end para mapear textos em sequências CAD, e o desempenho é avaliado em métricas de qualidade visual, precisão paramétrica e acurácia geométrica. A arquitetura é ilustrada na Figura 13.

O Text2CAD demonstra o potencial da linguagem natural como interface para CAD, mas permanece restrito à **modalidade textual**, sem integração explícita com imagens ou outros sinais multimodais. O dataset textual associado, por outro lado, torna-

Figura 13 – Arquitetura do Text2CAD: Transformer autoregressivo mapeia descrições textuais em sequências CAD paramétricas.



Fonte: (KHAN et al., 2024b)

se um recurso importante para trabalhos que pretendem combinar texto e imagem, como o CoCAD.

2.2.7 Síntese e justificativa da escolha

Os trabalhos discutidos mostram uma trajetória clara: do pioneirismo do DeepCAD na representação sequencial e geração incondicional, passando por representações hierárquicas mais expressivas (SkexGen, HNC-CAD), até modelos condicionados em imagens (GenCAD, CADCrafter) e textos (Text2CAD). Apesar dos avanços, observa-se que:

- não há, até o momento, um modelo amplamente disponível que trate, de forma integrada, condicionamento multimodal *imagem + texto* para CAD paramétrico;
- os modelos de melhor desempenho prático (GenCAD, CADCrafter) mantêm a representação sequencial *sketch-extrude* do DeepCAD, o que reforça sua adequação para geração condicionada;
- as representações hierárquicas mais ricas trazem ganhos de controle, mas aumentam substancialmente a complexidade do modelo e do *pipeline*.

Diante desse cenário, o CoCAD adota a representação sequencial *sketch-extrude* e busca avançar na direção de um sistema **multimodal**, explorando a combinação de imagens e texto para gerar modelos parametrizáveis e editáveis.

2.3 Modelos fundamentais de visão, linguagem e geração

Além dos modelos específicos de CAD, o CoCAD se apoia em arquiteturas fundamentais de visão, linguagem e geração, que são aqui apresentadas de forma resumida.

2.3.1 Arquiteturas Transformer e modelos GPT

A arquitetura Transformer (VASWANI et al., 2023) introduziu o mecanismo de *self-attention*, permitindo modelar relações de longo alcance em sequências sem a necessidade de recorrência. Essa arquitetura tornou-se o padrão de fato para tarefas de linguagem natural e vem sendo amplamente empregada em modelos de sequência CAD, como DeepCAD, GenCAD e Text2CAD.

Os modelos GPT (*Generative Pre-trained Transformer*) (RADFORD et al., 2018; BROWN et al., 2020) são um exemplo de aplicação bem-sucedida dessa arquitetura à geração textual: um Transformer autoregressivo é pré-treinado em larga escala para prever o próximo token, aprendendo uma distribuição generativa sobre textos. Esses modelos demonstram que Transformers são capazes de produzir saídas longas, coerentes e semanticamente estruturadas, o que motiva seu uso em tarefas análogas de geração de sequências CAD.

2.3.2 Modelo de visão DINOv2

O DINOv2 (OQUAB et al., 2024) é um modelo de visão treinado de forma auto-supervisionada, baseado em arquiteturas do tipo Vision Transformer (ViT). Seu treinamento utiliza objetivos de auto-distilação que incentivam a consistência entre diferentes vistas (*crops*) da mesma imagem, dispensando rótulos manuais.

As principais características que justificam seu uso neste trabalho são:

- **Representações robustas:** os embeddings extraídos são invariantes a transformações geométricas e fotométricas, o que é desejável ao lidar com diferentes pontos de vista e variações de iluminação em imagens de peças CAD.
- **Transferência eficiente:** DINOv2 apresenta bom desempenho como extrator de características em uma ampla gama de tarefas, mesmo com *fine-tuning* limitado ou ausente.

No CoCAD, DINOv2 é utilizado para extrair vetores de características de imagens sintéticas de modelos CAD (e, potencialmente, imagens reais), que são então utilizados como condicionamento para o modelo generativo no espaço latente.

2.3.3 Modelo de linguagem BERT

O BERT (*Bidirectional Encoder Representations from Transformers*) (DEVLIN et al., 2019) é um modelo de linguagem baseado em Transformer, treinado com os objetivos de *masked language modeling* e predição de próxima sentença. Diferentemente de modelos autoregressivos como GPT, o BERT aprende representações bidirecionais, capturando contexto dos dois lados de cada token.

Suas principais propriedades relevantes para este trabalho são:

- **Embeddings semânticos ricos:** as representações de sentenças e tokens capturam relações semânticas e sintáticas de forma robusta, o que é útil para interpretar descrições textuais de peças CAD com níveis variados de detalhe.
- **Facilidade de adaptação:** versões *base* e *large* do BERT podem ser utilizadas diretamente como extratores de características (*text encoders*), sem necessidade de treinamento do zero.

No CoCAD, um modelo BERT pré-treinado é utilizado para codificar descrições textuais em vetores fixos de dimensão 768, que são combinados com as características visuais extraídas pelo DINOv2 no módulo multimodal.

2.3.4 Modelos generativos baseados em *flow matching*

Modelos generativos modernos têm se concentrado principalmente em duas famílias: modelos de difusão (HO; JAIN; ABBEEL, 2020; ROMBACH et al., 2022) e modelos contínuos baseados em fluxos (ODEs ou SDEs). Embora modelos de difusão apresentem excelente desempenho, seu processo de amostragem é inerentemente lento, pois requer a simulação de centenas de passos reversos de denoising. Nesse contexto, o *flow matching* — introduzido por Lipman et al. (LIPMAN et al., 2023) e posteriormente generalizado em (LIU; GONG; LIU, 2022) — surge como uma alternativa contínua, determinística e mais eficiente para aprendizado generativo.

Formulação geral

A ideia central do *flow matching* é aprender um **campo de velocidade** $v_\theta(x, t)$ que transporta suavemente uma distribuição simples p_0 (por exemplo, gaussiana) até a

distribuição de dados p_1 . Para isso, considera-se uma curva de distribuições $\{p_t\}_{t \in [0,1]}$ conectando p_0 a p_1 por meio de um fluxo determinístico governado pela ODE:

$$\frac{dx}{dt} = v_\theta(x, t, c),$$

onde c representa um vetor de condicionamento (texto, imagem, ou ambos).

O treinamento é formulado para aproximar o **campo de transporte ótimo** associado ao problema de *Optimal Transport* (OT), no qual se busca o fluxo de menor custo que transforma p_0 em p_1 . No *flow matching*, em vez de resolver explicitamente um problema de OT, amostra-se diretamente pares de pontos (x_0, x_1) e instantes t , e constrói-se um vetor alvo:

$$v^*(x_t, t) = \frac{d}{dt}((1-t)x_0 + tx_1) = x_1 - x_0,$$

com $x_t = (1-t)x_0 + tx_1$. O modelo é treinado com uma loss do tipo:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{x_0, x_1, t} \left[\left\| v_\theta(x_t, t, c) - (x_1 - x_0) \right\|^2 \right],$$

que obriga v_θ a imitar o campo de transporte ideal, sem necessidade de um processo de difusão estocástico. Essa construção fornece uma aproximação eficiente do fluxo ótimo entre distribuições.

Vantagens em relação à difusão

- **Amostragem eficiente:** a geração requer apenas integrar uma ODE, podendo utilizar poucos passos com métodos como Euler ou Runge–Kutta.
- **Modelo determinístico:** não depende de ruído progressivo nem estimativas de score.
- **Condicionamento direto:** o condicionamento multimodal é incorporado no próprio campo de velocidade v_θ , de forma simples e contínua.
- **Simplicidade conceitual:** aprende-se diretamente o transporte entre duas distribuições, em vez de inverter um processo Markoviano ruidoso.

Aplicação ao CoCAD

No contexto do CoCAD, utiliza-se o *flow matching* para aprender um fluxo entre:

$$p_0(z) = \mathcal{N}(0, I) \quad \text{e} \quad p_1(z | c) = \text{distribuição real de latentes CAD condicionados.}$$

Ou seja, o modelo aprende a transformar um vetor latente simples em um latente CAD válido, alinhado às características semânticas da entrada multimodal (imagem, texto ou ambas). Essa abordagem substitui o difusor latente empregado em GenCAD (ALAM; AHMED, 2025) e CADCrafter (CHEN et al., 2025), oferecendo:

- maior eficiência de amostragem;
- maior estabilidade em treinamento;
- arquitetura mais simples e modular;
- compatibilidade natural com condicionamento multimodal via FiLM.

2.3.5 Blocos residuais com condicionamento FiLM

O condicionamento multimodal desempenha papel fundamental na geração de latentes CAD que reflitam as características visuais e textuais da entrada. Para isso, o CoCAD incorpora blocos residuais do tipo FiLM (*Feature-wise Linear Modulation*), originalmente introduzidos por (PEREZ et al., 2017), que permitem modular dinamicamente as ativações internas da rede a partir do condicionamento.

Formulação geral

Seja $h \in \mathbb{R}^d$ uma ativação intermediária e c o vetor de condicionamento resultante da fusão entre embeddings de imagem e texto. O bloco FiLM aprende duas funções:

$$\gamma(c), \beta(c) \in \mathbb{R}^d,$$

que modulam h de forma afim:

$$\text{FiLM}(h, c) = \gamma(c) \odot h + \beta(c).$$

Em arquiteturas residuais, essa modulação é aplicada dentro de um bloco:

$$h' = h + f(\text{FiLM}(h, c)),$$

onde f é uma transformação não linear (MLP, ReLU/SwiGLU, etc.).

FiLM no contexto do modelo de fluxo

Ao aplicar FiLM diretamente no campo de velocidade $v_\theta(x, t, c)$, obtêm-se benefícios importantes:

- **Condicionamento contínuo no fluxo:** o vetor multimodal influencia cada passo da integração da ODE.
- **Maior flexibilidade:** a rede adapta sua dinâmica para diferentes modalidades (imagem, texto ou ambas).
- **Evita concatenação direta:** concatenar c a x aumenta a dimensionalidade e dificulta a aprendizagem; o FiLM modula de forma mais suave e eficiente.
- **Compatível com arquiteturas residuais:** ideal para MLPs profundos, como o *Residual FiLM-style MLP* empregado no CoCAD.

2.4 Síntese

Neste capítulo, discutimos a escolha da representação de modelos CAD como sequências de comandos *sketch-extrude*, as principais contribuições e limitações dos trabalhos relacionados (DeepCAD, SkexGen, HNC-CAD, GenCAD, CADCrafter e Text2CAD) e os modelos fundamentais de visão, linguagem e geração (Transformers, DINOv2, BERT, *flow matching* e *FiLM*) que servem de base para o CoCAD. Esses elementos conceituais sustentam as decisões de projeto apresentadas nos capítulos seguintes, em particular no que diz respeito à arquitetura multimodal proposta e ao uso de fluxos condicionados para geração de latentes CAD a partir de imagens e descrições textuais.

3 Método do trabalho

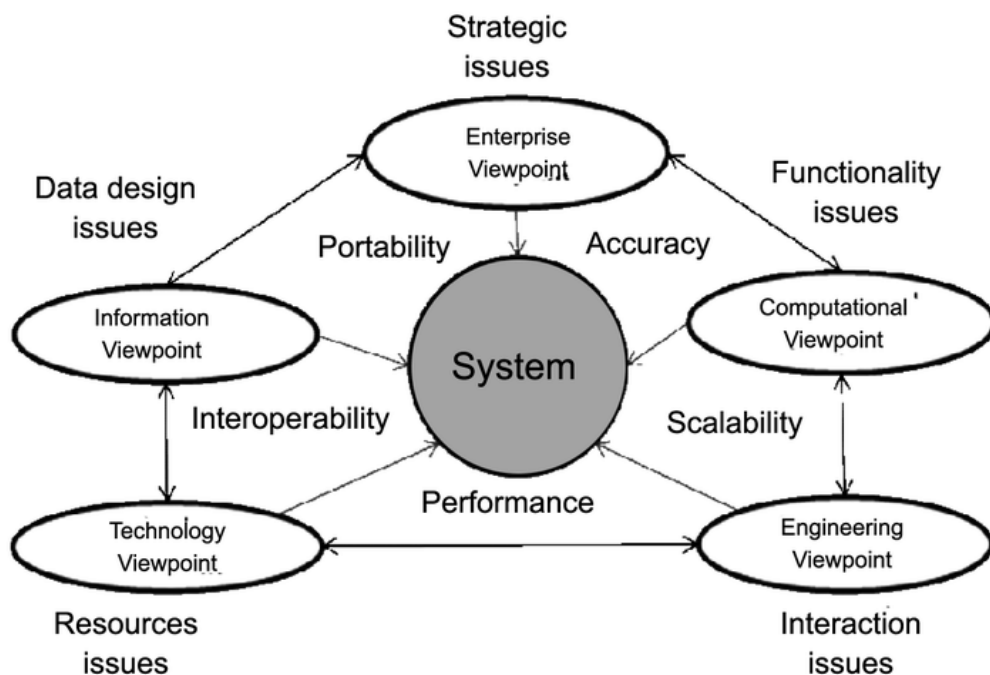
Este capítulo apresenta o método adotado para orientar o desenvolvimento do projeto CoCAD. Foi utilizado o modelo de referência RM-ODP (*Reference Model for Open Distributed Processing*), formalizado pela norma ISO/IEC 10746 (ISO/IEC, 1998; ISO/IEC, 2009). O método organiza a especificação de sistemas complexos por meio de diferentes visões (*viewpoints*), permitindo separar preocupações, esclarecer requisitos e orientar a arquitetura e implementação.

Como este trabalho não trata de um sistema distribuído completo, mas sim de uma plataforma de geração de modelos CAD orientada por IA, foi utilizada uma versão *adaptada* do RM-ODP. A seguir descrevem-se tanto o modelo original quanto a adaptação empregada, justificando sua aplicação neste projeto.

3.1 O Modelo RM-ODP Original

O RM-ODP define cinco visões complementares que descrevem um sistema de forma ortogonal, cada uma tratada de maneira independente, mas coerente no conjunto. A Figura 14 ilustra essas visões, que são as seguintes (ISO/IEC, 1998; ISO/IEC, 2009):

Figura 14 – As cinco visões do RM-ODP original.



Fonte: Adaptado de (CRETU, 2014).

- **Enterprise Viewpoint (Visão Empresarial)**. Descreve o propósito, escopo, regras, políticas e objetivos do sistema no contexto organizacional.
- **Information Viewpoint (Visão Informacional)**. Define os tipos de informação manipulados, suas estruturas, semântica, restrições e relacionamentos.
- **Computational Viewpoint (Visão Computacional)**. Descreve a funcionalidade do sistema como um conjunto de componentes computacionais e suas interfaces, independentemente da tecnologia usada.
- **Engineering Viewpoint (Visão de Engenharia)**. Especifica os mecanismos de suporte, infraestrutura e recursos necessários para permitir comunicação, distribuição e execução do sistema.
- **Technology Viewpoint (Visão Tecnológica)**. Detalha tecnologias, linguagens, plataformas, hardware e demais elementos concretos utilizados na implementação.

A separação em cinco visões é essencial em sistemas distribuídos de grande porte. Entretanto, para aplicações de aprendizagem de máquina com foco em modelagem conceitual e implementação, a estrutura completa pode se tornar excessivamente detalhada. Por essa razão, este trabalho adota uma versão adaptada, descrita a seguir.

3.2 Versão Adaptada do RM-ODP para o Projeto CoCAD

Embora o RM-ODP original inclua cinco visões formais (ISO/IEC, 1998; ISO/IEC, 2009), a estrutura completa é mais apropriada para sistemas distribuídos de grande porte. Para projetos de modelagem conceitual e implementação de sistemas baseados em aprendizado de máquina, é comum utilizar uma adaptação que preserve os princípios de separação de preocupações, mas reorganize as visões de forma mais adequada ao processo de desenvolvimento.

Assim, a versão adaptada utilizada neste trabalho compreende quatro visões:

1. **Visão Empresarial** (corresponde diretamente ao *Enterprise Viewpoint*). Define o contexto, os objetivos do sistema, os atores envolvidos e a motivação organizacional que fundamenta o projeto.
2. **Visão de Requisitos** (integra elementos da *Information Viewpoint* e da *Computational Viewpoint*). Esta visão especifica de forma clara, técnica e mensurável dois tipos de requisitos:
 - **Requisitos funcionais** — descrevem os comportamentos, capacidades e serviços que o sistema deve fornecer, em conformidade com os princípios clássicos de engenharia de requisitos apresentados em (SOMMERVILLE, 2011);

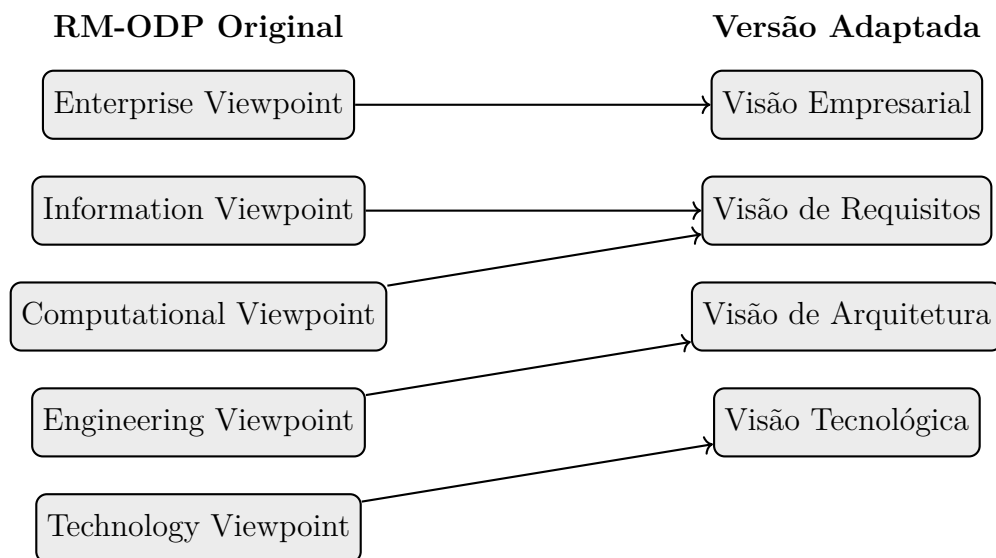
- **Requisitos não funcionais** — descrevem atributos de qualidade, desempenho, segurança, confiabilidade e demais propriedades externas do sistema, estruturados segundo o modelo de qualidade da norma internacional **ISO/IEC 25010:2011** (ISO/IEC, 2011).

A Visão de Requisitos atua, assim, como ponte entre os objetivos da Visão Empresarial e as decisões técnicas das visões subsequentes.

3. **Visão de Arquitetura** (relacionada ao *Engineering Viewpoint*, porém em nível conceitual). Descreve a organização estrutural do sistema — módulos, componentes, interfaces, fluxos internos e dependências — definindo *como* a solução deve ser estruturada para atender aos requisitos definidos.
4. **Visão Tecnológica** (corresponde ao *Technology Viewpoint*). Especifica as escolhas concretas de implementação, incluindo frameworks, bibliotecas, modelos de aprendizado de máquina, hardware e infraestrutura de execução, que materializam a Visão de Arquitetura.

A Figura 15 apresenta como as cinco visões originais do RM-ODP são reinterpretadas e reorganizadas na versão adaptada empregada neste projeto.

Figura 15 – Transformação das visões originais do RM-ODP na versão adaptada utilizada neste projeto.



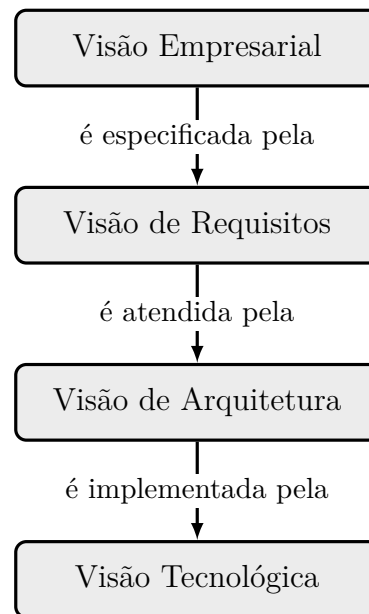
Fonte: Elaborado pelo autor.

A adaptação foi escolhida porque simplifica a aplicação dos conceitos do RM-ODP ao desenvolvimento de sistemas de IA, mantendo a rastreabilidade entre necessidades organizacionais, requisitos, arquitetura e implementação tecnológica.

3.2.1 Relação entre as Visões na Versão Adaptada

As visões da versão adaptada do RM-ODP organizam o desenvolvimento do projeto CoCAD em um fluxo lógico no qual cada visão orienta e fundamenta a próxima. A Figura 16 apresenta esse fluxo de forma esquemática.

Figura 16 – Relação entre as visões da versão adaptada do RM-ODP.



Fonte: Elaborado pelo autor.

Dessa forma, a relação entre essas visões pode ser descrita da seguinte forma:

- A **Visão Empresarial** estabelece o problema, o propósito e o contexto em que o sistema será desenvolvido.
- A **Visão de Requisitos** traduz a Visão Empresarial em uma especificação técnica, completa e mensurável, definindo claramente o que deve ser atendido pelo sistema.
- A **Visão de Arquitetura** descreve como atender aos requisitos, projetando uma solução conceitual composta por módulos, fluxos e componentes que não dependem de tecnologia.
- A **Visão Tecnológica** implementa a Visão de Arquitetura por meio de escolhas concretas de tecnologias, frameworks e plataformas, assegurando que a solução arquitetural seja realizada corretamente.

3.3 Fases de Desenvolvimento do Projeto

O método de trabalho adotado orienta a execução do projeto CoCAD segundo um conjunto de fases estruturadas e coerentes com as visões descritas. Cada fase é relacionada

aos capítulos posteriores da monografia, nos quais o trabalho é detalhado.

1. **Definição do tema e delimitação inicial** Identificação do problema de pesquisa, motivação e relevância. Relaciona-se ao Capítulo 1 e à **Visão Empresarial** na Seção 4.1.
2. **Revisão da literatura** Estudo aprofundado sobre representações CAD, modelos generativos, multimodalidade, *flow matching* e trabalhos relacionados. Esta revisão fundamenta o Capítulo 2.
3. **Definição do escopo e dos objetivos** Decisão sobre os limites funcionais do sistema, modalidades de entrada (imagem, texto), tipos de saída (sequência CAD) e requisitos gerais. Faz parte da **Visão Empresarial** da Seção 4.1.
4. **Especificação dos requisitos funcionais e não funcionais** Inclui requisitos de desempenho, segurança, qualidade, datasets, formas de condicionamento, métricas de geração e interface. Componente central da **Visão de Requisitos**, que será detalhada na Seção 4.2 do Capítulo 4.
5. **Definição da arquitetura do sistema** Projeto da estrutura conceitual do CoCAD: autoencoder, módulos multimodais, *flow matching*, pipelines de dados, componentes da interface e fluxos de execução. Esta fase compõe a **Visão de Arquitetura** e será aprofundada na Seção 5.1 do Capítulo 5.
6. **Escolha das tecnologias** Seleção das ferramentas: PyTorch Lightning, Hydra, DINOv2, BERT, OpenCASCADE, Gradio, Python, CUDA, entre outras. Relaciona-se à **Visão Tecnológica** e será apresentada na Seção 5.2 do Capítulo 5.
7. **Implementação** Construção das etapas do sistema:
 - tratamento e processamento dos dados;
 - implementação dos modelos de IA;
 - treinamento dos módulos (autoencoder, módulos multimodais, modelo de fluxo);
 - construção da interface interativa;
 - integração dos módulos na pipeline final;
 - documentação técnica.

Todas essas atividades serão descritas na Seção 5.3 do Capítulo 5.

8. **Avaliação e testes finais** Comparação dos resultados com o estado da arte, avaliação por métricas geométricas e validação da interface multimodal. Esta etapa é apresentada na Seção 5.4 do Capítulo 5.

9. **Conclusões e propostas futuras** Consolidação dos resultados obtidos, análise crítica e definição de extensões possíveis. Relaciona-se ao Capítulo 6.

O conjunto dessas fases fornece uma estrutura clara e orientada ao método RM-ODP adaptado, permitindo que o desenvolvimento do CoCAD avance de maneira organizada e rastreável, desde a concepção inicial até a implementação final.

4 Especificação Técnica

Este capítulo apresenta a especificação técnica do sistema CoCAD, estruturada em conformidade com as diretrizes metodológicas descritas no Capítulo 3, com base na versão adaptada do RM-ODP. Inicia-se pela *Visão Empresarial*, responsável por contextualizar os objetivos e o valor do sistema, seguida da *Visão de Requisitos*, que compreende histórias de usuário, casos de uso e requisitos funcionais e não funcionais. Essa estrutura estabelece o fundamento conceitual para os capítulos posteriores, nos quais são detalhados o projeto arquitetural, a implementação e a avaliação do sistema.

4.1 Visão Empresarial

A Visão Empresarial, correspondente ao *Enterprise Viewpoint* do RM-ODP, descreve o contexto organizacional, os objetivos do sistema e os atores envolvidos. No domínio da engenharia e do design de produtos, ferramentas de CAD paramétrico como SOLIDWORKS (Dassault Systèmes, 2025b), CATIA (Dassault Systèmes, 2025a), AutoCAD (Autodesk, Inc., 2025a) e Fusion 360 (Autodesk, Inc., 2025b) constituem o núcleo dos fluxos industriais de modelagem, análise e fabricação. Embora poderosas, tais plataformas permanecem fortemente dependentes da criação manual de esboços, parâmetros e operações, o que torna o processo de projeto demorado, repetitivo e dependente de experiência especializada.

Nesse cenário, funcionalidades que permitam gerar modelos CAD automaticamente a partir de entradas simples — como imagens ou descrições textuais — apresentam elevado potencial para acelerar ciclos de concepção, reduzir retrabalho, democratizar o acesso a ferramentas de modelagem e ampliar a produtividade de engenheiros e designers. Além disso, tais mecanismos podem atuar como assistentes inteligentes integrados a ferramentas CAD existentes, colaborando com o usuário ao fornecer geometrias iniciais plausíveis que podem posteriormente ser refinadas manualmente.

O sistema CoCAD se posiciona como um **protótipo funcional** dessa visão: uma ferramenta capaz de gerar modelos CAD paramétricos preliminares condicionados a imagem, texto ou ambas as modalidades. Embora desenvolvido como sistema independente, o CoCAD é concebido de forma a poder ser integrado futuramente a plataformas industriais consagradas, atuando como assistente de *co-criação* de modelos CAD.

Os principais atores identificados nesta visão são:

- **Profissionais de projeto** (engenheiros e designers) responsáveis pela criação e

modificação de modelos CAD;

- **Pesquisadores e usuários técnicos** interessados em automação de fluxos CAD, aprendizado de máquina e prototipagem rápida;
- **Organizações** que buscam reduzir custos, acelerar o desenvolvimento de produtos e incorporar ferramentas inteligentes ao processo de design.

Essa visão define o propósito geral do sistema e orienta as histórias de usuário, casos de uso e requisitos apresentados nas seções seguintes.

4.2 Visão de Requisitos

A Visão de Requisitos integra elementos que, no RM-ODP original, pertencem às visões *Information* e *Computational*. Essa visão especifica, de forma clara e verificável, o comportamento esperado do sistema e os critérios de qualidade que ele deve satisfazer. A elaboração segue as diretrizes da ISO/IEC/IEEE 29148:2018 ([ISO/IEC/IEEE, 2018](#)), enquanto a classificação dos requisitos não funcionais é orientada pela ISO/IEC 25010:2011 ([ISO/IEC, 2011](#)).

4.2.1 Histórias de Usuário

As histórias de usuário descrevem, a partir da perspectiva do ator principal — o **profissional de projeto** (engenheiro ou designer) — as formas essenciais de interação com o sistema CoCAD. Elas expressam necessidades reais do processo de concepção de produtos e servem como fundamento para os casos de uso e requisitos funcionais apresentados posteriormente. Cada história considera um cenário de uso específico: geração a partir de imagem, geração a partir de texto e geração multimodal combinando ambas as entradas.

4.2.1.1 História de Usuário 1 — Geração de modelo CAD a partir de imagem

Como profissional de projeto,
quero enviar a imagem de uma peça mecânica (render, desenho técnico digitalizado ou fotografia),
para que o sistema gere automaticamente um modelo CAD paramétrico que represente a geometria observada.

Critérios de aceitação:

1. O sistema deve aceitar imagens em formatos comuns (PNG, JPEG).

2. A imagem deve passar por validação de formato e tamanho antes do processamento.
3. O sistema deve gerar um modelo CAD paramétrico válido, visualizável em 3D.
4. O tempo de geração deve ser compatível com um fluxo iterativo de projeto (segundos a poucos minutos).
5. O usuário deve conseguir visualizar o modelo e exportá-lo em formato padrão.

4.2.1.2 História de Usuário 2 — Geração de modelo CAD a partir de texto

Como profissional de projeto,
quero descrever em linguagem natural o formato de uma peça,
para que o sistema gere automaticamente um modelo CAD paramétrico correspondente à descrição fornecida.

Critérios de aceitação:

1. O sistema deve aceitar descrições textuais em linguagem natural, desde que não vazias, muito curtas ou longas demais.
2. O sistema deve realizar pré-processamento textual e extrair características semânticas compatíveis com a geração CAD.
3. O modelo CAD gerado deve ser válido, visualizável em 3D e estruturado a partir de comandos paramétricos.
4. O tempo de geração deve ser compatível com um fluxo iterativo de projeto.
5. O usuário deve poder visualizar e exportar o modelo resultante.

4.2.1.3 História de Usuário 3 — Geração de modelo CAD multimodal (imagem + texto)

Como profissional de projeto,
quero fornecer simultaneamente uma imagem e uma descrição textual complementar da peça,
para que o sistema combine ambas as informações e gere um modelo CAD paramétrico mais fiel à minha intenção de projeto.

Critérios de aceitação:

1. O sistema deve permitir o envio simultâneo de imagem e texto na mesma interface.
2. A imagem e o texto devem ser validados individualmente (formato e tamanho da imagem; tamanho do texto).

3. O sistema deve extrair características visuais e semânticas e combiná-las em uma representação multimodal.
4. O modelo CAD gerado deve refletir tanto a geometria observada quanto os detalhes fornecidos na descrição textual.
5. O modelo deve ser válido, visualizável em 3D e exportável em formato padrão.
6. O tempo de geração deve permanecer compatível com ciclos iterativos rápidos de design.

4.2.2 Casos de Uso

Os casos de uso a seguir detalham, de forma estruturada, as interações descritas nas histórias de usuário da Seção 4.2.1. Em todos eles, o ator principal é o **profissional de projeto** (engenheiro ou designer), que utiliza o sistema CoCAD para gerar modelos CAD paramétricos a partir de entradas visuais e/ou textuais.

A Figura 17 apresenta o diagrama geral de casos de uso.

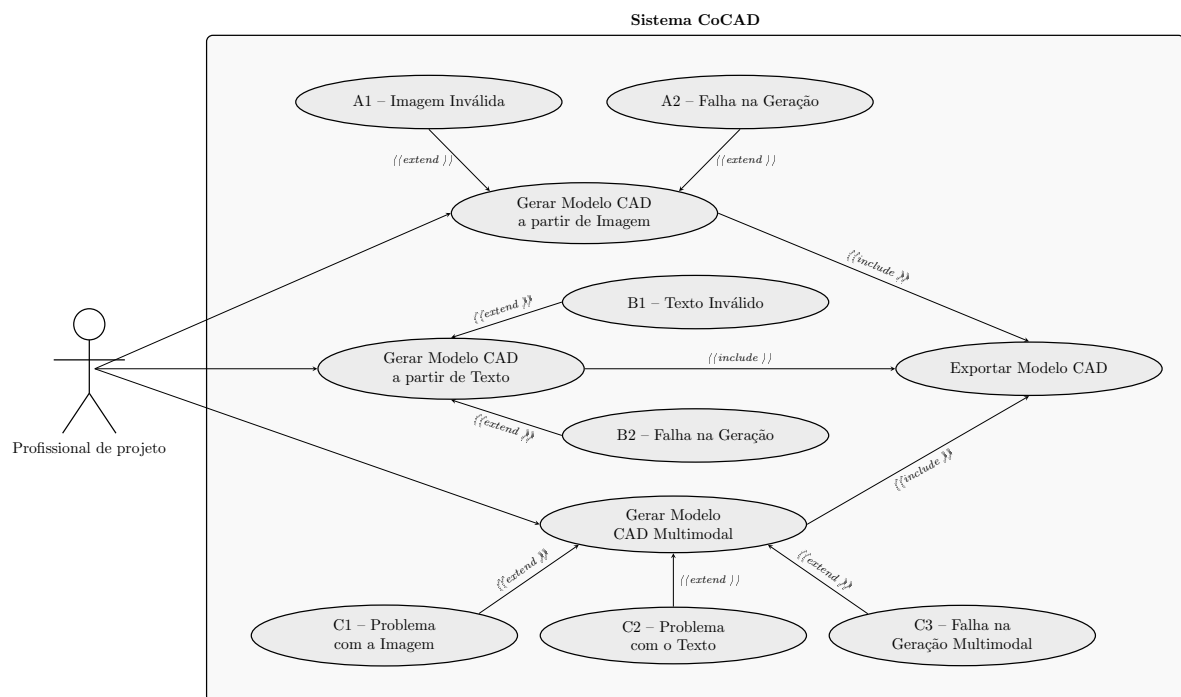


Figura 17 – Diagrama de casos de uso do sistema de geração de modelos CAD.

4.2.2.1 Caso de Uso 1 – Geração de Modelo CAD a partir de Imagem

Ator principal: Profissional de projeto

Objetivo: Gerar um modelo CAD paramétrico a partir de uma imagem.

Descrição: O usuário fornece uma imagem da peça, e o sistema CoCAD processa essa imagem para gerar uma sequência de comandos CAD paramétricos (*sketch-extrude*), renderizando o modelo para visualização e exportação.

Fluxo principal:

1. O usuário acessa a interface unificada do sistema CoCAD.
2. O usuário realiza o upload de uma imagem (PNG ou JPEG).
3. O sistema valida o tamanho e o formato do arquivo enviado.
4. O sistema executa o pré-processamento da imagem (normalização, extração de características visuais).
5. O sistema utiliza o modelo de geração para produzir uma sequência de comandos CAD.
6. O sistema converte essa sequência em um modelo CAD e o renderiza em uma visualização 3D interativa.
7. O usuário analisa o modelo gerado.
8. O usuário opta por exportar o modelo em um formato padrão.
9. O sistema exporta o arquivo e informa a conclusão da operação.

Fluxos alternativos:

- **A1 – Imagem inválida:**

- 3a. Se o formato não for suportado ou o arquivo exceder o tamanho permitido, o sistema rejeita o upload.
- 3b. O sistema exibe uma mensagem explicando o motivo.
- 3c. O usuário envia uma nova imagem válida.

- **A2 – Falha na geração:**

- 5a. Se a sequência CAD gerada for inválida ou inconsistente, o sistema registra a falha.
- 5b. O sistema informa ao usuário que a geração não pôde ser concluída.
- 5c. O usuário pode tentar novamente ou fornecer nova imagem.

4.2.2.2 Caso de Uso 2 – Geração de Modelo CAD a partir de Texto

Ator principal: Profissional de projeto

Objetivo: Gerar um modelo CAD paramétrico a partir de uma descrição textual em linguagem natural.

Descrição: O usuário insere uma descrição textual da peça. O sistema processa o texto, extrai características semânticas e gera uma sequência de comandos CAD correspondente.

Fluxo principal:

1. O usuário acessa a interface unificada do sistema.
2. O usuário insere a descrição textual da peça.
3. O sistema valida apenas o tamanho do texto (não vazio e não excedendo o limite permitido).
4. O sistema executa o pré-processamento textual (tokenização e extração de características semânticas).
5. O sistema gera uma sequência CAD paramétrica correspondente.
6. O sistema renderiza o modelo em uma visualização 3D.
7. O usuário analisa o modelo gerado.
8. O usuário decide exportar o modelo.
9. O sistema realiza a exportação e informa a conclusão.

Fluxos alternativos:

- **B1 – Texto inválido:**

- 3a. O sistema rejeita a entrada textual por não atender aos limites estabelecidos (vazio, muito curto ou longo demais).
- 3b. O sistema solicita que o usuário revise o texto.
- 3c. O usuário insere uma nova descrição.

- **B2 – Falha na geração:**

- 5a. A sequência CAD gerada é inválida.
- 5b. O sistema informa ao usuário que não foi possível gerar o modelo.
- 5c. O usuário decide corrigir o texto ou encerrar a operação.

4.2.2.3 Caso de Uso 3 – Geração de Modelo CAD Multimodal (Imagem + Texto)

Ator principal: Profissional de projeto

Objetivo: Gerar um modelo CAD paramétrico combinando informações de imagem e texto.

Descrição: O usuário fornece simultaneamente uma imagem e uma descrição textual complementar da peça. O sistema processa ambas as entradas, realiza fusão multimodal e gera uma sequência CAD paramétrica que reflete a combinação das duas modalidades.

Fluxo principal:

1. O usuário acessa a interface unificada do sistema CoCAD.
2. O usuário faz upload da imagem.
3. O usuário insere a descrição textual complementar.
4. O sistema valida o formato e tamanho da imagem.
5. O sistema valida apenas o tamanho do texto.
6. O sistema extrai características visuais da imagem.
7. O sistema extrai características semânticas do texto.
8. O sistema combina ambas as modalidades em um espaço latente multimodal.
9. O sistema gera a sequência CAD paramétrica correspondente.
10. O sistema renderiza o modelo em 3D.
11. O usuário analisa o modelo gerado.
12. O usuário exporta o modelo.
13. O sistema conclui a exportação e informa o usuário.

Fluxos alternativos:

- **C1 – Problema com a imagem:**
 - 4a. A imagem está em formato inválido ou excede o tamanho permitido.
 - 4b. O sistema informa o erro.
 - 4c. O usuário envia uma nova imagem.
- **C2 – Problema com o texto:**

- 5a. O texto é vazio, muito curto ou longo demais.
- 5b. O sistema solicita que o usuário revise a descrição.
- 5c. O usuário fornece novo texto.

- **C3 – Falha na geração multimodal:**

- 9a. A sequência CAD resultante é inválida.
- 9b. O sistema informa que não pôde gerar o modelo.
- 9c. O usuário decide corrigir uma ou ambas as entradas, ou tentar uma modalidade única.

4.2.3 Requisitos Funcionais

Os requisitos funcionais descrevem as capacidades e comportamentos que o sistema deve oferecer ao usuário, conforme princípios clássicos de engenharia de requisitos (SOMMERVILLE, 2011). Eles derivam diretamente das histórias de usuário e dos casos de uso apresentados. Os requisitos funcionais do sistema CoCAD são descritos na Tabela 2.

Tabela 2 – Requisitos Funcionais do sistema CoCAD

Código	Nome	Descrição
RF01	Entrada de imagem	O usuário deve ser capaz de enviar uma imagem para ser utilizada como fonte de condicionamento no processo de geração do modelo CAD.
RF02	Entrada de texto	O usuário deve ser capaz de fornecer uma descrição textual em linguagem natural para orientar a geração do modelo CAD.
RF03	Entrada multimodal	A interface deve possibilitar o envio simultâneo de imagem e texto, combinando ambas as modalidades durante a geração.
RF04	Pré - processamento	As entradas fornecidas devem passar por um módulo de pré-processamento responsável por extrair características visuais e/ou textuais relevantes.
RF05	Geração CAD	A geração CAD deve resultar em uma sequência de comandos CAD paramétricos válida, estruturada principalmente por operações do tipo <i>sketch-extrude</i> .
RF06	Visualização 3D	A interface deve disponibilizar uma visualização tridimensional interativa do modelo CAD produzido.
RF07	Exportação	O modelo CAD produzido deve ser passível de exportação em formatos padronizados e compatíveis com ferramentas de CAD.
RF08	Histórico de geração	Informações essenciais das gerações, como entradas utilizadas, parâmetros e modelos resultantes, devem ser registradas para consulta posterior.

4.2.4 Requisitos Não Funcionais

Os requisitos não funcionais estabelecem critérios de qualidade, desempenho e restrições que regulam o comportamento do sistema. A classificação segue as categorias definidas pela norma ISO/IEC 25010:2011 (ISO/IEC, 2011). Os requisitos não funcionais do sistema CoCAD são descritos na Tabela 3.

Tabela 3 – Requisitos Não Funcionais do sistema CoCAD

Código	Nome	Descrição
RNF01	Desempenho	O sistema deve gerar um modelo CAD preliminar em tempo compatível com um fluxo iterativo de design, não excedendo alguns minutos.
RNF02	Escalabilidade	A solução deve ser capaz de lidar com conjuntos de dados de grande porte (na ordem de centenas de milhares de modelos CAD) e permitir expansão para volumes ainda maiores, incluindo a execução em ambientes distribuídos quando necessário.
RNF03	Precisão e confiabilidade	O sistema deve alcançar taxa de geração válida superior a 85%, garantindo consistência nas sequências de comandos produzidas.
RNF04	Validade geométrica	Os modelos CAD gerados devem ser geometricamente válidos e adequados para edição em ferramentas de CAD paramétrico.
RNF05	Usabilidade	A interface deve ser simples, intuitiva e acessível a usuários sem experiência em aprendizado de máquina.
RNF06	Portabilidade	O sistema deve permitir execução em diferentes ambientes computacionais, incluindo execução local e em nuvem.
RNF07	Interoperabilidade	O sistema deve utilizar formatos abertos para garantir compatibilidade com plataformas CAD variadas.
RNF08	Manutenibilidade	O sistema deve apresentar facilidade de manutenção, possibilitando a modificação, substituição ou extensão de funcionalidades com impacto mínimo no restante da base de código. Essa característica deve facilitar correções, melhorias e evolução contínua do sistema.

5 Desenvolvimento do Trabalho

5.1 Visão de Arquitetura

A Visão de Arquitetura descreve a organização técnica do sistema CoCAD, apresentando como seus componentes principais são estruturados e como interagem para realizar o processo de geração de modelos CAD paramétricos condicionados a imagem, texto ou informações multimodais. Essa visão fornece uma ponte entre os requisitos definidos no Capítulo 4 e as decisões de implementação discutidas nas seções posteriores, permitindo compreender *o que* compõe o sistema e *como* seu funcionamento emerge da colaboração entre os módulos. Em particular, a arquitetura foi projetada para atender aos requisitos funcionais de entrada de imagem, texto e sinais multimodais (RF01, RF02, RF03), pré-processamento (RF04), geração CAD paramétrica (RF05), visualização 3D (RF06), exportação (RF07) e registro de histórico (RF08), bem como aos requisitos não funcionais de desempenho (RNF01), escalabilidade (RNF02), precisão e confiabilidade (RNF03), validade geométrica (RNF04), usabilidade (RNF05), portabilidade (RNF06), interoperabilidade (RNF07) e manutenibilidade (RNF08).

A arquitetura é descrita sob duas perspectivas complementares:

- **Modelagem Estática:** apresenta a decomposição estrutural do sistema em componentes, suas responsabilidades e relações de dependência. Essa visão enfatiza a organização modular e os papéis desempenhados por cada elemento da arquitetura, apoiando diretamente o requisito de manutenibilidade (RNF08), ao favorecer baixo acoplamento e alta coesão entre módulos.
- **Modelagem Dinâmica:** descreve o comportamento do sistema no tempo, detalhando o fluxo de mensagens entre os componentes durante a execução dos principais casos de uso. São utilizados diagramas de sequência da UML para ilustrar o processamento das diferentes modalidades de entrada e a geração do modelo CAD, evidenciando como os fluxos de interação implementam os requisitos funcionais de geração a partir de imagem, texto e combinação multimodal (RF01, RF02, RF03), visualização (RF06) e exportação (RF07), seguindo as metas de desempenho e confiabilidade (RNF01, RNF03).

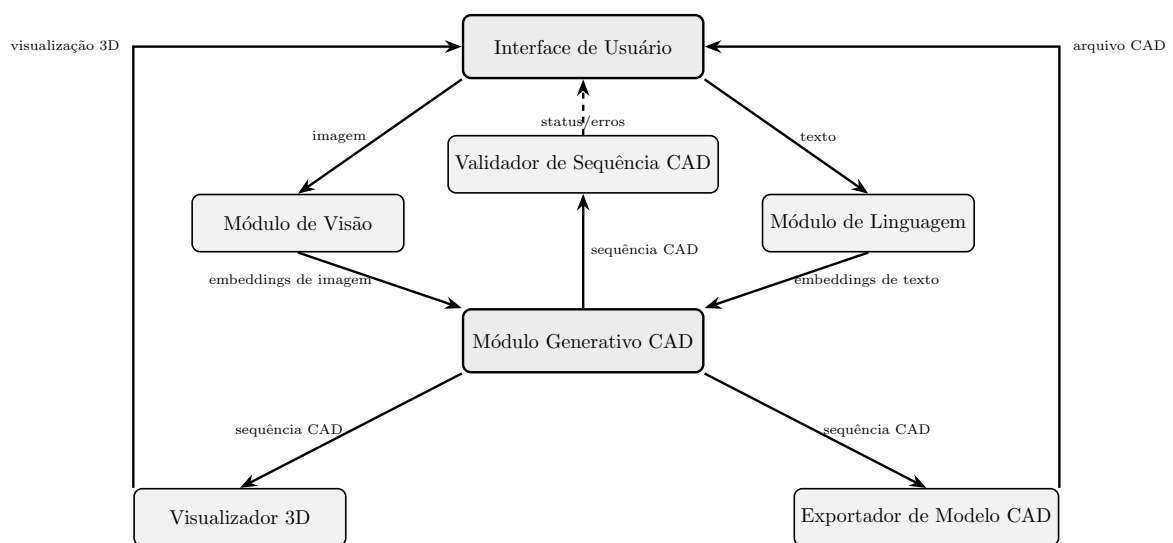
Em conjunto, essas duas modelagens fornecem uma compreensão abrangente da arquitetura do CoCAD, estabelecendo a base conceitual necessária para a análise das decisões de projeto, da implementação concreta e dos experimentos apresentados nas próximas seções.

5.1.1 Modelagem Estática

A modelagem estática descreve a decomposição estrutural do sistema CoCAD em componentes arquiteturais, suas responsabilidades e relações de dependência. Essa visão evidencia *o que* compõe o sistema, destacando sua organização em módulos coesos e de baixo acoplamento, sem ainda tratar dos fluxos de execução — abordados posteriormente na modelagem dinâmica. Essa decomposição em camadas e componentes especializados é uma decisão arquitetural voltada a garantir facilidade de evolução e manutenção do sistema (RNF08), bem como suporte à escalabilidade e à experimentação com diferentes variantes de modelos (RNF02).

O CoCAD é estruturado em cinco camadas principais: (1) Interface e Orquestração, (2) Processamento Multimodal, (3) Modelagem e Geração CAD, (4) Pós-processamento e Apresentação e (5) Persistência. A Figura 18 apresenta a organização geral desses componentes.

Figura 18 – Diagrama de Componentes do Sistema CoCAD.



Fonte: Elaborado pelo autor.

A seguir, descrevem-se os módulos presentes em cada camada.

1. Interface e Orquestração

Interface do Usuário. Responsável pela interação com o profissional de projeto. Permite o envio de imagem, texto ou ambas as modalidades, bem como a visualização e a exportação do modelo CAD gerado. Dessa forma, implementa diretamente os requisitos de entrada de imagem, texto e multimodalidade (RF01, RF02, RF03), visualização 3D do modelo (RF06) e exportação em formatos padronizados (RF07), ao mesmo tempo em que

busca respeitar o requisito de usabilidade, oferecendo uma interface simples e acessível a não especialistas em aprendizado de máquina (RNF05).

Serviço de Orquestração. Coordena o fluxo de processamento do sistema. Realiza validações iniciais, encaminha dados aos módulos especializados, aciona o pipeline de inferência, agrega resultados e organiza a resposta final apresentada ao usuário. Atua como intermediário entre a interface e os módulos de aprendizado de máquina. Essa centralização de controle contribui para o atendimento dos requisitos de desempenho (RNF01), ao evitar processamento redundante, e de manutenibilidade (RNF08), ao isolar a lógica de orquestração das implementações específicas dos modelos. Além disso, o serviço de orquestração registra informações relevantes das requisições, suportando o histórico de gerações (RF08).

2. Processamento Multimodal

Módulo de Processamento de Imagem. Executa normalização e extração de características visuais a partir de um *encoder* pré-treinado (DINOv2), produzindo vetores latentes adequados para a etapa de fusão multimodal. Esse módulo concretiza o requisito de pré-processamento das entradas visuais (RF04) e contribui para os requisitos de precisão e confiabilidade (RNF03), ao fornecer embeddings robustos e consistentes a partir das imagens recebidas (RF01).

Módulo de Processamento de Texto. Realiza tokenização e extração de representações semânticas com um modelo de linguagem pré-treinado (BERT), fornecendo informações textuais complementares às visuais. Esse módulo implementa o pré-processamento da entrada textual (RF04) para descrições em linguagem natural (RF02) e, assim como o módulo de imagem, apoia o atendimento ao requisito de precisão e confiabilidade (RNF03).

3. Modelagem e Geração CAD

Módulo de Fusão Multimodal. Combina embeddings visuais e textuais em um espaço latente conjunto, preservando relações semânticas e geométricas relevantes para a geração CAD. Esse módulo atende diretamente ao requisito de entrada multimodal (RF03), permitindo que o sistema integre informações de imagem e texto de forma coerente para orientar a geração, o que contribui para a qualidade dos resultados (RNF03).

Módulo Generativo CAD (Latent Flow Matching). A partir do vetor multimodal, produz um latente CAD condicionado utilizando uma rede baseada em *flow matching*. Esse latente sintetiza a intenção geométrica inferida a partir das entradas do usuário. O módulo é o núcleo da capacidade de geração CAD do sistema (RF05) e está diretamente associado aos requisitos não funcionais de precisão, confiabilidade (RNF03) e desempenho (RNF01), uma vez que sua eficiência e qualidade impactam o tempo de resposta e a taxa de gerações válidas.

Autoencoder CAD. Reconstrói a sequência paramétrica de comandos CAD. É composto por:

- **Decoder CAD:** utilizado na inferência para converter o latente gerado em uma sequência estruturada do tipo *sketch-extrude*;
- **Encoder CAD:** empregado na fase de treinamento para aprender representações latentes compactas e informativas.

Esse componente é responsável por materializar o requisito de geração CAD paramétrica estruturada (RF05), produzindo sequências de comandos compatíveis com ferramentas de CAD paramétrico, além de contribuir para a validade geométrica e editabilidade dos modelos gerados (RNF04).

4. Pós-processamento e Apresentação

Interpretador CAD. Converte a sequência paramétrica em um sólido tridimensional, aplicando as operações geométricas correspondentes. Esse módulo garante que as sequências geradas sejam transformadas em modelos efetivamente manipuláveis, apoiando os requisitos de validade geométrica (RNF04) e de visualização 3D (RF06).

Validador CAD. Verifica a consistência geométrica e topológica do modelo resultante, sem alterar a estrutura gerada. Essa verificação é fundamental para os requisitos de precisão e confiabilidade (RNF03) e para o requisito de validade geométrica (RNF04), ao identificar e filtrar modelos inconsistentes.

Visualizador 3D. Renderiza o modelo CAD para inspeção interativa, permitindo rotação, zoom e manipulação da cena. Esse componente implementa diretamente o requisito de visualização tridimensional do modelo CAD (RF06) e contribui para a usabilidade da solução (RNF05), ao fornecer feedback visual claro ao usuário.

Exportador de Arquivos CAD. Gera arquivos em formatos padronizados, assegurando interoperabilidade com plataformas CAD utilizadas na indústria. Assim, atende ao requisito funcional de exportação (RF07) e ao requisito de interoperabilidade (RNF07), permitindo que os modelos gerados sejam reutilizados em diferentes ferramentas de CAD.

5. Persistência

Repositório de Artefatos. Armazena pesos dos modelos, estatísticas, arquivos auxiliares e histórico das gerações realizadas, garantindo reprodutibilidade e suporte à evolução do sistema. Esse repositório está diretamente ligado ao requisito de histórico de geração (RF08), além de apoiar a escalabilidade e a experimentação contínua (RNF02) e a manutenibilidade (RNF08), ao facilitar a gestão de versões de modelos e configurações.

A modelagem estática estabelece, portanto, a estrutura fundamental do CoCAD, delimitando seus componentes e suas responsabilidades. Essa visão serve de base para a modelagem dinâmica (Subseção 5.1.2), que descreve como esses componentes interagem ao longo do fluxo de geração CAD para concretizar os requisitos funcionais (RF01–RF08) e apoiar o cumprimento dos requisitos não funcionais (RNF01–RNF08).

5.1.2 Modelagem Dinâmica

A modelagem dinâmica complementa a modelagem estática ao descrever o comportamento do sistema ao longo do tempo, enfatizando como os componentes interagem e como as mensagens fluem entre eles durante a execução dos principais casos de uso. Enquanto a modelagem estática define a estrutura e as responsabilidades de cada componente (Seção 5.1.1), a modelagem dinâmica evidencia *como* esses componentes colaboram operacionalmente para produzir o modelo CAD solicitado pelo usuário, implementando na prática os requisitos de geração a partir de imagem, texto e entradas multimodais (RF01, RF02, RF03), geração paramétrica (RF05), visualização (RF06), exportação (RF07) e registro de histórico (RF08), sob as restrições de desempenho, confiabilidade e validade geométrica (RNF01, RNF03, RNF04).

Para representar esses fluxos de interação, são utilizados **diagramas de sequência** da UML, que mostram passo a passo a troca de mensagens entre os componentes envolvidos — Interface do Usuário, Serviço de Orquestração, Módulos de Visão e de Linguagem, Módulo de Fusão Multimodal, Gerador CAD (flow matching), Decoder CAD, Interpretador CAD, Validador, Visualizador 3D e Exportador. Esses diagramas permitem visualizar o trajeto completo dos dados, desde a entrada fornecida pelo usuário até a geração, validação e exportação do modelo CAD paramétrico, em conformidade com os requisitos especificados no Capítulo 4.

A seguir, são apresentados os diagramas referentes aos três casos de uso principais do sistema CoCAD, conforme especificado na Seção 4.2.2.

Caso de Uso 1 — Geração de Modelo CAD a partir de Imagem

O primeiro diagrama ilustra o fluxo de execução quando o usuário fornece apenas uma imagem da peça (RF01). Após a validação do arquivo, o Módulo de Visão extrai características visuais (*embeddings*) utilizadas pelo Módulo Generativo CAD para prever o latente condicionado. Em seguida, o Decoder CAD reconstrói a sequência paramétrica *sketch-extrude* (RF05), que é convertida em geometria pelo Interpretador CAD. Por fim, o modelo é validado, exibido ao usuário (RF06) e oferecido para exportação (RF07).

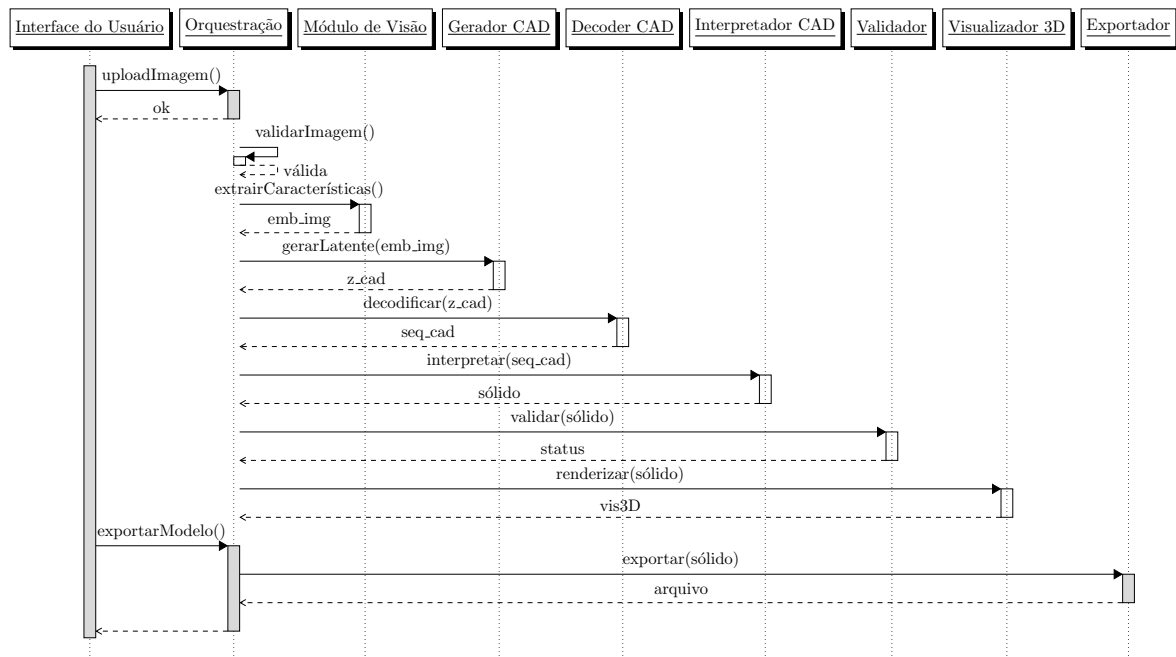


Figura 19 – Diagrama de sequência para geração de modelo CAD a partir de imagem.

Caso de Uso 2 — Geração de Modelo CAD a partir de Texto

O segundo diagrama descreve o comportamento do sistema quando a entrada consiste somente em uma descrição textual (RF02). Após a validação do texto, o Módulo de Linguagem processa a entrada e produz uma representação semântica (*embedding* textual). Essa representação alimenta o Módulo Generativo CAD, que produz o latente correspondente. Os componentes subsequentes executam os mesmos passos do fluxo anterior: decodificação da sequência CAD (RF05), interpretação geométrica, validação e visualização do resultado (RF06), seguida da possibilidade de exportação (RF07).

Caso de Uso 3 — Geração Multimodal (Imagem + Texto)

O terceiro diagrama apresenta o cenário multimodal, em que imagem e texto são fornecidos simultaneamente (RF03). Os módulos de Visão e de Linguagem processam suas entradas em paralelo, e suas representações são integradas pelo Módulo de Fusão Multimodal. O vetor combinado resultante alimenta o Módulo Generativo CAD, permitindo que o modelo incorpore tanto a geometria implícita na imagem quanto as especificações adicionais contidas no texto. O restante do fluxo — decodificação, interpretação geométrica, validação, visualização e exportação — segue o mesmo padrão dos casos anteriores (RF05, RF06, RF07).

Em conjunto, esses diagramas fornecem uma visão abrangente do comportamento dinâmico do CoCAD, evidenciando como os componentes definidos na modelagem estática interagem para atender aos requisitos funcionais especificados no Capítulo 4 (RF01–RF08).

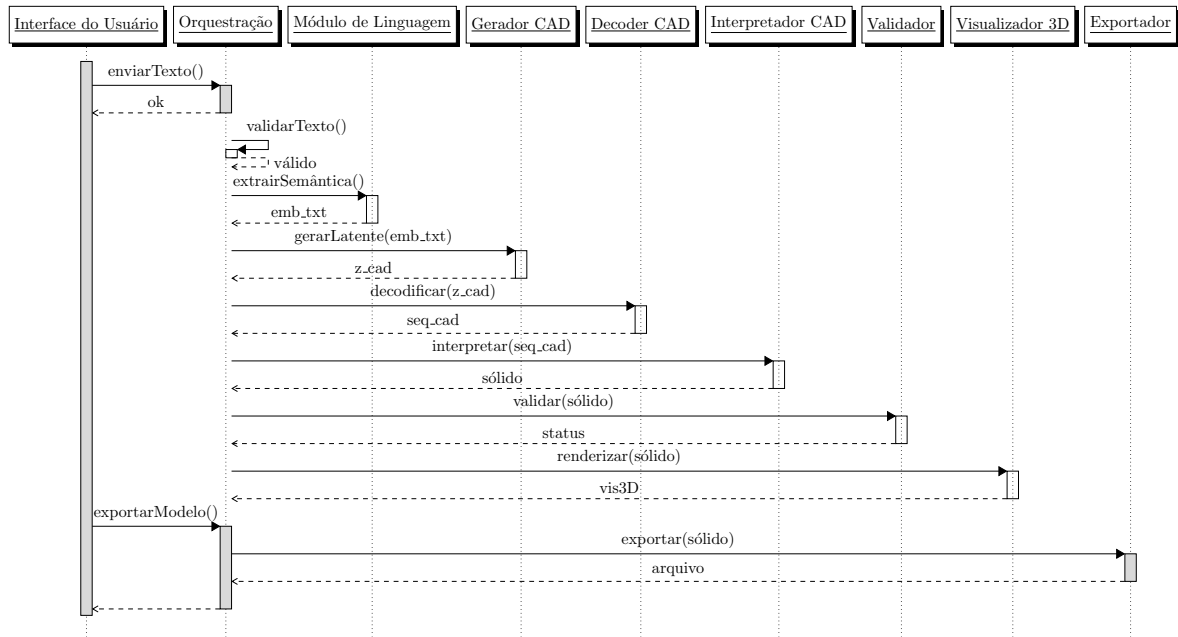


Figura 20 – Diagrama de sequência para geração de modelo CAD a partir de texto.

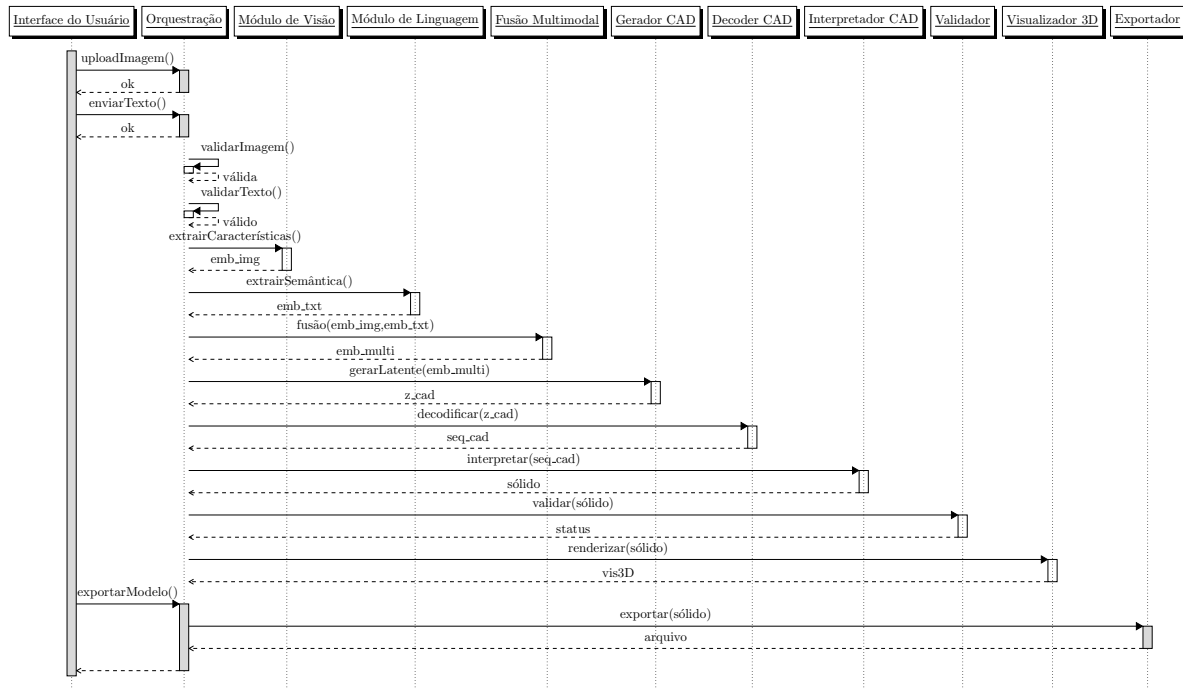


Figura 21 – Diagrama de sequência para geração multimodal de modelos CAD.

e apoiar o cumprimento dos requisitos não funcionais (RNF01–RNF08). Essa compreensão é fundamental para a próxima seção, dedicada à implementação detalhada dos módulos e das pipelines de treinamento e inferência.

5.2 Visão Tecnológica

Esta seção descreve o conjunto de tecnologias adotadas no desenvolvimento do sistema CoCAD, justificando sua seleção à luz dos requisitos funcionais e não funcionais estabelecidos no Capítulo 4. As tecnologias escolhidas suportam tanto o treinamento dos modelos de aprendizado profundo quanto a implementação da interface interativa e dos módulos responsáveis pela geração e validação de modelos CAD, em alinhamento especial com os requisitos de geração CAD paramétrica (RF05), entrada multimodal (RF01, RF02, RF03), pré-processamento (RF04), visualização e interação (RF06), exportação (RF07) e histórico (RF08), bem como com desempenho (RNF01), escalabilidade (RNF02), precisão e confiabilidade (RNF03), validade geométrica (RNF04), usabilidade (RNF05), portabilidade (RNF06), interoperabilidade (RNF07) e manutenibilidade (RNF08).

5.2.1 PyTorch e PyTorch Lightning

O **PyTorch** (PASZKE, 2019) constitui a base do desenvolvimento dos modelos de visão, linguagem, fusão multimodal e geração CAD. Sua flexibilidade para definição de arquiteturas personalizadas, manipulação de tensores e execução acelerada por GPU o torna adequado para sistemas de pesquisa que exigem controle fino sobre o fluxo computacional, como é o caso do CoCAD. Essa escolha tecnológica está diretamente relacionada às necessidades de desempenho (RNF01) e escalabilidade em cenários com centenas de milhares de amostras (RNF02), bem como à implementação dos módulos responsáveis pelo pré-processamento (RF04) e pela geração CAD condicionada (RF05).

Durante a fase de treinamento, o projeto faz uso extensivo do **PyTorch Lightning** (FALCON, 2019), um framework que organiza o código em estruturas modulares (*LightningModule* e *LightningDataModule*), separando de forma clara o modelo, os dados e o procedimento de treinamento. As principais vantagens de seu uso incluem:

- redução significativa de código repetitivo, permitindo foco na lógica do modelo;
- suporte nativo a múltiplas GPUs, *mixed precision* e callbacks de checkpointing, o que contribui diretamente para os requisitos de desempenho e escalabilidade do treinamento (RNF01, RNF02);
- maior reprodutibilidade experimental, alinhada aos requisitos de manutenção e evolução do sistema (RNF08) e ao registro e reexecução de experimentos associados ao histórico de gerações e resultados (RF08).

Apesar de Lightning ser utilizado no treinamento, a etapa de inferência integrada à interface interativa utiliza somente PyTorch puro, garantindo execução mais leve e direta

em ambiente de demonstração, em consonância com o requisito de desempenho em fluxo iterativo de design (RNF01).

5.2.2 Hydra para Gerenciamento de Configurações

O **Hydra** (YADAN, 2019) é empregado para o gerenciamento estruturado de configurações do sistema, permitindo organizar hiperparâmetros, caminhos de dados, versões de modelos, esquemas de treinamento e parâmetros de inferência em arquivos YAML hierárquicos. Essa abordagem facilita:

- rastreabilidade de experimentos;
- reprodução de resultados;
- variação controlada de configurações sem alterar o código-fonte.

Essas capacidades atendem diretamente aos requisitos de reprodutibilidade e histórico (RF08), bem como ao requisito de manutenibilidade (RNF08), ao permitir a evolução do sistema sem acoplamento excessivo entre código e configurações. Em um sistema multimodal como o CoCAD, em que diferentes componentes (visão, linguagem, fusão, gerador e decodificador CAD) possuem parâmetros próprios que devem ser sincronizados (RF01–RF05), o Hydra contribui ainda para a organização e escalabilidade das configurações (RNF02).

5.2.3 Modelos Pré-treinados: DINOv2 e BERT

O sistema utiliza dois modelos pré-treinados amplamente adotados em tarefas de visão e linguagem:

- **DINOv2 ViT-L/14** (OQUAB, 2023) para extração de características visuais robustas. Esse modelo fornece vetores de dimensão 1024, obtidos a partir de uma imagem pré-processada, compondo a entrada do módulo de fusão multimodal.
- **BERT-base-uncased** (DEVLIN, 2018) para extração de representações semânticas do texto. Suas embeddings capturam informações lexicais e estruturais utilizadas como condicionamento textual na geração CAD.

A adoção de modelos pré-treinados atende requisitos de desempenho e confiabilidade, reduzindo a necessidade de grandes volumes de dados rotulados e garantindo qualidade nas representações multimodais. Isso se relaciona diretamente ao pré-processamento de imagem e texto (RF04), à geração CAD condicionada a essas modalidades (RF01, RF02, RF03, RF05) e aos requisitos de precisão e confiabilidade das gerações (RNF03), ao fornecer embeddings estáveis e informativos sobre as entradas.

5.2.4 Ambiente de Treinamento: Google Colab

O treinamento dos modelos foi realizado no ambiente de desenvolvimento **Google Colab** (GOOGLE, 2024), que oferece GPUs acessíveis e configuração rápida para experimentação. Essa escolha atende aos requisitos de portabilidade e escalabilidade, permitindo:

- execução de múltiplos experimentos em GPUs modernas;
- integração com Google Drive para armazenamento de checkpoints e artefatos;
- facilidade de compartilhamento e reprodutibilidade do processo de treinamento.

Dessa forma, o uso do Google Colab está diretamente ligado aos requisitos de portabilidade (RNF06), escalabilidade de treinamento em cenários de dados volumosos (RNF02) e manutenibilidade e evolução do sistema (RNF08), ao facilitar experimentação iterativa e ajustes de arquitetura para melhorar desempenho (RNF01) e qualidade de geração (RNF03).

5.2.5 Hugging Face para Acesso a Dados e Modelos

A plataforma **Hugging Face** (WOLF, 2020) é utilizada para obter o dataset textual e visual do projeto Text2CAD, que disponibiliza pares contendo descrições e imagens de modelos CAD. Essa fonte é essencial para a construção do módulo textual e para experimentos de condicionamento multimodal.

O acesso é realizado via `hf_hub_download`, garantindo controle de versão e armazenamento local dos dados necessários. Essa abordagem favorece o atendimento aos requisitos funcionais de entrada de imagem e texto (RF01, RF02), de suporte a multimodalidade (RF03) e de pré-processamento consistente (RF04), além de contribuir para a escalabilidade do uso de dados (RNF02) e para o histórico e rastreabilidade de conjuntos de dados utilizados em treinamento e validação (RF08).

5.2.6 Ferramentas de Visualização e Gerenciamento de Treinamento

O CoCAD utiliza o **TensorBoard** (ABADI, 2015) para registro e visualização de métricas, perdas de treinamento, curvas de validação e estrutura dos modelos. Essa ferramenta atende aos requisitos de transparência e rastreabilidade, permitindo acompanhar o desempenho ao longo do treinamento.

Na prática, o uso do TensorBoard contribui para o monitoramento de métricas relacionadas à precisão, taxa de gerações válidas e estabilidade do treinamento (RNF03), bem como para o requisito de manutenção e evolução do sistema (RNF08), ao fornecer subsídios

para diagnosticar problemas, comparar versões de modelos e documentar experimentos em linha com o histórico do sistema (RF08).

5.2.7 OCP / CadQuery para Reconstrução e Exportação CAD

Para reconstrução geométrica e exportação de modelos, o sistema utiliza a biblioteca **cadquery-ocp**, que integra o mecanismo geométrico **OpenCascade (OCP)** (SAS, 2020). Ela permite converter sequências CAD geradas em sólidos editáveis, além de exportar em formatos padrão como STEP, atendendo aos requisitos de interoperabilidade do sistema.

Essas bibliotecas são fundamentais para garantir que as sequências paramétricas geradas (RF05) resultem em modelos CAD geometricamente válidos e adequados para edição em ferramentas de CAD paramétrico (RNF04). Além disso, o suporte a formatos abertos como STEP atende explicitamente ao requisito funcional de exportação (RF07) e ao requisito não funcional de interoperabilidade com diferentes plataformas CAD (RNF07).

5.2.8 Gradio para Interface Interativa

A interface do CoCAD é implementada em **Gradio** (ABID, 2019), escolhida por sua capacidade de integrar modelos PyTorch diretamente a uma interface web leve e interativa. Entre os benefícios, destacam-se:

- suporte nativo a entrada de imagem, texto e parâmetros ajustáveis;
- possibilidade de incorporar visualizadores 3D para inspeção do modelo CAD;
- rápida prototipação e acessibilidade via navegador, garantindo usabilidade e facilidade de teste por não especialistas.

O Gradio atende diretamente aos requisitos funcionais ligados à visualização e interação com o modelo gerado, permitindo o envio de imagem e texto pelo usuário (RF01, RF02, RF03), a apresentação do modelo CAD em um visualizador tridimensional (RF06) e a configuração de parâmetros de geração em um fluxo iterativo de design. Do ponto de vista de requisitos não funcionais, contribui fortemente para a usabilidade (RNF05), ao tornar a interface simples e acessível, e para a portabilidade (RNF06), ao viabilizar acesso via navegador sem necessidade de instalação complexa no lado do usuário.

5.2.9 Linguagem Python e Bibliotecas de Apoio

O desenvolvimento geral do sistema é realizado em **Python** (ROSSUM; DRAKE, 2009), utilizando bibliotecas amplamente adotadas:

- **NumPy** ([HARRIS, 2020](#)) e **Pandas** ([MCKINNEY, 2010](#)) para manipulação e análise de dados;
- **TorchVision** ([CONTRIBUTORS, 2016](#)) para pré-processamento de imagens;
- **Matplotlib** ([HUNTER, 2007](#)) para inspeções e depuração visual durante o treinamento.

Essas bibliotecas fornecem o ecossistema necessário para consolidar os módulos de aprendizado de máquina, processamento de dados e reconstrução CAD. Elas contribuem diretamente para a implementação eficiente do pré-processamento (RF04), para o desempenho e análise de treinamento (RNF01, RNF03) e para a manutenibilidade (RNF08), ao apoiar um fluxo de desenvolvimento baseado em ferramentas amplamente consolidadas e bem documentadas.

Em conjunto, as tecnologias apresentadas formam uma base sólida para implementar a arquitetura do CoCAD, permitindo desde o treinamento dos modelos de aprendizado profundo até a execução interativa da geração CAD multimodal. Cada ferramenta foi selecionada de modo a oferecer modularidade, robustez e compatibilidade com os requisitos de desempenho (RNF01), escalabilidade (RNF02), precisão e confiabilidade (RNF03), validade geométrica (RNF04), usabilidade (RNF05), portabilidade (RNF06) e interoperabilidade (RNF07), além de apoiar a manutenibilidade e evolução contínua do sistema (RNF08) e os requisitos funcionais de geração e uso prático dos modelos CAD (RF01–RF08).

5.3 Projeto e Implementação

Esta seção descreve em profundidade o projeto e a implementação do sistema CoCAD, abordando desde as escolhas arquiteturais para os módulos de aprendizado de máquina, passando pelo processo completo de treinamento e teste dos modelos, até a implementação final da interface interativa em Gradio. São apresentados também os detalhes do dataset utilizado, bem como as razões para as decisões e configurações adotadas.

5.3.1 Visão Geral da Arquitetura de Aprendizado de Máquina

A arquitetura principal do CoCAD é composta por três módulos, representados na Figura 22. Cada módulo desempenha um papel específico na tarefa de mapear entradas multimodais (imagem e texto) para um modelo CAD paramétrico gerado automaticamente.

Os três módulos são:

1. **Módulo 1 — Autoencoder de Sequências CAD:** Responsável por mapear uma sequência paramétrica de comandos CAD para um vetor latente denso de

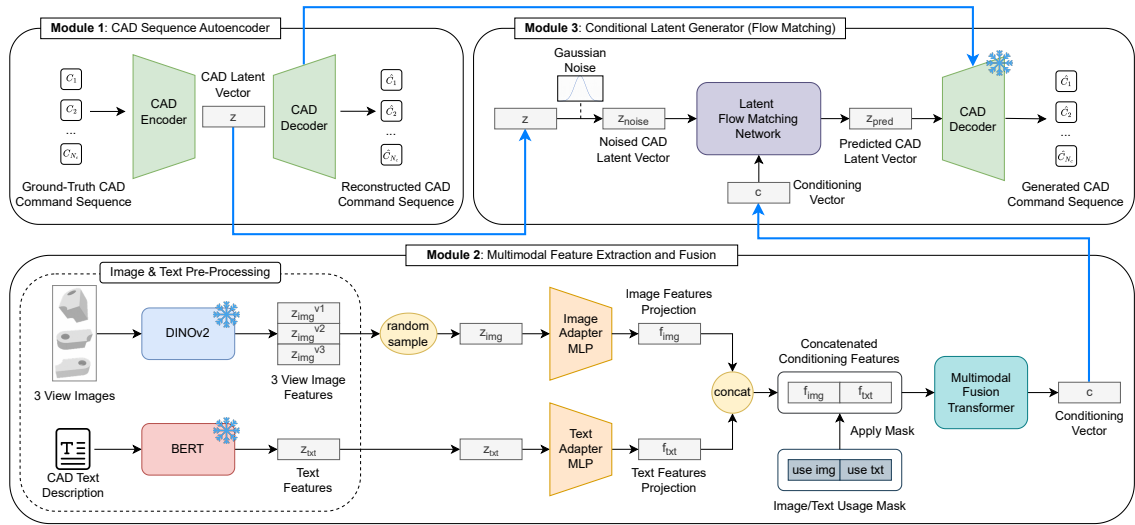


Figura 22 – Visão geral dos três módulos de aprendizado de máquina do CoCAD: (1) Autoencoder de Sequências CAD, (2) Extração e Fusão Multimodal, (3) Gerador Condicional de Latentes por Flow Matching.

dimensão fixa e, a partir dele, reconstruir a sequência original. Implementado como um Transformer Encoder–Decoder, este módulo define o *espaço latente estrutural* no qual as representações CAD são aprendidas. A qualidade deste espaço é crucial, pois todos os demais componentes dependem da sua estabilidade e expressividade.

2. **Módulo 2 — Extração e Fusão Multimodal**: Extrai *features* visuais e textuais usando DINOv2 (selecionando aleatoriamente uma entre três vistas disponíveis para robustez) e o modelo BERT-base, respectivamente. Em seguida, essas representações são projetadas por MLPs e combinadas por meio de um *Multimodal Fusion Transformer*, resultando em um vetor de condicionamento c que sintetiza a informação fornecida em linguagem natural e imagem.
3. **Módulo 3 — Gerador Condicional de Latentes (Flow Matching)**: A partir do vetor de condicionamento c , o módulo aplica um modelo de *flow matching* para transformar um vetor de ruído gaussiano z_{noise} em um vetor latente CAD z_{pred} . Esse módulo aprende campos vetoriais contínuos que aproximam a distribuição verdadeira de latentes produzida pelo autoencoder, permitindo gerar representações CAD compatíveis com o espaço aprendido no Módulo 1.

O Módulo 1 é treinado de forma independente, enquanto os Módulos 2 e 3 são treinados conjuntamente. Isso se deve ao fato de que os pesos dos MLPs e do *Multimodal Fusion Transformer* no Módulo 2 devem ser ajustados simultaneamente ao modelo de *flow matching* do Módulo 3, garantindo que o vetor de condicionamento c seja adequadamente informativo para guiar a geração dos latentes CAD.

Durante a inferência, os três módulos são integrados em sequência: a entrada multimodal é processada pelos Módulos 2 e 3 para gerar um vetor latente z_{pred} , que é então decodificado pelo Módulo 1 em uma sequência CAD completa. Esse processo é detalhado na Seção 5.3.6.

5.3.2 Conjunto de Dados Utilizado

O treinamento do CoCAD baseou-se em um conjunto extenso de modelos CAD e dados multimodais provenientes dos artigos **DeepCAD** (WU; XIAO; ZHENG, 2021) e **Text2CAD** (KHAN et al., 2024b). Esses dados incluem sequências paramétricas, imagens renderizadas e descrições textuais.

5.3.2.1 Modelos CAD do DeepCAD

O artigo DeepCAD disponibiliza **178.238 modelos CAD**, cada um representado como uma sequência de comandos estruturados (ver Seção 2.2.1). O dataset segue a divisão padrão adotada na literatura:

95% treino, 5% validação, 5% teste.

Essas sequências são utilizadas para treinar o autoencoder e estabelecer o espaço latente geométrico sobre o qual o modelo de *flow matching* opera.

5.3.2.2 Imagens e Textos do Text2CAD

O artigo Text2CAD fornece um conjunto multimodal disponível na plataforma *HuggingFace* (KHAN et al., 2024a), contendo representações visuais e textuais dos modelos CAD:

- **Imagens:** nove vistas renderizadas por modelo. Para este trabalho, foram selecionadas apenas **três vistas aleatórias** de cada modelo, reduzindo significativamente o espaço em disco e o custo de pré-processamento.
- **Descrições textuais:** várias descrições são fornecidas por modelo; utilizou-se apenas a coluna **description**, contendo textos curtos em linguagem natural. Essa escolha visa aproximar o conjunto de treinamento do tipo de entrada textual que o usuário realmente fornece na interface.

Após remoção de modelos inválidos, o *dataset* contém **176.020 modelos CAD**. Em seguida, aplicou-se um filtro adicional para manter somente modelos que possuísem simultaneamente imagens e descrições não vazias, totalizando **175.993 modelos CAD**. A Tabela 4 apresenta os valores finais por divisão:

Tabela 4 – Tamanhos das divisões do conjunto Text2CAD antes e após o filtro

Split	Original (Text2CAD)	Após filtro
Treino	159049	159026
Validação	8925	8922
Teste	8046	8045
Total	176020	175993

Apesar de cada modelo possuir três vistas selecionadas, utiliza-se **apenas uma vista aleatória durante o treinamento**. Essa decisão atende a duas motivações principais:

1. adequar-se ao cenário de uso da interface, em que o usuário fornece apenas uma imagem;
2. aumentar a robustez do modelo, já que a variação aleatória do ponto de vista reduz o risco de superajuste a uma vista específica e favorece a generalização para novos modelos.

5.3.3 Modelos e Arquiteturas dos Módulos de Aprendizado de Máquina

Esta subseção detalha as arquiteturas internas dos três módulos de aprendizado de máquina do CoCAD, conectando-as às formulações conceituais apresentadas no Capítulo 2, em particular às Seções 2.3.4 e 2.3.5. São descritos o autoencoder de sequências CAD (Módulo 1), o bloco de extração e fusão multimodal (Módulo 2) e o gerador condicional de latentes baseado em *flow matching* (Módulo 3), bem como as funções de perda, normalização de latentes e estratégias de condicionamento empregadas.

5.3.3.1 Módulo 1: Autoencoder de Sequências CAD

O primeiro módulo é um *Transformer* Encoder–Decoder treinado para reconstruir sequências de comandos CAD em uma representação discreta do tipo *DeepCAD*. Cada modelo é representado por uma sequência de comprimento máximo $N_c = 60$, onde cada passo contém um comando discreto e um vetor de parâmetros quantizados com $N_P = 16$ argumentos.

Arquitetura.

A sequência de entrada é composta por:

- um índice de comando $t_i \in \{0, \dots, N_{\text{cmd}} - 1\}$;

- um vetor de parâmetros discretizados $\mathbf{p}_i \in \{0, \dots, N_{\text{bins}} - 1\}^{N_P}$.

Os comandos são mapeados para um espaço de embedding de dimensão $d_{\text{model}} = 256$, enquanto os parâmetros são incorporados por meio de embeddings compartilhados sobre os bins quantizados. Além disso, são adicionadas:

1. **embeddings de posição**, que codificam a ordem sequencial dos comandos;
2. **embeddings de grupo**, que indicam a qual grupo geométrico (esboço, extrusão, fechamento, etc.) cada comando pertence, seguindo ideia semelhante à utilizada em GenCAD e CADCrafter. No CoCAD, utiliza-se um embedding de grupos com até 30 grupos distintos, o que permite capturar padrões estruturais de alto nível ao longo da sequência.

O *encoder* é composto por 4 camadas de *Transformer* com 8 cabeças de *self-attention* cada, dimensão intermediária (*feed-forward*) de 512 neurônios e *dropout* de 0,1 em atenção e camadas totalmente conectadas. Esses blocos produzem um embedding contextualizado da sequência completa; um *pooling* adequado (por exemplo, uso do token inicial ou média ponderada) produz o vetor latente $z \in \mathbb{R}^{256}$ que sintetiza a geometria e a estrutura paramétrica do modelo CAD. A dimensão do latente z coincide com d_{model} , o que simplifica a interface entre encoder e decoder.

O *decoder* recebe z e uma sequência alvo deslocada (*teacher forcing*), composta por comandos e parâmetros, e aplica também 4 camadas Transformer com 8 cabeças de atenção, dimensão intermediária de 512 e *dropout* 0,1, combinando autoatenção e atenção cruzada ao latente. As saídas são projeções lineares que produzem:

- logits sobre os comandos $\hat{t}_i$;
- logits sobre os parâmetros $\hat{\mathbf{p}}_{i,j}$.

Função de perda do autoencoder.

Seguindo a formulação do **DeepCAD** (WU; XIAO; ZHENG, 2021), a *loss* total do autoencoder combina dois termos de *cross-entropy*:

$$\mathcal{L}_{\text{AE}} = \sum_{i=1}^{N_c} \ell(\hat{t}_i, t_i) + \beta \sum_{i=1}^{N_c} \sum_{j=1}^{N_P} \ell(\hat{\mathbf{p}}_{i,j}, \mathbf{p}_{i,j}),$$

em que:

- ℓ é a *cross-entropy loss*;
- N_c é o número de comandos na sequência CAD;

- $N_P = 16$ é o número de parâmetros por comando;
- t_i e $\mathbf{p}_{i,j}$ são o comando e o j -ésimo parâmetro verdadeiros;
- $\hat{t}_i$ e $\hat{\mathbf{p}}_{i,j}$ são as previsões correspondentes;
- β é um hiperparâmetro que pondera a importância relativa dos parâmetros em relação aos comandos.

Neste trabalho, adotou-se $\beta = 2$, como em DeepCAD, enfatizando a reconstrução precisa dos parâmetros geométricos, que têm impacto direto na forma final do sólido CAD. Em termos de pesos de *loss*, isso equivale a atribuir peso 1,0 ao termo de comandos e peso 2,0 ao termo de parâmetros, refletindo a maior importância geométrica destes últimos.

5.3.3.2 Módulo 2: Extração e Fusão Multimodal

O segundo módulo tem como objetivo transformar as modalidades de entrada (imagem e texto) em um **vetor de condicionamento** c que possa guiar a geração de latentes CAD no Módulo 3. Este módulo combina *embeddings* pré-computados de DINOv2 (para imagens) e BERT-base (para textos), projetando-os para uma dimensão comum e fundindo-os via Transformer multimodal.

Pré-processamento multimodal.

Antes do treinamento do modelo de *flow matching*, é executada uma etapa de pré-processamento:

1. **Imagens:** para cada modelo CAD, o conjunto Text2CAD disponibiliza nove vistas renderizadas; selecionam-se apenas três vistas por modelo, por economia de armazenamento e custo computacional. Cada uma dessas vistas é codificada por um modelo DINOv2 ViT-L/14 (OQUAB, 2023; OQUAB et al., 2024), resultando em vetores $z_{\text{img}}^{v_i} \in \mathbb{R}^{1024}$, com $i \in \{1, 2, 3\}$.
2. **Textos:** utiliza-se o modelo BERT-base (`bert-base-uncased`) (DEVLIN, 2018) para codificar a coluna `description` do dataset, obtendo vetores $z_{\text{txt}} \in \mathbb{R}^{768}$ que sintetizam descrições curtas em linguagem natural. Durante o treinamento do CoCAD, os pesos do BERT são mantidos *congelados*, de forma que apenas os adaptadores e o bloco de fusão multimodal sejam treinados.

Esses *embeddings* são pré-calculados e armazenados, de forma que, durante o treinamento do Módulo 3, não seja necessário reexecutar DINOv2 e BERT a cada *batch*. Em termos práticos, os textos são truncados a um comprimento máximo de 128 tokens, o que equilibra custo computacional e capacidade de representação.

Durante o treinamento, **apenas uma vista aleatória** dentre as três disponíveis é amostrada para cada modelo. Essa escolha:

- reflete a configuração de inferência, em que o usuário tipicamente fornece apenas uma imagem;
- promove robustez a diferentes pontos de vista, reduzindo o risco de superajuste a uma vista específica.

Adaptação de dimensão e fusão.

Os vetores $z_{\text{img}} \in \mathbb{R}^{1024}$ e $z_{\text{txt}} \in \mathbb{R}^{768}$ são projetados para o mesmo espaço de dimensão $d_c = 256$ por meio de dois *Multi-Layer Perceptrons* (MLPs) independentes:

$$f_{\text{img}} = \phi_{\text{img}}(z_{\text{img}}) \in \mathbb{R}^{256}, \quad f_{\text{txt}} = \phi_{\text{txt}}(z_{\text{txt}}) \in \mathbb{R}^{256}.$$

Cada um desses MLPs utiliza dimensão intermediária compatível com 256 unidades e *dropout* de 0,1, alinhado ao restante da arquitetura, de forma a regularizar o condicionamento e evitar sobreajuste às características específicas de uma modalidade.

Esses vetores são então combinados por um *Multimodal Fusion Transformer* com 2 camadas e 4 cabeças de atenção, operando também com dimensão de modelo 256 e *dropout* 0,1. A sequência de entrada do Transformer é composta por:

- um token para a imagem (quando presente);
- um token para o texto (quando presente);
- tokens auxiliares que codificam a máscara de modalidades (`use_img`, `use_txt`).

Ao final da fusão, obtém-se um vetor de condicionamento:

$$c \in \mathbb{R}^{256},$$

que sintetiza, em um espaço compacto, a informação visual e textual disponível. Esse vetor c é repassado ao modelo de *flow matching* para condicionar o campo de velocidade v_θ (Seção 2.3.4).

Modality dropout.

Para que o CoCAD suporte os três modos de operação (apenas imagem, apenas texto, ou ambos), aplica-se **modality dropout** durante o treinamento. Em cada *batch*, escolhe-se entre:

$$p(\text{imagem}) = 0.15, \quad p(\text{texto}) = 0.15, \quad p(\text{imagem+texto}) = 0.70.$$

A máscara booleana (`use_img`, `use_txt`) indica quais modalidades estão ativas, permitindo que o mesmo modelo aprenda uma função de geração coerente para todos os modos. A escolha de utilizar o modo multimodal na maior parte das iterações (70%) decorre do fato de que:

- a combinação de imagem e texto fornece um condicionamento mais rico e menos ambíguo;
- esse é o cenário mais alinhado ao uso final da ferramenta, em que o usuário pode fornecer ambas modalidades;
- ainda assim, o treinamento ocasional com modalidades individuais garante robustez quando apenas uma fonte de informação está disponível.

5.3.3.3 Módulo 3: Gerador Condicional de Latentes por Flow Matching

O terceiro módulo implementa o modelo generativo condicional baseado em *flow matching*, conforme a formulação da Seção 2.3.4. Seu objetivo é aprender um campo de velocidade $v_\theta(z_t, t, c)$ que transporta uma distribuição inicial simples $p_0(z) = \mathcal{N}(0, I)$ até a distribuição de latentes CAD do autoencoder, condicionada pelo vetor c .

Normalização dos latentes

Antes de treinar o modelo de fluxo, os latentes do autoencoder $z \in \mathbb{R}^{256}$ são codificados para todos os exemplos do conjunto de treino, e calcula-se:

$$\mu = \mathbb{E}[z], \quad \sigma = \sqrt{\mathbb{E}[(z - \mu)^2]}.$$

Os latentes normalizados são então dados por:

$$\tilde{z} = \frac{z - \mu}{\sigma}.$$

Esses vetores normalizados são utilizados como alvo pelo modelo de *flow matching*. Essa normalização é uma prática comum em modelos generativos, pois estabiliza a distribuição de entrada, reduzindo diferenças de escala entre dimensões e aproximando a nuvem de pontos de uma distribuição aproximadamente esférica. Com isso, o campo de velocidade v_θ é treinado em um espaço mais homogêneo, o que facilita a otimização e tende a melhorar a qualidade das amostras geradas.

Arquitetura do campo de velocidade.

O campo $v_\theta(z_t, t, c)$ é implementado como uma MLP residual profunda, com camadas intermediárias de dimensão 512 e 8 blocos residuais empilhados. Cada bloco utiliza

dropout de 0,1 para regularização e incorpora **condicionamento FiLM** (Seção 2.3.5), no qual:

$$\gamma(c), \beta(c) \in \mathbb{R}^d$$

modulam as ativações internas h de forma afim:

$$\text{FiLM}(h, c) = \gamma(c) \odot h + \beta(c),$$

seguida de uma transformação não linear e conexão residual:

$$h' = h + f(\text{FiLM}(h, c)).$$

A dimensão do latente e do condicionamento neste módulo é mantida em 256 (isto é, $z_t, c \in \mathbb{R}^{256}$), garantindo compatibilidade direta com o espaço latente do autoencoder e com o vetor c produzido pelo módulo multimodal. Além disso, o tempo $t \in [0, 1]$ é incorporado por meio de um *time embedding* de dimensão 64, concatenado ou somado às ativações iniciais. Dessa forma, o modelo recebe como entrada (z_t, t, c) e retorna a velocidade estimada $v_\theta(z_t, t, c)$ no ponto intermediário z_t .

Treinamento com *flow matching* e *loss* auxiliar.

Seguindo a formulação da Seção 2.3.4, durante o treinamento amostram-se:

- um latente de ruído inicial $z_0 \sim \mathcal{N}(0, I)$;
- um latente de destino $\tilde{z}_{\text{true}}$ (normalizado) proveniente do autoencoder;
- um tempo $t \sim \mathcal{U}(0, 1)$;

e constrói-se um ponto intermediário na trajetória:

$$z_t = (1 - t)z_0 + t\tilde{z}_{\text{true}}.$$

O campo de velocidade alvo v_{target} é dado pela derivada da interpolação linear:

$$v_{\text{target}}(z_t, \tilde{z}_{\text{true}}) = \tilde{z}_{\text{true}} - z_0,$$

resultando na *loss* de *flow matching*:

$$\mathcal{L}_{\text{flow}} = \|v_\theta(z_t, c) - v_{\text{target}}(z_t, \tilde{z}_{\text{true}})\|^2.$$

Essa *loss* ensina o modelo a prever a *velocidade* correta da trajetória contínua que conecta o ruído inicial z_0 ao latente verdadeiro $\tilde{z}_{\text{true}}$, aproximando o fluxo determinístico que transforma p_0 em p_1 . Na implementação do CoCAD, o treinamento é discretizado

em 6 passos de tempo internos por amostra (*training num steps* = 6), o que fornece um compromisso entre custo computacional e qualidade da aproximação do fluxo contínuo.

Periodicamente, adiciona-se uma **loss auxiliar** que atua diretamente no latente final reconstruído. Após integrar numericamente o campo de velocidade e obter o latente gerado $\tilde{z}_{\text{pred}}$, aplica-se:

$$\mathcal{L}_{\text{aux}} = \|\tilde{z}_{\text{pred}} - \tilde{z}_{\text{true}}\|^2.$$

Enquanto $\mathcal{L}_{\text{flow}}$ foca em aproximar o campo de velocidades ao longo de toda a curva, $\mathcal{L}_{\text{aux}}$ reforça a *correspondência final* entre o latente gerado e o latente real. Na prática, essa *loss* auxiliar atua como termo estabilizador, reduzindo derivações ao longo do treinamento e acelerando a convergência, sobretudo nas primeiras épocas, quando o campo de velocidade ainda não está bem formado. No CoCAD, essa *loss* auxiliar é aplicada com peso λ_{aux} e é computada em intervalos regulares (a cada poucos *batches*, por exemplo a cada 4 *batches*), evitando sobrecarregar o custo de treinamento.

A *loss* total utilizada no treinamento do Módulo 3 é, portanto:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{flow}} + \lambda_{\text{aux}}\mathcal{L}_{\text{aux}},$$

em que λ_{aux} é um hiperparâmetro que pondera a importância da *loss* auxiliar (neste trabalho, $\lambda_{\text{aux}} = 0,05$ – valor escolhido empiricamente).

Durante o teste, o modelo gera latentes z_{pred} (após desnormalização) a partir de condicionamentos multimodais e esses vetores são comparados aos latentes verdadeiros z via *erro quadrático médio* (MSE) e similaridade do cosseno, como discutido na Seção 5.4. Em seguida, os latentes gerados são decodificados pelo *Transformer CAD Decoder* congelado, produzindo sequências CAD completas que podem ser avaliadas tanto estrutural quanto geometricamente.

5.3.4 Fluxo Completo de Treinamento e Teste

Embora os detalhes arquiteturais e das funções de perda tenham sido apresentados na Subseção 5.3.3, o processo de desenvolvimento do CoCAD seguiu uma **pipeline modular** bem definida, organizada em etapas sucessivas de pré-processamento, treinamento e avaliação. Esta subseção resume a ordem em que essas etapas foram executadas, destacando as dependências entre os módulos.

1. Pré-processamento multimodal

Inicialmente, são extraídos e armazenados os *embeddings* de imagem e texto:

- imagens renderizadas dos modelos CAD são codificadas por DINOv2 ViT-L/14 (OQUAB, 2023; OQUAB et al., 2024), produzindo vetores $z_{\text{img}}^{v_i} \in \mathbb{R}^{1024}$ para até três vistas por modelo;

- descrições textuais na coluna **description** são codificadas por BERT-base (DEVLIN, 2018), produzindo vetores $z_{\text{txt}} \in \mathbb{R}^{768}$.

Esses *embeddings* alimentam posteriormente o Módulo 2 de extração e fusão multimodal.

2. Treinamento do autoencoder de sequências CAD

Em seguida, treina-se o autoencoder Transformer (Módulo 1) para reconstruir sequências CAD do dataset DeepCAD a partir de latentes $z \in \mathbb{R}^{256}$. O modelo é otimizado para minimizar a *loss* de reconstrução $\mathcal{L}_{\text{AE}}$, combinando termos de *cross-entropy* sobre comandos e parâmetros. Essa etapa estabelece o espaço latente geométrico que será utilizado pelo modelo generativo.

3. Teste e avaliação do autoencoder

Após o treinamento, o autoencoder é avaliado reconstruindo sequências CAD de validação e teste, e comparando-as ao *ground truth* por meio de métricas estruturais e geométricas (command accuracy, parameter accuracy, Chamfer distance e *invalid ratio*), descritas na Seção 5.4.

4. Codificação das sequências CAD no espaço latente

Com o autoencoder treinado, todas as sequências CAD do conjunto são propagadas apenas pelo *encoder*, produzindo latentes $z \in \mathbb{R}^{256}$ por amostra. Esses vetores são armazenados e servem como alvo para o modelo de *flow matching*.

5. Cálculo das estatísticas dos latentes

A partir do conjunto completo de latentes de treino, calculam-se a média μ e o desvio padrão σ de cada dimensão. Essas estatísticas são utilizadas para normalizar os latentes e definir o espaço de treinamento do Módulo 3, conforme descrito na Subseção 5.3.3.

6. Treinamento do modelo de *flow matching* condicional

Em seguida, treina-se o modelo de *flow matching* (Módulo 3) em conjunto com os MLPs adaptadores e o *Multimodal Fusion Transformer* (Módulo 2). A cada *batch*, amostram-se:

- um latente de ruído inicial;
- um latente normalizado do autoencoder;
- um condicionamento c construído a partir de imagem, texto ou ambos, com modality dropout.

O treinamento segue a *loss* de *flow matching* combinada a uma *loss* auxiliar sobre o latente final, conforme detalhado na Subseção 5.3.3 e na Seção 2.3.4.

7. Teste do modelo de *flow matching*

Com o modelo treinado, realiza-se uma etapa de teste em que, para cada amostra de teste e combinação de modalidades, gera-se um latente z_{pred} a partir de ruído e do vetor de condicionamento. Esses latentes são comparados aos latentes verdadeiros do autoencoder utilizando métricas em espaço latente (MSE e similaridade do cosseno), apresentadas na Seção 5.4.

8. Decodificação e avaliação geométrica das sequências geradas

Finalmente, os latentes gerados são desnormalizados e decodificados pelo *Transformer CAD Decoder* congelado (Módulo 1), produzindo sequências CAD completas. Essas sequências são convertidas em sólidos CAD e avaliadas estruturalmente e geometricamente pelas mesmas métricas usadas na avaliação do autoencoder, permitindo comparar:

- a qualidade de reconstrução direta do autoencoder;
- a qualidade de geração condicional via *flow matching* a partir de imagem, texto ou ambos.

Esse fluxo modular garante que cada componente seja treinado sobre uma base sólida: primeiro se estabelece um espaço latente CAD estável e expressivo; depois, aprende-se um modelo generativo condicional capaz de navegar nesse espaço a partir de entradas multimodais; por fim, avalia-se o sistema de ponta a ponta, desde texto e imagem até a geometria 3D reconstruída.

5.3.5 Configurações e Estratégias de Treinamento dos Modelos

O treinamento do CoCAD envolve dois estágios principais: (i) aprendizado do autoencoder de sequências CAD (Módulo 1), responsável por definir o espaço latente estrutural, e (ii) aprendizado conjunto do modelo de *latent flow matching* com os módulos de condicionamento multimodal (Módulos 2 e 3). Embora ambos utilizem o otimizador AdamW, suas configurações de *learning rate*, *batch size*, agendadores e critérios de parada foram ajustadas de acordo com a natureza específica de cada tarefa.

5.3.5.1 Treinamento do Autoencoder de Sequências CAD

O autoencoder é treinado para reconstruir sequências de comandos CAD a partir de um vetor latente $z \in \mathbb{R}^{256}$, seguindo a função de perda descrita na Seção 2.2.1. Para lidar com o grande volume de dados e explorar eficientemente a paralelização em GPU, foi adotado um *batch size* de **256** sequências por iteração. Esse valor fornece um bom compromisso entre estabilidade da estimativa de gradiente e utilização da memória de GPU disponível.

O otimizador escolhido foi o **AdamW**, com parâmetros:

$$\text{lr} = 1 \times 10^{-4}, \quad \text{weight decay} = 0.01, \quad \beta_1 = 0.9, \quad \beta_2 = 0.95.$$

O uso de *weight decay* relativamente alto (0.01) ajuda a regularizar os pesos do Transformer e a mitigar overfitting em um dataset com mais de 150 mil sequências de treino, enquanto a escolha de $\beta_2 = 0.95$ torna o estimador de variância do gradiente ligeiramente mais responsivo a mudanças ao longo do treinamento.

O treinamento é executado com até **500 épocas** (`max_epochs = 500`), mas o uso de *early stopping* garante que o treinamento seja interrompido assim que a perda de validação deixe de melhorar de forma consistente. O critério de *early stopping* monitora a métrica `val/loss`, com:

$$\text{patience} = 10, \quad \text{min_delta} = 0.001,$$

em modo *min*. Ou seja, se durante 10 épocas consecutivas não houver melhora superior a 10^{-3} na perda de validação, o treinamento é encerrado, evitando gasto computacional desnecessário em um regime de ganhos marginais.

Para registro e reprodutibilidade, utiliza-se *checkpointing* da melhor época, com:

- salvamento do melhor modelo segundo `val/loss` (`save_top_k = 1`);
- salvamento adicional do último estado de treinamento (`save_last = true`).

Isso permite comparar o desempenho do melhor modelo de validação com o modelo no final do treinamento, além de facilitar eventuais retomadas.

O treinamento é realizado em **precisão simples** (32 bits), adequada para a estabilidade numérica do autoencoder, e as métricas são logadas a cada 50 *steps* (`log_every_n_steps = 50`), garantindo um acompanhamento suficientemente fino da curva de aprendizagem sem sobrecarregar o sistema de *logging*.

5.3.5.2 Treinamento do Modelo de *Latent Flow Matching* com Condicionamento Multimodal

O segundo estágio de treinamento envolve o modelo de *flow matching* (Módulo 3), treinado em conjunto com os componentes de condicionamento multimodal (Módulo 2), isto é, os MLPs adaptadores de imagem e texto e o *Multimodal Fusion Transformer*. Nesse estágio, o objetivo é aprender um campo de velocidade $v_\theta(z_t, t, c)$ que transporte vetores de ruído Gaussianos até os latentes CAD normalizados do autoencoder, conforme a formulação apresentada na Seção 2.3.4.

Como a entrada do modelo são vetores latentes pré-computados (e não sequências), é possível utilizar um *batch size* maior, definido como **384**. Esse aumento melhora a

eficiência da GPU e estabiliza a estimação do gradiente para o campo vetorial, sem exceder os limites de memória.

Também aqui é utilizado o otimizador **AdamW**, agora com:

$$\text{lr} = 2 \times 10^{-4}, \quad \text{weight decay} = 0.01, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999.$$

A *learning rate* ligeiramente maior reflete o fato de que o modelo de fluxo opera em um espaço latente de menor dimensão e pode se beneficiar de passos de otimização mais agressivos. O valor clássico $\beta_2 = 0,999$ é adequado para suavizar a variância do gradiente em modelos generativos, que tendem a apresentar superfícies de perda mais irregulares.

O treinamento é configurado com até **400 épocas** (`max_epochs = 400`), novamente com *early stopping* monitorando `val/loss`, mas com uma tolerância temporal maior:

$$\text{patience} = 40, \quad \text{min_delta} = 0.001.$$

Essa escolha se justifica pelo fato de que o campo de fluxo é uma função mais complexa e sua perda pode apresentar flutuações e platôs temporários; assim, é desejável permitir mais tempo para que o modelo se ajuste antes de encerrar o treinamento.

Para melhorar a estabilidade numérica em redes profundas condicionadas via FiLM (Seção 2.3.5), emprega-se **gradient clipping** com valor máximo de norma:

$$\text{gradient_clip_val} = 0.5,$$

evitando explosão de gradientes em passos isolados de treinamento.

O treinamento do *latent flow* é realizado em **precisão mista** (16-mixed), o que reduz o uso de memória e acelera o treinamento, mantendo estabilidade adequada graças ao uso de *scaling* automático de perda fornecido pela *framework* de treinamento.

Assim como no autoencoder, aplica-se *checkpointing* monitorando `val/loss`, salvando o melhor modelo e o último estado. As métricas são registradas a cada 50 passos de treinamento, com validação a cada duas épocas (`check_val_every_n_epoch = 2`), o que reduz o custo de validação em comparação ao autoencoder, sem sacrificar o acompanhamento da performance.

5.3.5.3 Learning Rate Schedulers

A escolha do *learning rate scheduler* influencia diretamente a estabilidade e a velocidade de convergência dos modelos. No CoCAD, foram empregados dois agendadores distintos, cada um alinhado às características do módulo correspondente.

Antes do regime principal de treinamento, ambos os modelos utilizam uma etapa inicial de *warmup*, na qual a *learning rate* cresce linearmente a partir de zero até o valor nominal. Essa prática é especialmente útil em arquiteturas baseadas em Transformers, pois

evita instabilidades numéricas nas primeiras iterações, quando os pesos ainda estão pouco calibrados. Assim como no trabalho original DeepCAD, adotou-se um *warmup* de **2000 training steps** para o autoencoder. Já para o modelo de *flow matching*, cuja dinâmica de otimização é mais sensível à escala inicial, utilizou-se um *warmup* mais longo, com **5000 training steps**.

Após o *warmup*, os modelos passam a seguir diferentes estratégias de agendamento:

- **Autoencoder:** utiliza *warmup + ReduceLROnPlateau*. Esse agendador reduz a *learning rate* quando a função de perda de validação deixa de melhorar, sendo adequado para tarefas de reconstrução estáveis e relativamente determinísticas. Como o autoencoder aprende uma função bem definida (mapear sequências CAD para sua forma reconstruída), sua perda tende a decrescer progressivamente, tornando o *ReduceLROnPlateau* uma escolha natural para refinar o aprendizado nas etapas finais.
- **Latent Flow Matching:** utiliza *warmup + cosine decay*. Esse agendador produz uma redução suave e contínua da *learning rate* ao longo do treinamento, característica desejável em modelos generativos condicionais. O *cosine decay* favorece uma exploração inicial mais ampla do espaço de parâmetros, seguida de uma convergência gradual, o que ajuda a estabilizar o campo vetorial aprendido pelo modelo de *flow matching*.

A escolha por *schedulers* distintos reflete, portanto, as diferenças fundamentais entre os módulos: o autoencoder é uma arquitetura de reconstrução supervisionada e estável, enquanto o modelo de *flow matching* é essencialmente generativo e se beneficia de uma taxa de aprendizado suavemente decrescente ao longo de todo o processo de otimização.

5.3.6 Pipeline de Inferência e Interface Interativa

A fase de inferência integra os três módulos do CoCAD em um pipeline único e contínuo, acessível ao usuário por meio de uma interface interativa desenvolvida em Gradio. Essa interface encapsula todo o processo de extração multimodal, geração de latentes e reconstrução CAD, permitindo a criação de modelos paramétricos a partir de uma imagem, de um texto descritivo ou da combinação de ambos.

Fluxo geral da inferência

O processo completo segue as seguintes etapas:

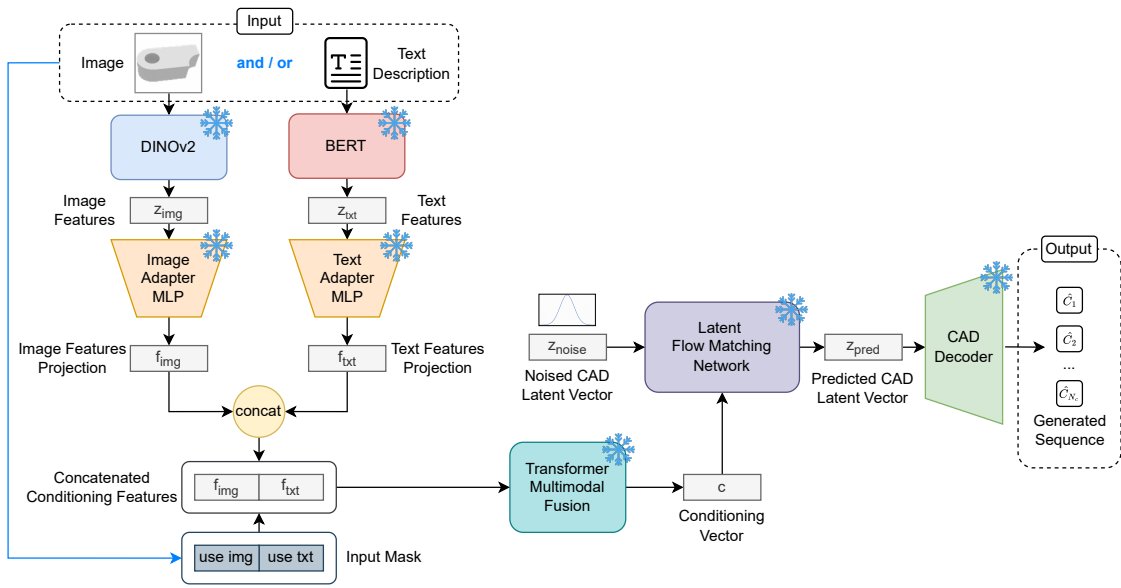


Figura 23 – Fluxo completo de inferência do CoCAD, desde o processamento multimodal até a geração final do sólido CAD.

1. **Processamento multimodal:** Se fornecida, a imagem é normalizada e processada por um extrator pré-treinado baseado em DINOv2, produzindo um vetor de *features* de dimensão 1024. Simultaneamente, o texto fornecido é processado pelo módulo de condicionamento textual, que utiliza um modelo do tipo BERT para gerar embeddings semânticos compactos.
2. **Fusão e condicionamento:** As representações visuais e textuais são projetadas para um espaço comum de dimensão 256 e fundidas por meio do *Multimodal Fusion Transformer*, resultando no vetor de condicionamento c . Esse vetor sintetiza a informação fornecida pelo usuário e serve como entrada para o modelo generativo.
3. **Geração do latente via *flow matching*:** O modelo de fluxo recebe c e integra numericamente a ODE aprendida durante o treinamento, produzindo um latente z_{pred} . O número de passos dessa integração pode ser escolhido pelo usuário, ajustando o grau de precisão da trajetória no espaço latente.
4. **Decodificação CAD:** O latente gerado é repassado ao decodificador do autoencoder, que reconstrói a sequência CAD discreta de comandos e parâmetros. Essa sequência representa o esboço paramétrico do modelo.
5. **Construção geométrica:** A sequência gerada é convertida em um sólido 3D utilizando uma biblioteca geométrica baseada em OpenCASCADE. Em seguida, o objeto é validado geometricamente e exportado para formatos STL e STEP, conforme o caso.

Interface interativa

A interface desenvolvida em Gradio materializa esse pipeline de forma acessível ao usuário. Ela oferece:

- campos para upload de imagem e inserção de texto;
- um controle deslizante para selecionar o número de passos na integração do *flow*;
- exibição em tempo real do modelo STL gerado;
- botão de download para o arquivo STEP;
- mensagens de verificação de validade geométrica.

Internamente, a interface organiza todo o pipeline de inferência, incluindo carregamento dos modelos, pré-processamento das entradas, construção de máscaras de modalidade, amostragem do latente, decodificação CAD e exportação dos arquivos. Dessa forma, o usuário interage apenas com uma camada intuitiva, enquanto a lógica de geração permanece totalmente automatizada.

A Figura 24 apresenta a interface interativa desenvolvida.

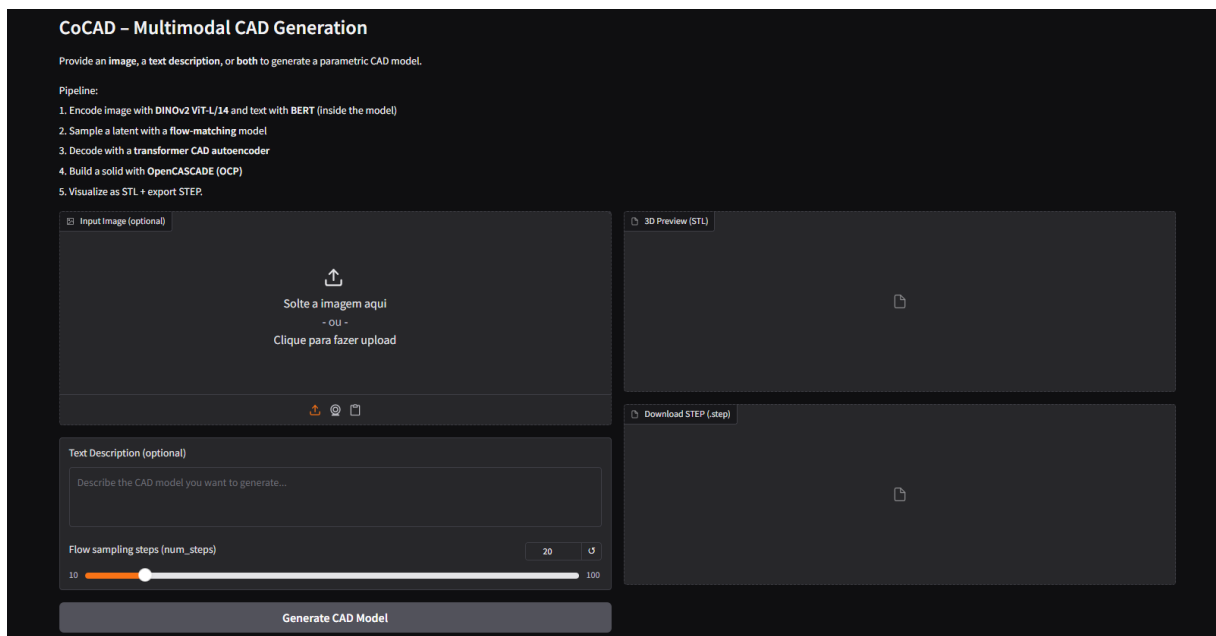


Figura 24 – Interface interativa do CoCAD implementada em Gradio.

5.4 Testes e Avaliação

Esta seção apresenta de forma detalhada o processo de avaliação do sistema CoCAD, incluindo as métricas utilizadas, os resultados obtidos em cada etapa intermediária e uma

análise crítica considerando reconstrução simbólica, reconstrução geométrica e consistência estrutural no espaço latente.

Esta seção apresenta de forma detalhada o processo de avaliação do sistema CoCAD, incluindo as métricas utilizadas, os resultados obtidos em cada etapa intermediária e uma análise crítica considerando reconstrução simbólica, reconstrução geométrica e consistência estrutural no espaço latente. Todas as métricas adotadas seguem rigorosamente os protocolos estabelecidos na literatura, em particular pelos trabalhos *DeepCAD* (WU; XIAO; ZHENG, 2021), *GenCAD* (ALAM; AHMED, 2025), *CADCrafter* (CHEN et al., 2025) e *Text2CAD* (KHAN et al., 2024b), garantindo comparabilidade direta com estudos prévios no domínio de modelagem paramétrica generativa.

A avaliação foi conduzida em três níveis distintos:

1. **Avaliação do Autoencoder:** mede a capacidade do modelo de reconstruir sequências CAD a partir de seus próprios vetores latentes.
2. **Avaliação do Flow Matching:** mede a capacidade do modelo de prever latentes próximos aos produzidos pelo autoencoder.
3. **Avaliação de Geração CAD:** mede a qualidade das sequências CAD produzidas pelo pipeline completo (imagem, texto ou ambos) em comparação com o ground truth.

As métricas utilizadas são descritas nas subseções seguintes.

5.4.1 Métricas Utilizadas

A avaliação combina métricas simbólicas e geométricas para medir a fidelidade estrutural do modelo CAD reconstruído, bem como métricas contínuas no espaço latente para avaliar o comportamento do modelo generativo.

5.4.1.1 Command Accuracy

A métrica de *command accuracy* segue exatamente a definição adotada em *DeepCAD* (WU; XIAO; ZHENG, 2021). Seja uma sequência CAD com N_c comandos verdadeiros t_i e comandos previstos $\hat{t}_i$. Define-se:

$$\text{ACC}_{\text{cmd}} = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbb{I}[t_i = \hat{t}_i],$$

em que $\mathbb{I}[\cdot]$ é a função indicadora, que vale 1 quando a condição é verdadeira e 0 caso contrário.

Essa métrica quantifica a porcentagem de comandos corretamente recuperados e, portanto, avalia a fidelidade estrutural do *esqueleto* da sequência CAD (tipos de operações, ordem dos comandos, presença correta de extrusões, esboços, fechamentos, etc.).

5.4.1.2 Parameter Accuracy

A *parameter accuracy* também segue a definição oficial do *benchmark* DeepCAD, incorporando uma tolerância para os parâmetros geométricos quantizados. Seja N_c o número de comandos na sequência, $\mathbf{p}_i$ o vetor de parâmetros verdadeiros do comando i e $\hat{\mathbf{p}}_i$ o vetor previsto correspondente. Denotando por $\mathbf{p}_{i,j}$ e $\hat{\mathbf{p}}_{i,j}$ o j -ésimo parâmetro verdadeiro e previsto, define-se:

$$\text{ACC}_{\text{param}} = \frac{1}{K} \sum_{i=1}^{N_c} \sum_{j=1}^{|\hat{\mathbf{p}}_i|} \mathbb{I} \left[\left| \mathbf{p}_{i,j} - \hat{\mathbf{p}}_{i,j} \right| < \eta \right] \mathbb{I} [t_i = \hat{t}_i],$$

onde

$$K = \sum_{i=1}^{N_c} \mathbb{I} [t_i = \hat{t}_i] |\mathbf{p}_i|$$

é o número total de parâmetros considerados (apenas para comandos cujo tipo foi corretamente previsto).

Tanto $\mathbf{p}_{i,j}$ quanto $\hat{\mathbf{p}}_{i,j}$ são valores inteiros resultantes da quantização em 8 bits (256 níveis), conforme a representação DeepCAD. O limiar η é escolhido para compensar os efeitos dessa quantização; na prática, utiliza-se $\eta = 3$, isto é, diferenças menores que três níveis quantizados ainda são consideradas acertos. Dessa forma, a métrica foca na reconstrução geométrica *coerente* dos parâmetros, sem penalizar discretizações pequenas que não afetam de maneira significativa a forma final do sólido.

5.4.1.3 Chamfer Distance (Mediana)

A avaliação geométrica compara nuvens de pontos do modelo CAD reconstruído com o modelo de referência. A *Chamfer distance* (CD) bidirecional entre dois conjuntos de pontos X e Y é definida por:

$$d_{\text{CD}}(X, Y) = \frac{1}{|X|} \sum_{x \in X} \min_{y \in Y} \|x - y\|^2 + \frac{1}{|Y|} \sum_{y \in Y} \min_{x \in X} \|y - x\|^2.$$

Para o conjunto de teste, calcula-se essa distância para cada par (reconstrução, *ground truth*), e reporta-se a **mediana** das *Chamfer distances*, métrica mais robusta a outliers do que a média simples. Esse indicador resume a qualidade geométrica típica das reconstruções, penalizando descontinuidades, distorções e erros grossos na forma do sólido.

5.4.1.4 Invalid Ratio

Nem toda sequência CAD discretizada resulta em um sólido geométrica e topologicamente válido quando reconstruída por meio do kernel geométrico (OpenCASCADE).

Define-se, então, a métrica:

$$\text{Invalid Ratio} = 1 - \frac{\#\text{válidos}}{\#\text{totais}},$$

onde $\#\text{válidos}$ é o número de modelos cuja reconstrução CAD foi bem-sucedida (sem erros de *B-Rep*) e $\#\text{totais}$ é o número total de amostras avaliadas.

Essa métrica captura diretamente a robustez geométrica do sistema: valores baixos indicam que a maior parte das sequências geradas corresponde a sólidos utilizáveis em um ambiente CAD real.

5.4.1.5 MSE e Cosine Similarity entre Latentes

Para avaliar o modelo de *flow matching*, compara-se o latente previsto z_{pred} com o latente verdadeiro z_{true} produzido pelo autoencoder (após normalização adequada). São empregadas duas métricas complementares:

- **Erro quadrático médio (MSE):**

$$\text{MSE} = \|z_{\text{pred}} - z_{\text{true}}\|^2,$$

que mede a discrepância em magnitude entre os vetores no espaço latente.

- **Similaridade do cosseno:**

$$\cos(\theta) = \frac{z_{\text{pred}} \cdot z_{\text{true}}}{\|z_{\text{pred}}\| \|z_{\text{true}}\|},$$

que avalia o alinhamento direcional entre os dois vetores, independentemente de sua norma.

Enquanto a MSE é sensível a diferenças de escala, a similaridade do cosseno captura a coerência direcional entre latentes, aspecto importante em espaços onde a direção codifica a semântica e a norma pode variar por efeitos de normalização ou treinamento. Em conjunto, essas duas métricas fornecem uma visão mais completa da qualidade do espaço latente aprendido pelo *flow*.

5.4.2 Avaliação 1: Reconstrução CAD pelo Autoencoder

Após treinado, o autoencoder foi testado no conjunto de teste contendo 8045 modelos CAD. Os resultados obtidos para o CoCAD são apresentados na Tabela 5, seguindo exatamente as métricas adotadas na literatura, conforme descrito na Subsecção 5.4.1.

Para contextualizar o desempenho, a Tabela 6 apresenta os valores médios reportados em DeepCAD (WU; XIAO; ZHENG, 2021) e GenCAD (ALAM; AHMED, 2025) comparados aos resultados obtidos pelo CoCAD.

Tabela 5 – Resultados de reconstrução CAD do autoencoder do CoCAD.

Métrica	Valor
ACC_{cmd}	98,94%
ACC_{param}	96,15%
Med. CD	$0,886 \times 10^{-3}$
IR	4,81%

Tabela 6 – Comparação da reconstrução CAD com métodos da literatura.

Método	$ACC_{cmd} \uparrow$	$ACC_{param} \uparrow$	Med. CD ($\times 10^{-3}$) $\downarrow$	IR $\downarrow$
DeepCAD	99,36%	97,59%	0,783	3,44%
GenCAD	99,51%	97,78%	0,762	3,32%
CoCAD	98,94%	96,15%	0,886	4,81%

Análise

Os resultados mostram que o autoencoder do CoCAD aprende de forma muito eficaz a estrutura simbólica e paramétrica das sequências CAD, As métricas $ACC_{cmd} = 98,94\%$ e $ACC_{param} = 96,15\%$ indicam que a arquitetura Transformer captura adequadamente tanto a sintaxe quanto a geometria implícita nos comandos, reconstruindo quase integralmente os modelos do conjunto de teste.

A análise geométrica reforça a qualidade da reconstrução: a

$$\text{Mediana da Chamfer Distance} = 0.886 \times 10^{-3}$$

demonstra alta fidelidade tridimensional.

Quanto à robustez geométrica, o **Invalid Ratio de 4,81%** situa-se ligeiramente acima dos valores reportados por DeepCAD (3,44%) e GenCAD (3,32%). Essa diferença, embora pequena, é esperada e metodologicamente justificável. Primeiramente, aqueles modelos foram treinados com **maior número de épocas, tuning extensivo de hiper-parâmetros e grande foco exclusivo na tarefa de reconstrução**. Já no CoCAD, o autoencoder foi treinado por aproximadamente **245 épocas**, sem otimização fina, uma vez que seu objetivo principal não era atingir o estado da arte isoladamente, mas **fornecer um espaço latente estável e semântico** para o módulo generativo multimodal (flow matching).

Além disso:

- DeepCAD e GenCAD dedicam toda a modelagem a maximizar a reconstrução simbólica e geométrica, enquanto no CoCAD o autoencoder se integra a um sistema maior, servindo como backbone para condicionamento e geração multimodal;

- o espaço latente do CoCAD é pressionado a manter estrutura adicional que facilite o aprendizado do modelo generativo, o que pode introduzir leve degradação no IR final;
- o tempo reduzido para ajuste fino (especialmente escolhas de *dropout*, *heads*, *warmup* e *plateau scheduling*) limitou margens de otimização que DeepCAD e GenCAD aplicam extensivamente.

No conjunto, apesar de treinado por menos tempo e sem *hyperparameter tuning* profundo, o autoencoder do CoCAD apresenta desempenho competitivo com os melhores modelos da literatura. A pequena diferença na taxa de invalidez é compatível com as diferenças de metodologia e com o fato de o CoCAD priorizar a integração harmoniosa entre reconstrução CAD e geração condicionada multimodal. O módulo cumpre, portanto, seu papel central com alta qualidade: fornecer um espaço latente coerente, preciso e estruturado para o pipeline gerativo completo.

5.4.3 Avaliação 2: Similaridade entre Latentes Gerados e Latentes Reais

A Tabela 7 apresenta os resultados da etapa intermediária de avaliação do modelo de *flow matching*, quantificando o quão próximos os latentes previstos (z_{pred}) estão dos latentes reais produzidos pelo autoencoder (z_{true}). Conforme discutido na Seção 5.3.3, esses resultados refletem o uso de um termo auxiliar de perda com peso $\lambda_{\text{aux}} = 0,05$.

Foram testados os valores $\{0,0 ; 0,05 ; 0,1\}$ para esse hiperparâmetro, todos suficientemente pequenos para não comprometer a capacidade do modelo em aprender o campo vetorial subjacente ao espaço latente. O valor $\lambda_{\text{aux}} = 0,05$ mostrou-se o mais eficaz, proporcionando melhor qualidade de previsão latente ao mesmo tempo em que preservou a estabilidade do treinamento.

A avaliação utiliza duas métricas contínuas no espaço latente: o erro quadrático médio (MSE), que mede discrepâncias absolutas de magnitude, e a similaridade do cosseno, que avalia o alinhamento direcional entre z_{pred} e z_{true} . Ambas fornecem perspectivas complementares sobre a capacidade do *flow* de capturar a estrutura geométrica do espaço latente aprendido pelo autoencoder.

Tabela 7 – Similaridade entre latentes reais e previstos pelo flow matching.

Modo	MSE ↓	Cosine Similarity ↑
Imagem + Texto	0,0938	0,4602
Somente Imagem	0,0992	0,4338
Somente Texto	0,1068	0,3900

Análise

Os resultados revelam um comportamento consistente com a natureza multimodal do sistema:

- O modo **imagem + texto** fornece os melhores valores tanto de MSE quanto de similaridade do cosseno, evidenciando que a combinação das modalidades produz um vetor de condicionamento mais informativo.
- O modo **somente imagem** supera claramente o modo **somente texto**, o que é esperado, pois as imagens capturam de forma mais direta a estrutura geométrica do objeto, enquanto descrições curtas em linguagem natural tendem a omitir detalhes finos.
- A similaridade do cosseno entre 0.39 e 0.46 sugere que o espaço latente aprendido pelo autoencoder é complexo e de alta dimensionalidade. O modelo de *flow matching* consegue capturar parte relevante dessa estrutura, aproximando-se do latente real, mas sem alinhamento perfeito — comportamento típico em fluxos condicionados operando em espaços latentes densos e altamente estruturados.

No conjunto, os resultados mostram que o modelo de flow matching internaliza bem as regularidades do espaço latente, especialmente quando dispõe de condicionamento multimodal, preparando terreno para uma etapa final de decodificação simbólica e geométrica coerente.

5.4.4 Avaliação 3: Geração de Sequências CAD a partir de Latentes Previstos

A etapa final de avaliação consiste em executar o pipeline completo de inferência do CoCAD — desde a entrada multimodal até a geração do sólido CAD:

$$\text{imagem/texto} \longrightarrow c \longrightarrow z_{\text{pred}} \longrightarrow \text{decoder CAD} \longrightarrow \text{sequência CAD}.$$

Diferentemente das Avaliações 1 e 2, que medem capacidades internas dos módulos, esta avaliação captura o desempenho real do sistema *end-to-end*. Os resultados obtidos nos três modos de inferência (imagem, texto e multimodal) são comparados com métodos especializados da literatura: CADCrafter (CHEN et al., 2025) (imagem para CAD) e Text2CAD (KHAN et al., 2024b) (texto para CAD). A Tabela 8 resume os resultados.

Análise

Desempenho multimodal do CoCAD.

O modo **imagem + texto** supera claramente os modos unilaterais em todas as métricas. Este resultado confirma que a fusão multimodal fornece um vetor de condiciona-

Tabela 8 – Comparação da geração CAD com CoCAD, CADCrafter e Text2CAD.

Método / Modo	ACC _{cmd} ↑	ACC _{param} ↑	Med. CD ↓	IR ↓
CoCAD — Somente Imagem	71,21%	54,27%	0,1939	13,13%
CoCAD — Somente Texto	66,79%	50,72%	0,1885	14,43%
CoCAD — Imagem + Texto	73,89%	56,82%	0,1554	12,53%
CADCrafter (Imagem, single-view)	83,23%	71,82%	0,049	4,20%
Text2CAD (Texto, nível L3 expert)	—	—	0,000370	0,93%

mento mais informativo e reduz ambiguidades inerentes às modalidades individuais. Os valores de $ACC_{cmd} = 73,89\%$ e $ACC_{param} = 56,82\%$ representam um desempenho sólido para um modelo capaz de operar simultaneamente em três modos distintos, enquanto a mediana da Chamfer Distance permanece em uma faixa entre 0.15 e 0.19, indicando reconstruções geometricamente plausíveis, embora ainda inferiores ao nível do autoencoder.

Adicionalmente, os valores de Invalid Ratio obtidos nos três modos — 12,53% (multimodal), 13,13% (somente imagem) e 14,43% (somente texto) — indicam que o CoCAD gera mais de 85% de modelos válidos no conjunto de teste, atendendo diretamente ao requisito não funcional **RNF03 (Precisão e Confiabilidade)**, que estabelece a necessidade de manter a taxa de gerações válidas acima desse limiar.

Comparação com CADCrafter.

O CADCrafter apresenta desempenho superior (especialmente na *Chamfer Distance* e nas precisões simbólicas) mas isso é esperado devido a diferenças substanciais nos objetivos e no regime de treinamento:

- O CADCrafter é otimizado **exclusivamente para o modo “imagem → CAD”**, enquanto o CoCAD deve atender simultaneamente a três modos de entrada.
- O CADCrafter utiliza *multi-view to single-view distillation*, enquanto o CoCAD utiliza apenas **uma vista aleatória por modelo** durante o treinamento.
- O CADCrafter aplica uma etapa adicional de ***fine-tuning* via DPO (*Direct Preference Optimization*)**, técnica ausente no CoCAD.

Assim, o desempenho menor do CoCAD não indica limitação estrutural, mas sim uma diferença nos objetivos: enquanto CADCrafter é um pipeline altamente especializado para visão computacional, o CoCAD busca generalidade multimodal.

Comparação com Text2CAD.

Os resultados do Text2CAD dizem respeito apenas à métrica geométrica, pois o modelo não produz sequências discretizadas no formato DeepCAD. Ainda assim, a comparação é instrutiva:

- A Chamfer Distance do Text2CAD (0.000370) é **substancialmente melhor** que a do CoCAD no modo multimodal (0.1554). Isso reflete o fato de que o Text2CAD é um modelo especializado em *text-to-CAD*, com arquitetura e representação otimizadas especificamente para a modalidade textual.
- O Text2CAD utiliza descrições **expert-level (nível L3)**, ricas e detalhadas, enquanto o CoCAD foi treinado com descrições curtas e abstratas extraídas diretamente do dataset, carregando bem menos informação geométrica.

Portanto, embora opere em um cenário mais limitado (descrições curtas e não especializadas) e sem representação simbólica dedicada, o CoCAD ainda produz resultados geometricamente plausíveis. A diferença para o Text2CAD é esperada e reforça que modelos altamente especializados em texto tendem a capturar melhor os detalhes geométricos finos.

Conclusão da Avaliação 3.

Os resultados demonstram que:

- o CoCAD mantém desempenho consistente nos três modos de operação (*somente imagem, somente texto e multimodal*);
- o modo **imagem + texto** apresenta desempenho superior em todas as métricas analisadas, confirmando que a fusão multimodal fornece um condicionamento mais informativo;
- métodos especializados, como CADCrafter (otimizado exclusivamente para imagem) e Text2CAD (otimizado exclusivamente para texto), atingem valores melhores em seus modos específicos, o que é esperado, pois realizam *fine-tuning* direcionado, técnicas de *distillation*, ajustes extensivos de hiperparâmetros e utilizam representações adaptadas às suas respectivas tarefas;
- o CoCAD, por outro lado, deve equilibrar simultaneamente três pipelines distintos dentro de um único modelo generativo, sem especialização por modalidade e sem etapas avançadas de refinamento.

No conjunto, a Avaliação 3 confirma que o CoCAD alcança resultados competitivos no cenário multimodal, mesmo operando em condições mais desafiadoras que os modelos especializados da literatura.

5.4.5 Teste Funcional da Interface de Geração CAD

Além das avaliações quantitativas apresentadas nas seções anteriores, realizou-se também um teste funcional da interface interativa desenvolvida em Gradio, com o objetivo de verificar a operação integrada de todos os módulos do sistema: codificação multimodal, amostragem no espaço latente, decodificação CAD e renderização tridimensional. Esse teste é particularmente relevante pois avalia o sistema no contexto final de uso, sob condições semelhantes às de um cenário real de interação com o usuário.

O teste foi executado a partir de um ambiente em nuvem no *Google Colab*, demonstrando a portabilidade da solução e a viabilidade de operação remota. Apesar de envolver componentes de visão computacional, processamento textual, fluxo latente e decodificação geométrica, o tempo total de geração do modelo foi de **poucos segundos**, atendendo plenamente ao requisito de desempenho (RNF01) e evidenciando a adequação do sistema para ciclos iterativos rápidos de design.

Para a avaliação, selecionou-se um modelo pertencente ao *split* de teste, isto é, um modelo **não visto pelo sistema durante o treinamento**. A imagem correspondente foi carregada na interface, juntamente com uma descrição textual opcional, explorando assim as modalidades previstas nos requisitos RF01 (entrada de imagem), RF02 (entrada de texto) e RF03 (entrada multimodal). Em seguida, a interface acionou automaticamente o pipeline completo: pré-processamento multimodal (RF04), amostragem do vetor latente com o módulo de *flow matching*, decodificação paramétrica CAD (RF05), construção geométrica via *cadlib* e OpenCASCADE, visualização tridimensional em STL (RF06) e disponibilização do arquivo STEP para exportação (RF07).

A Figura 25 apresenta a interface durante esse processo, exibindo (i) a imagem utilizada como entrada, (ii) o campo de descrição textual, e (iii) a pré-visualização tridimensional do sólido CAD gerado. Esse teste demonstrou que a interface é simples e intuitiva de utilizar (RNF05), confirma a compatibilidade com diferentes ambientes computacionais, incluindo execução em nuvem (RNF06), e utiliza formatos abertos amplamente aceitos na indústria, como STL e STEP, garantindo interoperabilidade (RNF07).

Portanto, o teste funcional da interface evidencia que tanto os requisitos funcionais quanto os não funcionais associados à interação com o usuário foram adequadamente atendidos, comprovando a efetividade da solução integrada do sistema CoCAD.

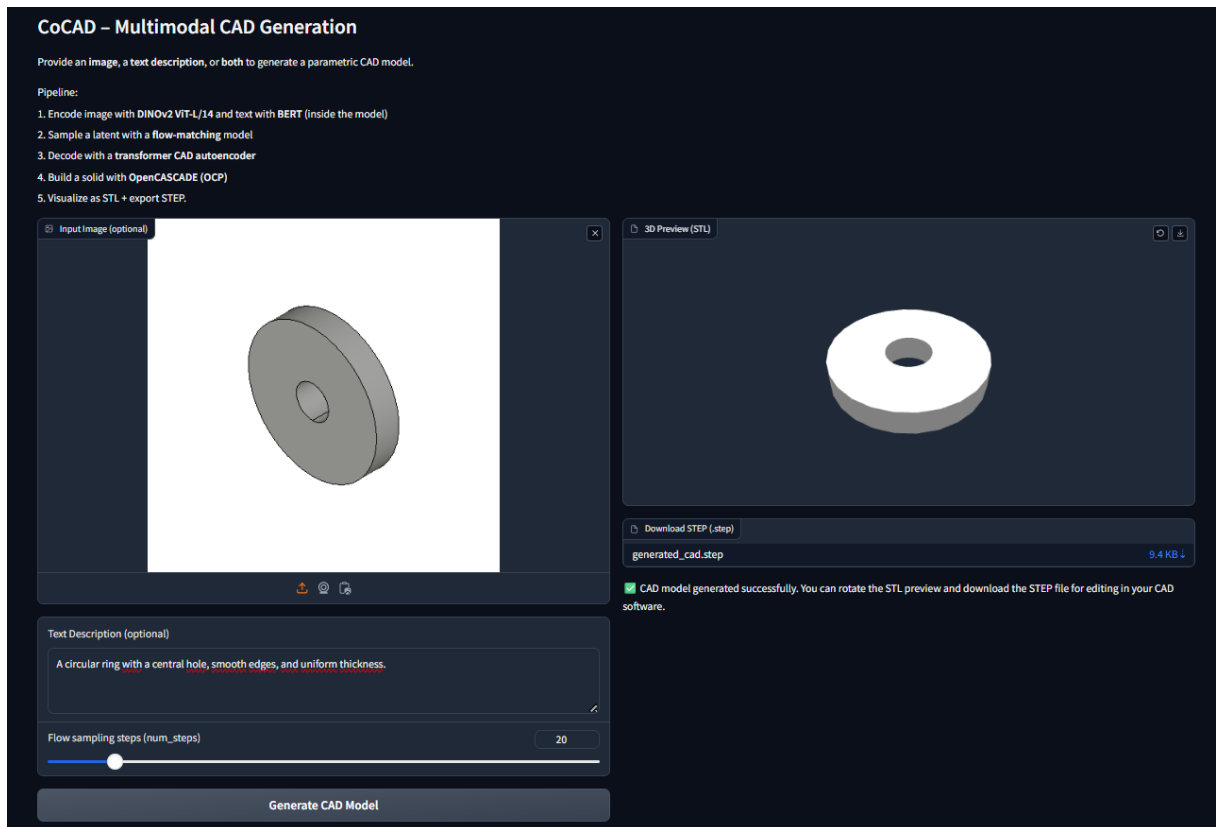


Figura 25 – Interface de geração CAD do CoCAD durante um teste funcional utilizando uma amostra do *split* de teste.

Conclusão Geral dos Testes

A análise integrada das avaliações realizadas permite concluir que o CoCAD apresenta:

- **excelente capacidade de reconstrução simbólica e geométrica** via autoencoder, com desempenho próximo ao estado da arte;
- **boa capacidade de prever latentes coerentes** no modo multimodal, demonstrando que o modelo de *flow matching* aprende parte relevante da estrutura do espaço latente CAD;
- **capacidade moderada de geração CAD completa**, especialmente fortalecida quando ambas as modalidades de entrada estão presentes;
- **operacionalidade prática comprovada pela interface**, que executou com sucesso o processo completo de geração CAD—da entrada multimodal à visualização 3D e exportação STEP—com tempos de resposta da ordem de poucos segundos.

O teste funcional da interface interativa, conduzido em ambiente de nuvem no Google Colab, reforça os resultados quantitativos ao demonstrar que o sistema está

apto para uso real. Ele valida diretamente diversos requisitos funcionais (RF01–RF07) ao confirmar o suporte a entradas multimodais, pré-processamento automático, geração paramétrica, visualização tridimensional e exportação do modelo. Também evidencia o atendimento a requisitos não funcionais centrais, como desempenho (RNF01), usabilidade (RNF05), portabilidade (RNF06) e interoperabilidade (RNF07).

Esses resultados em conjunto validam a viabilidade da abordagem proposta e indicam caminhos claros para trabalhos futuros. O CoCAD cumpre seu objetivo principal: **fornecer um modelo CAD generativo realmente multimodal**, capaz de operar de forma estável com diferentes formas de entrada e oferecendo uma interface funcional que demonstra seu potencial de aplicação prática, ainda que sem a especialização extrema típica de modelos dedicados a um único modo.

6 Considerações Finais

Este capítulo apresenta as considerações finais do projeto de formatura, destacando as conclusões alcançadas, as contribuições realizadas e as perspectivas de continuidade para futuros trabalhos.

6.1 Conclusões do Projeto de Formatura

O presente trabalho teve como objetivo desenvolver o CoCAD, um sistema multimodal capaz de gerar modelos CAD paramétricos a partir de entradas visuais e/ou textuais. A arquitetura proposta integra três componentes principais: um autoencoder de sequências CAD, um módulo de extração e fusão multimodal e um gerador latente condicionado baseado em *flow matching*. Além disso, foi desenvolvida uma interface interativa em Gradio, permitindo ao usuário explorar o processo completo de geração, desde a especificação das entradas até a visualização e exportação do modelo CAD. A eficácia dessa interface foi demonstrada experimentalmente na Seção 5.4.5, onde um teste funcional confirmou sua capacidade de acionar corretamente o pipeline completo de inferência e gerar um modelo CAD válido em poucos segundos.

Do ponto de vista dos requisitos funcionais, o sistema implementado atende às funcionalidades previstas: suporte a entrada de imagem (RF01), texto (RF02) e combinação multimodal (RF03); pré-processamento das modalidades por *encoders* dedicados (RF04); geração de sequências CAD paramétricas estruturadas no estilo *sketch-extrude* (RF05); visualização tridimensional do modelo gerado (RF06); exportação em formatos padronizados compatíveis com ferramentas de CAD (RF07); e registro de histórico de execuções e artefatos gerados (RF08). A execução da interface em Gradio reforça empiricamente o atendimento desses requisitos, ao demonstrar que o pipeline multimodal opera de forma integrada e acessível ao usuário final.

Em relação aos resultados atingidos, o autoencoder demonstrou excelente capacidade de reconstrução, atingindo níveis elevados de fidelidade simbólica e geométrica. Com $ACC_{cmd} = 98,94\%$ e $ACC_{param} = 96,15\%$, aliadas a uma mediana de Chamfer Distance de $0,886 \times 10^{-3}$ e uma taxa de modelos inválidos (IR) de 4,81%, ficou demonstrado que o espaço latente aprendido é altamente estruturado e adequado para uso em modelos generativos. Embora ainda ligeiramente abaixo dos resultados de DeepCAD e GenCAD, o desempenho é competitivo considerando-se que o autoencoder do CoCAD não foi otimizado exclusivamente para reconstrução, mas projetado como backbone de um pipeline multimodal. Esses resultados contribuem diretamente para o atendimento dos requisitos de precisão e confiabilidade (RNF03) e de validade geométrica (RNF04) no contexto de

reconstrução CAD.

O módulo de *flow matching* obteve desempenho satisfatório, especialmente no modo multimodal (imagem + texto), aproximando-se de forma significativa dos latentes produzidos pelo autoencoder. A Avaliação 2 mostrou que o modo multimodal apresenta os melhores valores de MSE e similaridade do cosseno, evidenciando que a combinação das modalidades produz um vetor de condicionamento mais informativo. A combinação entre as duas modalidades mostrou-se consistentemente superior aos modos unimodais, o que reforça a complementaridade entre informação visual e textual na tarefa de condicionamento em CAD (RF03). No entanto, ainda existe uma lacuna considerável entre a qualidade dos latentes previstos pelo *flow* e os latentes reconstruídos diretamente pelo autoencoder, indicando espaço para melhorias na modelagem generativa e no treinamento, principalmente em termos de desempenho e robustez (RNF01, RNF03).

A avaliação da geração completa de sequências CAD evidenciou que, embora o sistema seja capaz de produzir modelos plausíveis e estruturalmente coerentes, o desempenho simbólico e geométrico permanece sensivelmente inferior ao obtido pelo autoencoder, o que é esperado dada a maior complexidade da tarefa de geração condicionada em relação à reconstrução. Ainda assim, o modo multimodal superou os modos *somente imagem* e *somente texto* em todas as métricas, destacando-se como o cenário mais promissor. Além disso, os valores de Invalid Ratio de 12,53% (multimodal), 13,13% (somente imagem) e 14,43% (somente texto) indicam que o CoCAD gera mais de 85% de modelos válidos no conjunto de teste em todos os modos de operação, atendendo diretamente ao requisito não funcional **RNF03 (Precisão e Confiabilidade)**. Ao mesmo tempo, a análise geométrica dos resultados confirma que os modelos gerados permanecem adequados para visualização e edição em ferramentas paramétricas, em linha com o requisito de validade geométrica (RNF04).

Um elemento adicional relevante é o teste funcional apresentado na Seção 5.4.5, que demonstra que o sistema não apenas funciona de forma isolada em ambiente experimental, mas também opera corretamente em contexto de uso real. Esse teste reforça o atendimento aos requisitos RF01–RF07 e valida a interface como ferramenta prática de inspeção, interação e exportação do modelo gerado. Além disso, o tempo de resposta observado — da ordem de poucos segundos — confirma o atendimento ao requisito de desempenho (RNF01) em cenário de protótipo e evidencia a portabilidade da solução (RNF06), uma vez que todo o pipeline foi executado a partir de um ambiente em nuvem no Google Colab.

Alguns objetivos, contudo, não foram plenamente atingidos. A taxa de invalidez geométrica nas gerações end-to-end ainda é relativamente alta, entre 12% e 14%, quando comparada a modelos especializados da literatura. O desempenho com entradas textuais simples também foi limitado, o que se deve em grande parte à natureza breve e superficial das descrições fornecidas pelo dataset. Esses fatores indicam que, embora o requisito

de precisão e confiabilidade (RNF03) tenha sido cumprido em termos de limiar mínimo estabelecido, há margem significativa para aprimorar a qualidade geométrica e a aderência às instruções textuais.

Do ponto de vista dos requisitos não funcionais mais amplos, o CoCAD mostra-se alinhado às diretrizes estabelecidas no Capítulo 4. O uso de PyTorch, PyTorch Lightning, Hydra e um conjunto de scripts modulares favorece a manutenibilidade (RNF08), permitindo evolução incremental dos componentes do sistema. A execução bem-sucedida do treinamento em um dataset com cerca de 170 mil modelos CAD e a integração com Google Colab e Hugging Face demonstram a viabilidade da solução em cenários de dados de grande porte, atendendo ao requisito de escalabilidade (RNF02). A interface em Gradio, por sua vez, atende ao requisito de usabilidade (RNF05), ao oferecer uma forma simples de interação com o modelo para usuários não especialistas. A exportação em formatos padrão como STEP e STL, via OpenCascade/CadQuery, garante interoperabilidade com ferramentas CAD industriais (RNF07) e apoia o requisito funcional de exportação (RF07). Finalmente, o tempo de resposta obtido em ambiente de protótipo é compatível com um fluxo iterativo de design, ainda que com espaço para otimizações futuras, contribuindo para o requisito de desempenho (RNF01) e para a experiência de uso em contexto exploratório.

Apesar das limitações identificadas, o projeto cumpriu seu objetivo central: demonstrar a viabilidade de um pipeline completo de geração CAD multimodal, estabelecer uma base sólida para evolução futura e validar experimentalmente os componentes construídos. No conjunto, os resultados obtidos em reconstrução, previsão de latentes, geração end-to-end e validação da interface indicam que o CoCAD constitui uma prova de conceito robusta para sistemas de geração CAD condicionados a texto e imagem, atendendo aos principais requisitos funcionais e a uma parcela significativa dos requisitos não funcionais definidos no âmbito deste projeto.

6.2 Síntese de Atendimento aos Requisitos

Esta seção sintetiza, de forma consolidada, como os requisitos funcionais (RF01–RF08) e não funcionais (RNF01–RNF08), definidos no Capítulo 4, foram atendidos pela solução proposta. A Tabela 9 resume o atendimento aos requisitos funcionais a partir da arquitetura, das tecnologias adotadas (Seções 5.1 e 5.2), dos resultados obtidos nos experimentos (Seção 5.4) e do teste funcional da interface de geração CAD apresentado na Seção 5.4.5. A Tabela 10 apresenta a mesma síntese para os requisitos não funcionais, também considerando as evidências práticas observadas na execução da interface em ambiente de nuvem.

Tabela 9 – Síntese de atendimento aos requisitos funcionais do CoCAD.

Código	Requisito	Evidência de atendimento
RF01	Entrada de imagem	A interface em Gradio permite o envio de imagens pelo usuário, que são processadas pelo Módulo de Processamento de Imagem (Seção 5.1). O modo <i>somente imagem</i> é avaliado na Avaliação 3, com métricas reportadas na Tabela 8, demonstrando o funcionamento do pipeline <i>imagem</i> $\rightarrow$ <i>CAD</i> . Esse fluxo é ilustrado na prática no teste funcional da interface descrito na Seção 5.4.5.
RF02	Entrada de texto	A interface aceita descrições textuais, processadas pelo Módulo de Processamento de Texto. O modo <i>somente texto</i> é analisado na Avaliação 3 (Tabela 8), evidenciando geração CAD condicionada apenas a texto. No teste funcional da interface (Seção 5.4.5), foi utilizada também uma descrição textual opcional, demonstrando o suporte a esse modo de interação.
RF03	Entrada multimodal	A arquitetura inclui um Módulo de Fusão Multimodal que combina embeddings de imagem e texto. O modo <i>imagem + texto</i> apresenta os melhores resultados nas Avaliações 2 e 3 (Tabelas 7 e 8), confirmando o atendimento do requisito. O teste funcional executado na Seção 5.4.5 utiliza precisamente esse cenário multimodal, validando o comportamento integrado do sistema do ponto de vista do usuário final.
RF04	Pré-processamento das entradas	As entradas são processadas por encoders dedicados (DINOv2 e BERT), descritos nas Seções 5.1 e 5.2. A qualidade das representações é refletida nos resultados de similaridade latente (Tabela 7) e na geração final. No teste funcional da interface (Seção 5.4.5), todo o fluxo de pré-processamento é acionado automaticamente a partir das entradas fornecidas pelo usuário.

Continua na próxima página

Código	Requisito	Evidência de atendimento
RF05	Geração CAD paramétrica	O Módulo Generativo CAD (<i>flow matching</i>) combinado ao autoencoder produz sequências paramétricas do tipo <i>sketch-extrude</i> . A Avaliação 1 (Tabela 5) valida a qualidade da reconstrução e a Avaliação 3 demonstra geração CAD end-to-end. No teste funcional da interface (Seção 5.4.5), esse pipeline de geração é executado integralmente a partir de uma amostra do <i>split</i> de teste, confirmando sua operacionalidade prática.
RF06	Visualização 3D	O Visualizador 3D integrado à interface (via Gradio) permite inspeção interativa do modelo CAD gerado, conforme descrito na Seção 5.1. A Seção 5.4.5 apresenta um exemplo concreto dessa visualização tridimensional aplicada a um modelo não visto em treinamento.
RF07	Exportação CAD	O Exportador de Arquivos CAD, baseado em CadQuery/OCP, gera arquivos STEP/STL compatíveis com ferramentas industriais, conforme Seção 5.2. No teste funcional da interface (Seção 5.4.5), o arquivo STEP correspondente ao modelo gerado é disponibilizado para download, evidenciando o atendimento do requisito em um cenário de uso real.
RF08	Histórico de geração	O Repositório de Artefatos registra pesos, métricas e arquivos gerados, garantindo reprodutibilidade e rastreamento (Seção. 5.1). Adicionalmente, o uso da interface em Gradio associado aos artefatos produzidos (modelos, métricas e arquivos exportados) reforça a capacidade de documentar e reproduzir o processo de geração avaliado na Seção 5.4.5.

Tabela 10 – Síntese de atendimento aos requisitos não funcionais do CoCAD.

Código	Requisito	Evidência de atendimento
RNF01	Desempenho	A inferência completa ocorre em tempo adequado para uso exploratório na interface em Gradio. No teste funcional descrito na Seção 5.4.5, o modelo CAD é gerado em poucos segundos a partir de um ambiente em nuvem (Google Colab), demonstrando que o sistema é utilizável de forma interativa e alinhado a ciclos iterativos de design, ainda que com potencial para otimização.
RNF02	Escalabilidade	O treinamento foi realizado com aproximadamente 170 mil modelos CAD. O uso de PyTorch Lightning, Hydra e Colab/GPU permite operação em ambientes distribuídos e datasets extensos. Essa infraestrutura sustenta também o teste funcional da interface (Seção 5.4.5), executado sobre o modelo treinado nesse cenário de grande porte.
RNF03	Precisão e confiabilidade	A Avaliação 1 mostra alta exatidão na reconstrução (98,94% de ACC_{cmd} e IR de 4,81%). Na Avaliação 3, todos os modos superam 85% de modelos válidos, cumprindo o requisito. O teste funcional da interface, ao utilizar uma amostra do <i>split</i> de teste e gerar um modelo geométrico adequado para visualização e exportação (Seção 5.4.5), reforça empiricamente essa confiabilidade em um contexto de uso interativo.
RNF04	Validade geométrica	A maioria dos modelos gerados apresenta geometria adequada (Chamfer Distance e IR em Tabelas 5 e 8). CadQuery/OCP garante exportação somente de sólidos válidos. No teste funcional da interface (Seção 5.4.5), o modelo gerado pôde ser exportado em STEP e visualizado em STL, confirmando na prática a validade geométrica do sólido produzido.

Continua na próxima página

Código	Requisito	Evidência de atendimento
RNF05	Usabilidade	A interface em Gradio permite uso intuitivo por não especialistas. Foi empregada em toda a avaliação qualitativa e, em particular, no teste funcional descrito na Seção 5.4.5, demonstrando simplicidade, clareza dos campos de entrada e facilidade de interpretação dos resultados.
RNF06	Portabilidade	A solução roda em Python, PyTorch e navegador, podendo ser executada localmente ou na nuvem (Colab), atendendo ao requisito. A execução do teste funcional em um ambiente Google Colab (Seção 5.4.5) reforça essa portabilidade ao comprovar o funcionamento completo do pipeline em infraestrutura de nuvem.
RNF07	Interoperabilidade	Exportação em STEP/STL via OCP garante compatibilidade com plataformas CAD comerciais, validada nos testes. No teste funcional da interface (Seção 5.4.5), o arquivo STEP gerado é disponibilizado para download e pode ser importado em ferramentas CAD industriais, exemplificando esse requisito em um cenário concreto.
RNF08	Manutenibilidade	A arquitetura modular em camadas, o uso de Lightning/Hydra e scripts segmentados permitem evolução independente de componentes, atendendo plenamente ao requisito. A forma como a interface consome os modelos treinados e encapsula o pipeline de geração (Seção 5.4.5) também evidencia essa modularidade, ao isolar a lógica de inferência do código de apresentação.

No conjunto, as Tabelas 9 e 10, complementadas pelo teste funcional da interface apresentado na Seção 5.4.5, demonstram que o CoCAD atende aos principais requisitos funcionais e a uma parcela expressiva dos requisitos não funcionais definidos no escopo deste projeto, ainda que existam oportunidades claras de aprimoramento, especialmente em desempenho, qualidade geométrica e condicionamento textual.

6.3 Contribuições

O trabalho realizado apresentou contribuições relevantes tanto no âmbito técnico quanto conceitual. A principal delas foi o desenvolvimento da arquitetura CoCAD e de todo o pipeline de processamento, treinamento e inferência. O autoencoder de sequências CAD foi adaptado e treinado para produzir um espaço latente estável, informativo e adequado para modelagem generativa, atendendo às demandas de reconstrução precisa (RF05, RNF03, RNF04). O módulo de condicionamento multimodal foi projetado com adaptadores leves e um transformador de fusão, permitindo integração eficiente entre texto e imagem (RF01, RF02, RF03). O modelo de *flow matching* foi implementado para operar nesse espaço latente, incorporando técnicas como normalização de latentes, *modality dropout* e amostragem controlada, contribuindo para o atendimento dos requisitos de desempenho e escalabilidade (RNF01, RNF02).

Outra contribuição significativa foi o desenvolvimento de um conjunto completo de scripts para pré-processamento, treinamento, teste e geração, garantindo reprodutibilidade e modularidade. Esses scripts permitem executar todo o pipeline de forma segmentada e controlada, desde a extração de *features* de imagem e texto até a geração de arquivos STL e STEP para visualização e manipulação CAD. Essa infraestrutura dá suporte direto ao requisito de histórico e rastreabilidade (RF08) e reforça a manutenibilidade (RNF08), ao organizar o projeto em componentes bem definidos.

A interface em Gradio constitui uma contribuição prática essencial. Ela integra modelos complexos em uma ferramenta simples e acessível, permitindo a qualquer usuário fornecer entradas e visualizar o sólido CAD gerado, além de baixar o modelo final (RF01, RF02, RF03, RF06, RF07). Essa interface não apenas facilita demonstrações e avaliações qualitativas, como também aproxima o sistema de aplicações reais, alinhando-se ao requisito de usabilidade (RNF05) e à portabilidade (RNF06), por funcionar em ambiente web.

Por fim, destaca-se a organização e filtragem do dataset multimodal, a adaptação do formato de dados CAD para uso eficiente com Transformers e a formulação de métricas de avaliação alinhadas com o estado da arte. Todos esses elementos foram desenvolvidos pela equipe e formam um conjunto coerente de contribuições técnicas e metodológicas que dão suporte direto às Avaliações 1, 2 e 3, permitindo mensurar com precisão o grau de atendimento aos requisitos de precisão, validade geométrica e confiabilidade (RNF03, RNF04).

6.4 Perspectivas de Continuidade

O trabalho desenvolvido abre uma série de caminhos promissores para pesquisas futuras e amadurecimento da solução. Uma das direções mais claras consiste no atri-

moramento do modelo de *flow matching*. Arquiteturas mais profundas, técnicas recentes como *rectified flows*, *consistent flow matching* ou mesmo modelos híbridos de difusão podem melhorar substancialmente a qualidade dos latentes gerados, reduzindo a lacuna observada entre os latentes previstos e os latentes reais do autoencoder e, por consequência, aprimorando o atendimento aos requisitos de precisão e confiabilidade (RNF03).

Outra possibilidade importante é o enriquecimento da modalidade textual. O uso de descrições mais detalhadas ou embeddings textuais extraídos de modelos de linguagem de grande porte (LLMs) pode ampliar significativamente a capacidade do sistema de compreender instruções complexas e detalhadas, melhorando o condicionamento e a precisão da geração, especialmente no modo *somente texto*. Essa linha de evolução tem potencial para reforçar ainda mais os requisitos funcionais relacionados a entradas textuais e multimodais (RF02, RF03) e os requisitos de usabilidade e desempenho percebido (RNF05, RNF01).

Também há espaço para intervenções no nível geométrico. Métodos de correção topológica automática, restrições geométricas incorporadas ao decodificador ou regularizações no espaço latente do autoencoder poderiam reduzir a taxa de modelos inválidos e aumentar a robustez do pipeline, contribuindo para um atendimento ainda mais rigoroso dos requisitos de validade geométrica (RNF04) e confiabilidade (RNF03).

A integração com outras representações 3D, como campos implícitos, SDFs neurais ou modelos de difusão 3D, abre caminho para sistemas CAD híbridos, combinando precisão paramétrica e flexibilidade geométrica. Do ponto de vista de usabilidade, a interface pode evoluir para permitir edição incremental, condicionamento por esboços 2D ou seleção de estilos geométricos, aprofundando o atendimento ao requisito de usabilidade (RNF05) e aproximando o sistema de fluxos de trabalho reais em engenharia.

Por fim, um passo importante seria validar o sistema com usuários reais, como engenheiros, designers e profissionais de modelagem. A interação com especialistas permitiria identificar limitações práticas, aumentar a confiabilidade da ferramenta e direcionar esforços para funcionalidades de maior impacto em aplicações industriais, refinando assim a interpretação dos requisitos funcionais (RF01–RF08) sob a ótica de uso real.

Em conjunto, essas perspectivas indicam que o CoCAD constitui uma fundação sólida para futuras pesquisas em geração multimodal de CAD e pode evoluir tanto como ferramenta de experimentação quanto como solução aplicada em ambientes de engenharia assistida por computador, aprofundando progressivamente o atendimento aos requisitos funcionais e não funcionais definidos neste projeto.

Referências

ABADI, M. e. a. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. 2015. Disponível em: <<https://www.tensorflow.org>>. Citado na página 59.

ABID, A. e. a. Gradio: Hassle-free sharing and testing of ml models in the wild. In: *ICML Workshop on Human in the Loop Learning*. [S.l.: s.n.], 2019. Citado na página 60.

ALAM, M. F.; AHMED, F. *GenCAD: Image-Conditioned Computer-Aided Design Generation with Transformer-Based Contrastive Representation and Diffusion Priors*. 2025. Disponível em: <<https://arxiv.org/abs/2409.16294>>. Citado 8 vezes nas páginas 14, 15, 18, 21, 26, 33, 78 e 80.

AUTODESK. *State of Design and Make Report*. 2023. Industry Report. Disponível em: <<https://www.autodesk.com/hk/insights/research/state-of-design-and-make>>. Citado na página 16.

Autodesk, Inc. *AutoCAD – 2D and 3D CAD software*. 2025. <<https://www.autodesk.com/br/products/autocad/overview>>. Acesso em: 2025-12-08. Citado na página 41.

Autodesk, Inc. *Fusion 360 – CAD, CAM, CAE and PCB 3D modeling platform*. 2025. <<https://www.autodesk.com/br/products/fusion-360/overview>>. Acesso em: 2025-12-08. Citado na página 41.

BROWN, T. B. et al. *Language Models are Few-Shot Learners*. 2020. Disponível em: <<https://arxiv.org/abs/2005.14165>>. Citado 2 vezes nas páginas 13 e 30.

CHEN, C. et al. *CADCrafter: Generating Computer-Aided Design Models from Unconstrained Images*. 2025. Disponível em: <<https://arxiv.org/abs/2504.04753>>. Citado 7 vezes nas páginas 14, 15, 21, 27, 33, 78 e 83.

COMPANY, M. . *The Economic Potential of Generative AI — The Next Productivity Frontier*. 2023. Report. Disponível em: <<https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier>>. Citado na página 16.

CONTRIBUTORS, T. *TorchVision: PyTorch Computer Vision Library*. 2016. Disponível em: <<https://pytorch.org/vision>>. Citado na página 61.

CRETU, L. G. *Designing Enterprise Architecture Frameworks: Integrating Business Processes with IT Infrastructure*. Waretown, NJ: Apple Academic Press, 2014. ISBN 9781771880077. Citado na página 35.

Dassault Systèmes. *CATIA – 3D CAD, CAE and PLM software*. 2025. <<https://www.3ds.com/products/catia>>. Acesso em: 2025-12-08. Citado na página 41.

Dassault Systèmes. *SOLIDWORKS – 3D CAD software*. 2025. <<https://www.solidworks.com/pt-br>>. Acesso em: 2025-12-08. Citado 3 vezes nas páginas 7, 13 e 41.

- DEVLIN, J. et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. Disponível em: <<https://arxiv.org/abs/1810.04805>>. Citado na página 31.
- DEVLIN, J. e. a. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018. Citado 3 vezes nas páginas 58, 66 e 71.
- DOHYEON. *Deep Learning e CAD: Deep Learning com Modelos Sólidos*. 2025. Acessado em: 2025-12-07. Disponível em: <<https://dohyeon.tistory.com/93>>. Citado na página 18.
- FALCON, W. *PyTorch Lightning*. 2019. Disponível em: <<https://github.com/Lightning-AI/lightning>>. Citado na página 57.
- GKIOXARI, G.; MALIK, J.; JOHNSON, J. *Mesh R-CNN*. 2020. Disponível em: <<https://arxiv.org/abs/1906.02739>>. Citado na página 18.
- GOOGLE. *Google Colaboratory*. 2024. <<https://colab.research.google.com>>. Citado na página 59.
- GROUP, B. C. *Gen AI's Role in the Factory of the Future*. 2023. Industry Publication. Disponível em: <<https://www.bcg.com/publications/2023/gen-ai-role-in-factory-of-future>>. Citado na página 16.
- HARRIS, C. R. e. a. Array programming with numpy. *Nature*, 2020. Citado na página 61.
- HO, J.; JAIN, A.; ABBEEL, P. *Denoising Diffusion Probabilistic Models*. 2020. Disponível em: <<https://arxiv.org/abs/2006.11239>>. Citado na página 31.
- HUNTER, J. D. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 2007. Citado na página 61.
- ISO/IEC. *ISO/IEC 10746-1: Information Technology — Open Distributed Processing — Reference Model: Overview*. 1998. Citado 2 vezes nas páginas 35 e 36.
- ISO/IEC. *ISO/IEC 10746-2: Information Technology — Open Distributed Processing — Reference Model: Foundations*. 2009. Citado 2 vezes nas páginas 35 e 36.
- ISO/IEC. *ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Geneva, Switzerland: International Organization for Standardization, 2011. Standard. Citado 3 vezes nas páginas 37, 42 e 49.
- ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering*. Geneva, Switzerland: International Organization for Standardization, 2018. Standard. Citado na página 42.
- KHAN, M. S. et al. *Text2CAD Dataset*. 2024. <<https://huggingface.co/datasets/SadilKhan/Text2CAD>>. Accessed: 2025-01-10. Citado na página 63.
- KHAN, M. S. et al. *Text2CAD: Generating Sequential CAD Models from Beginner-to-Expert Level Text Prompts*. 2024. Disponível em: <<https://arxiv.org/abs/2409.17106>>. Citado 8 vezes nas páginas 14, 15, 21, 28, 29, 63, 78 e 83.

- LIPMAN, Y. et al. *Flow Matching for Generative Modeling*. 2023. Disponível em: <<https://arxiv.org/abs/2210.02747>>. Citado 2 vezes nas páginas 16 e 31.
- LIU, X.; GONG, C.; LIU, Q. *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. 2022. Disponível em: <<https://arxiv.org/abs/2209.03003>>. Citado 2 vezes nas páginas 16 e 31.
- MCKINNEY, W. Data structures for statistical computing in python. In: *Proceedings of the Python in Science Conference*. [S.l.: s.n.], 2010. Citado na página 61.
- OQUAB, M. et al. *DINOv2: Learning Robust Visual Features without Supervision*. 2024. Disponível em: <<https://arxiv.org/abs/2304.07193>>. Citado 3 vezes nas páginas 30, 66 e 70.
- OQUAB, M. e. a. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023. Citado 3 vezes nas páginas 58, 66 e 70.
- PASZKE, A. e. a. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 2019. Citado na página 57.
- PEREZ, E. et al. *FiLM: Visual Reasoning with a General Conditioning Layer*. 2017. Disponível em: <<https://arxiv.org/abs/1709.07871>>. Citado na página 33.
- RADFORD, A. et al. *Improving Language Understanding by Generative Pre-Training*. 2018. Disponível em: <https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf>. Citado 2 vezes nas páginas 13 e 30.
- RAMESH, A. et al. *Hierarchical Text-Conditional Image Generation with CLIP Latents*. 2022. Disponível em: <<https://arxiv.org/abs/2204.06125>>. Citado na página 14.
- ROMBACH, R. et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. Disponível em: <<https://arxiv.org/abs/2112.10752>>. Citado 2 vezes nas páginas 14 e 31.
- ROSSUM, G. van; DRAKE, F. L. *Python 3 Programming Language*. 2009. Disponível em: <<https://www.python.org>>. Citado na página 60.
- SAS, O. *OpenCascade Technology*. 2020. Disponível em: <<https://www.opencascade.com>>. Citado na página 60.
- Siemens AG. *Industry 4.0 — Vision to Reality: A vision into the future of manufacturing*. 2015. White paper. Disponível em: <<https://www.nottingham.ac.uk/ifam/documents/dtmi2015/1-1-a-vision-into-the-future-of-manufacturing.pdf>>. Citado na página 16.
- SITZMANN, V. et al. *DeepVoxels: Learning Persistent 3D Feature Embeddings*. 2019. Disponível em: <<https://arxiv.org/abs/1812.01024>>. Citado na página 18.
- SOMMERVILLE, I. *Software Engineering*. 9. ed. Boston: Addison-Wesley, 2011. Citado 2 vezes nas páginas 36 e 48.
- VASWANI, A. et al. *Attention Is All You Need*. 2023. Disponível em: <<https://arxiv.org/abs/1706.03762>>. Citado na página 30.
- VISIATIV. *What's new in Collaborative Designer for SolidWorks 2024*. 2024. Acessado em: 2025-12-09. Disponível em: <<https://www.visiativ.de/solidworks-blog/whats-new-in-collaborative-designer-for-solidworks-2024/>>. Citado na página 13.

- WILLIS, K. D. D. *Deep Learning with Solid Models*. 2025. Acessado em: 2025-12-07. Disponível em: <<https://www.karlddwillis.com/deep-learning-with-solid-models/>>. Citado na página 18.
- WOLF, T. e. a. Transformers: State-of-the-art natural language processing. *EMNLP*, 2020. Citado na página 59.
- WU, R.; XIAO, C.; ZHENG, C. *DeepCAD: A Deep Generative Network for Computer-Aided Design Models*. 2021. Disponível em: <<https://arxiv.org/abs/2105.09492>>. Citado 10 vezes nas páginas 14, 15, 20, 21, 22, 23, 63, 65, 78 e 80.
- XU, X. et al. *Hierarchical Neural Coding for Controllable CAD Model Generation*. 2023. Disponível em: <<https://arxiv.org/abs/2307.00149>>. Citado 2 vezes nas páginas 15 e 25.
- XU, X. et al. *BrepGen: A B-rep Generative Diffusion Model with Structured Latent Geometry*. 2024. Disponível em: <<https://arxiv.org/abs/2401.15563>>. Citado na página 18.
- XU, X. et al. *SkeXGen: Autoregressive Generation of CAD Construction Sequences with Disentangled Codebooks*. 2022. Disponível em: <<https://arxiv.org/abs/2207.04632>>. Citado 3 vezes nas páginas 15, 23 e 24.
- YADAN, O. *Hydra - A framework for elegantly configuring complex applications*. 2019. Disponível em: <<https://github.com/facebookresearch/hydra>>. Citado na página 58.
- YANG, G. et al. *PointFlow: 3D Point Cloud Generation with Continuous Normalizing Flows*. 2019. Disponível em: <<https://arxiv.org/abs/1906.12320>>. Citado na página 18.